A simpler algorithm for integer multiplication below $n \log n$, with a formally verified proof*

Piaoyang Cui†
Google DeepMind

Yiming Liu†
Google DeepMind

David P. Woodruff†
Google Research
Carnegie Mellon University

25 October 2026

Abstract

We give a simpler algorithm that multiplies two n-bit integers in $O\big(n(\log n)^{1-\kappa}\big)$ bit operations on a multitape Turing machine for an explicit constant $\kappa > 0$, and we formally verify this bound in Lean 4. The only previously known algorithm below $n \log n$ obtains its advantage from two small fixed linear networks but runs them inside the full architecture of the Harvey–van der Hoeven $n \log n$ algorithm. We keep the two networks and replace everything around them by a construction on a single power-of-two grid: one Fourier transform, preceded by a smoothing step that is later undone to working precision. The result is an algorithm whose every step is implemented in exact arithmetic and tested bit for bit, and whose complexity bound, including the certificates of the two networks, is verified in Lean 4 from Lean's standard axioms alone. As a by-product, the exponent κ improves from 2^{-182} to 2^{-46}.

*This work was carried out with substantial assistance from AI agents (primarily Gemini 4), including proof development, auditing, and the Lean 4 formalization; see Section 1.2.

†Authors are listed in alphabetical order.

1 Introduction

Multiplying two n-bit integers is a basic problem in the theory of computation. In the standard deterministic multitape Turing machine model, a fixed finite-state control operates a fixed number of one-dimensional tapes, and the running time counts every head step. In 1971, Schönhage and Strassen gave an $O(n \log n \log\log n)$ multiplication algorithm and conjectured that $n \log n$ steps are required up to constant factors. Harvey and van der Hoeven reached the $O(n \log n)$ bound in 2021. A 2026 preprint then gave an algorithm below $n \log n$, contradicting the conjecture; its construction is intricate, and simplifying it is the subject of this paper.

After Schönhage and Strassen [SS71], Fürer [Für07, Für09] replaced the $\log\log n$ factor by $2^{O(\log^* n)}$, with a modular variant by De, Kurur, Saha, and Saptharishi [DKSS13]. Harvey, van der Hoeven, and Lecerf [HvdHL16] sharpened the base of that exponent, Harvey and van der Hoeven [HvdH21] achieved $O(n \log n)$, and a recent preprint [OAI26] obtained $O(n(\log n)^{1-\kappa})$ with $\kappa = 2^{-182}$. That power saving comes from two small fixed linear networks, yet the algorithm runs them inside the full architecture of Harvey and van der Hoeven. Specifically, it splits the input across several prime-size grids, interpolates onto power-of-two grids, and computes the central transform through polynomial arithmetic. All of that surrounding machinery exists only to connect the two networks to integer multiplication. It accounts for most of the algorithmic complexity and most of the loss in κ; Section 2.2 gives the precise accounting.

In this paper, we keep the two finite linear networks of [OAI26] and replace everything around them by a construction on a single power-of-two grid. The construction is simple enough that every step can be implemented in exact arithmetic and tested bit for bit, and the whole proof verified formally in Lean 4. The $O(n \log n)$ algorithm of Harvey and van der Hoeven takes dimension 1729 in its proof and, to our knowledge, has never been executed.

First, our algorithm slices both n-bit inputs into short binary digits and packs them into one array of complex numbers. Second, it arranges that array on one multidimensional power-of-two grid and smooths each axis after the first, called a *spread axis*, using one data-independent localized kernel per spread axis. Third, it runs one discrete Fourier transform directly on that grid using the two networks, and undoes the smoothing to working precision by one two-term cyclic pass per spread axis. Fourth, it squares the transformed values pointwise, runs the same three-step transform pass once more, rounds to integers, and propagates carries to output the exact $2n$-bit product. Algorithm 1 is stated in full in Section 2.1. As a by-product, the exponent improves to $\kappa = 2^{-46}$.

1.1 Main results

Our primary algorithmic theorem states the $O(n(\log n)^{1-2^{-46}})$ running-time bound and records the layers that the outer reduction removes.

Theorem 1. *Algorithm 1 (Section 2.1), implemented on a deterministic multitape Turing machine with a fixed finite alphabet and a fixed finite number of tapes, multiplies two n-bit integers exactly in worst-case time*

$$O(n(\log n)^{1-2^{-46}}).$$

Structure. *The outer algorithm uses only power-of-two DFTs on one grid, one fixed convolution kernel per spread axis, and one two-term cyclic recurrence per spread axis; it uses no Chinese remainder map, prime transform length, coprime-length resampling, Fourier-grid expansion, Bluestein chirp, or polynomial-ring convolution. Ordinary products of $O((\log n)^2)$ bits (and inputs below a fixed threshold n_*) are delegated to any independent multiplier with $\mathsf{M}(N) = N(\log N)^{O(1)}$ (for instance that of Schönhage and Strassen [SS71]), with no recursive call to Algorithm 1 itself.*

Our second main theorem gives a complete formal proof of this complexity bound in the Lean 4 theorem prover without importing any external network bound.

Theorem 2 (Formally verified bound)**.** *The $O(n(\log n)^{1-2^{-46}})$ multiplication bound of Theorem 1 is formally proved in Lean 4 [dMU21, mC20] in Mathlib's finite-alphabet multi-stack Turing machine model over $\mathbb{N}$ (statement `SubNLogN.Challenge.IntMulSubNLogN` in [Lean26], available in our github repository; Section 8), depending only on Lean's three standard logical axioms (`propext`, `Classical.choice`, `Quot.sound`) and assuming no result of [OAI26].*

1.2 Executability, formal verification, and use of AI tools

Because asymptotic multiplication proofs combine rounding budgets with sequential tape invariants, we pair the written proof with an exact-arithmetic reference package and a complete formalization.

Executability. Every step of Algorithm 1 is implemented in exact arithmetic in Python 3.13, with no floating point on any data path of the multiplier or certifier [Code26]. Across 2,067 tests, 1,832 pass and one is deferred in the fast suite while 234 pass in the slow suite, checking every returned product bit for bit. All 71 transform runs with $d \in \{2, \ldots, 5\}$ and $R \leq 1024$ stay within our proved bounds. With its inner-multiplier hierarchy and no stub, the pipeline ran end to end on 70- and 96-bit inputs under a test schedule forcing the sub-$n \log n$ route with small substitute networks in place of the $h = 100$ networks. The unforced schedule selects the two-dimensional multiplier at every feasible size.

The $h = 100$ network certificates, including the 140-digit guard constant, are recomputed in exact arithmetic; the $h = 100$ networks themselves are certified, not executed. The package is a reference for bit-exact checking, not a fast multiplier, and no running time is measured or claimed. Table 12 in Appendix E summarizes the package in our github repository; our companion paper [MS] develops and formally verifies the two-dimensional instance. As the sub-$n \log n$ regime begins only at astronomical sizes, no practical speedup is claimed.

Formal verification. To remove doubt about hidden hypotheses or rounding-budget gaps, we formalized the $\kappa = 2^{-46}$ complexity bound in Lean 4, as detailed in Section 8. To our knowledge, this is the first formally verified proof of an $o(n \log n)$ integer-multiplication bound, as well as the first independent formal verification of the $h = 100$ finite-network certificates of [OAI26]. Building on Mathlib, the development spans roughly 299,000 lines across 1,062 files for the sub-$n \log n$ proof. It also includes 949 files and about 185,000 lines formalizing the two-dimensional $O(n \log n)$ multiplier of our companion paper [MS], which instantiates the delegated inner multiplier M. That inner formalization has headline theorem `IntMulTM.intMul_TM2_nlogn` and is kernel-checked within the same archive, whereas that companion paper distributes a separate Lean development with headline `IntMul.tmIntMul_evalsToInTime_n_log_n`.

Our formal development [Lean26] exports one closed theorem. We replayed it through the Lean kernel with `leanchecker` and checked it again with two independent external checkers, `comparator` and `nanoda`. What remains to be trusted is small: Lean's kernel, the Mathlib definitions named in the statement, the three standard axioms, and the 80-line statement file `SubNLogN/Challenge.lean`. Its core definitions are quoted in Section 8.

Remarks. Two remarks clarify how Theorems 1 and 2 relate to prior work and to the exponent.

- *What is removed and what is borrowed.* Our proof uses no result from [HvdH21] and discards its prime-size grids and polynomial-ring layers. The written proof of Theorem 1 borrows from [OAI26] only two self-contained black boxes: its linear-time tape-stream operations and its two fixed

linear networks at ground-set parameter $h = 100$ with their finite algebraic certificates, stated in Section 3.4. The Lean 4 proof of Theorem 2 assumes neither black box; it constructs and verifies the tape operations, both $h = 100$ networks, and their certificates from scratch.

- *The exponent is a by-product.* Simplifying onto one power-of-two grid lets us present one direct route, with the larger κ emerging as a by-product. Because the half-sum and half-difference Fourier step also permutes address-bit positions bit-exactly, removing the layout bottleneck raises κ to 2^{-56} under the constants in [OAI26]. Retuning with the certified constants of Appendix A reaches $\kappa = 2^{-46}$; Table 2 in Section 2.5 attributes the gain over 2^{-182}. These exponents are purely asymptotic, as the sub-$n \log n$ regime begins only at astronomical sizes; see Sections 2 and 9.

Use of AI tools. This work was developed with substantial assistance from AI agents (primarily Gemini 4), working under the direction of the authors. That assistance spanned the algorithm, the proofs in this paper, the exact-arithmetic reference implementation, and the Lean 4 formalization. The authors have checked the written proofs in this paper. No author has checked the formal Lean proof line by line. Rather, the correctness of the formal development rests on kernel verification by Lean 4 (replayed through the Lean kernel with `leanchecker` and independently checked with `comparator` and `nanoda`), and the authors have checked the formal statement in `SubNLogN/Challenge.lean` and its correspondence to Theorem 1. Agreement among AI systems was used only as a working heuristic during exploration, never as mathematical evidence: every claim in this paper is established by a complete written proof, an explicit citation to the literature, or a kernel-checked formal theorem.

1.3 Organization

Section 2 explains how the algorithm works; the full statements and proofs begin in Section 3. Section 3 fixes notation, the Turing machine and fixed-point conventions, discrete Fourier transform identities, and the two black-box network facts. Section 4 builds the multidimensional bridge that smooths each spread axis, undoes that smoothing to working precision, and fixes the precision budget. A reader who accepts that precision contract may skip its error proofs on a first reading and proceed to Section 5, which uses pairwise half-sum and half-difference steps to permute address bits bit-exactly.

Section 6 uses those address permutations to compute the multidimensional Fourier transform directly over the complex numbers. Section 7 combines the bridge and transform costs, fixes the schedule exponents, and proves Theorem 1. Section 8 describes the Lean 4 formalization of Theorem 2; its formal statement, trust boundary, and kernel-replay audit can be read independently of Sections 4–7. Section 9 examines where the exponent comes from, why activation sizes are astronomical, what became simpler, and what remains open.

The appendices hold supporting material that may be skipped on a first reading. Appendix A rederives the address-interchange and parallel Fourier-step bounds with general parameters and certifies the $h = 100$ network exponents. Appendix B proves two impossibility bounds for alternative smoothing kernels together with series bounds for precomputed tables. Appendix C analyzes the alternative polynomial-ring route and extra parameter rows for comparison. Appendix D records the notation map to prior work, and Appendix E summarizes the exact-arithmetic Python implementation.

2 Technical overview

This section explains how Algorithm 1 works in outline, with forward pointers to Sections 3–8. It also shows how retuning the schedule, removing the layout bottleneck, and certifying sharper

network constants combine to raise its saving exponent from 2^{-182} to 2^{-46}. Every known route below $n \log n$, like every Fourier-based multiplier since Schönhage and Strassen, evaluates a long discrete Fourier transform on an array of bit volume proportional to n without making $\log n$ full scans. The preprint [OAI26] breaks $n \log n$ by applying two fixed linear networks across many dimensions at once, each performing one basic tape operation in slightly less than linear time; the exponent of that saving is called the *network deficit*. To feed those networks, however, [OAI26] first splits the indices across several prime lengths with a *Chinese remainder map*, then interpolates each prime-length axis onto a power-of-two grid (*Gaussian resampling*), and finally rewrites the central transform through Bluestein's quadratic-phase trick as convolutions of blocks treated as polynomials.

We no longer route the input through prime-length grids, chirp factors, or polynomial rings; instead we work from start to finish on one power-of-two grid. First, we absorb cross-axis twiddles by smoothing each spread axis with one data-independent phase-modulated kernel, run one direct complex transform via *butterfly layers*, which combine pairs of entries whose addresses differ in one bit into half-sums and half-differences, and undo the smoothing to working precision. Second, three-step algebraic identities let the butterfly layer itself permute address-bit positions bit-exactly, removing the block-swapping layout pass that forced a quadratic loss in the network deficit in [OAI26]. With that layout bottleneck gone, the network deficit enters the saving exponent linearly, raising κ to 2^{-56} under printed network constants and to 2^{-46} with our certified constants. A first-time reader may read Section 2.1 for the complete recipe and skim Sections 2.2–2.5 as needed.

2.1 The algorithm in five steps

Algorithm 1 reduces n-bit integer multiplication to two approximate discrete Fourier transforms of power-of-two length $R = 2^k$, evaluated on one d-dimensional grid of balanced power-of-two side lengths $t_1 \times \cdots \times t_d$.

The simplest case: two axes. Consider first $d = 2$, where $R = t_1 r$ factors into two powers of two and a length-R vector forms a $t_1 \times r$ table with row index $j_1 \in [t_1]$ and column index $j_2 \in [r]$. In the Cooley–Tukey decomposition of F_R, row j_1 carries a cross-axis twiddle factor that acts as a *fractional shift* of its samples by j_1/t_1 of a grid cell along the row. Rather than multiplying by twiddle factors between passes, we realize that shift on the same $t_1 \times r$ grid by convolving row j_1 cyclically at offset j_1/t_1 with a data-independent smooth kernel modulated by a negative half-bin phase.

Applying a two-dimensional power-of-two Fourier transform to the smoothed table leaves, in every column, a linear combination of two adjacent frequency entries of $F_R x$ plus an exponentially small Gaussian tail. Because the half-bin phase keeps the two dominant weights well separated in every column, solving one two-term cyclic recurrence down each column undoes the smoothing to working precision and approximates $F_R x$; see Figure 1. Passing to general $d = \lfloor k^\varepsilon \rfloor \geq 2$ changes only two things. First, we smooth the spread axes $a = d, \ldots, 2$ in descending order and solve the two-term cyclic systems in ascending order $a = 2, \ldots, d$ because the multi-axis smoothing operators do not commute. Second, across the d-axis grid, the two fixed linear networks of [OAI26] accelerate axis swaps, bit-exact address permutations, and butterfly rounds in sub-linear time per axis.

The five named procedures. Below a fixed cutoff n_*, the computation delegates directly to an independent multiplier M; for $n \geq n_*$, it runs five named procedures on one power-of-two grid, specified in Algorithm 1 and shown in Figure 2. First, Pack cuts both n-bit inputs into radix-2^b digits and packs them into the real and imaginary parts of one length-R complex array. Second, inside Bridge, Spread scales onto the fixed-point grid and smooths the spread axes d down to 2 in place,

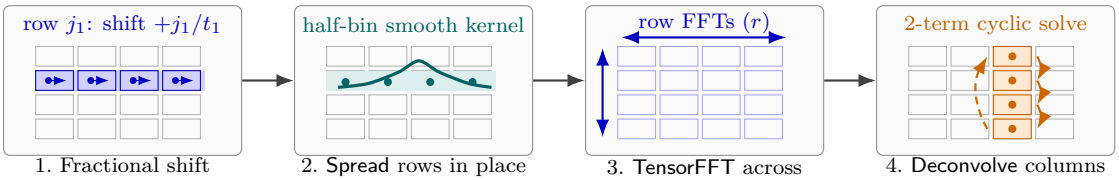


Figure 1: How the bridge approximates a one-dimensional Fourier transform in the simplest case of two axes ($d = 2$, $R = t_1 r$). Each row j_1 carries a fractional shift j_1/t_1, which Spread realizes on the same $t_1 \times r$ power-of-two grid at offset j_1/t_1 with a data-independent phase-modulated kernel. After a two-dimensional transform (TensorFFT), Deconvolve solves one two-term cyclic recurrence down each column to undo the smoothing to working precision.

using one data-independent negative half-bin kernel per spread axis to absorb cross-axis twiddles without the midpoint (Nyquist) cancellation that would make the column recurrence singular.

Third, TensorFFT evaluates the d-dimensional complex Fourier transform directly on that grid via simultaneous butterfly rounds and bit-exact *positional permutations*, which reorder address-bit positions through 2×2 *Walsh* half-sum and half-difference identities. Fourth, Deconvolve solves one two-term cyclic recurrence along each axis from 2 up to d to undo the smoothing to working precision, completing Bridge with an approximation to $F_R x$. Fifth, after conjugated pointwise squaring and a second Bridge call, grouped with recovery as Step 5 in Figure 2, Recover rounds the scaled imaginary parts via (8) and propagates carries for the exact $2n$-bit product. In [Code26], under `subnlogn/`, Pack and Recover run in `multiply.py` and `bits.py`, Bridge, Spread, and Deconvolve are `bridge_dft`, `spread_axis`, and `deconv_axis` in `bridge/`, and TensorFFT is `tensor_fft_walsh` in `walsh/fft.py`.

Algorithm 1 Integer multiplication on one power-of-two grid

Input: two n-bit integers $0 \le A, B < 2^n$. **Output:** the $2n$-bit product AB.
Parameters (Section 7.1)**:** $b = \lfloor \log_2 n \rfloor$, $N_{\text{dig}} = \lceil n/b \rceil$, $R = 2^k \ge 2N_{\text{dig}}$, balanced shape $\mathbf{t} = (t_1, \ldots, t_d)$ with $t_1 \cdots t_d = R$, $d = \lfloor k^\varepsilon \rfloor$, $t_d = r = 2^\ell$, $t_i \in \{r/2, r\}$; precisions p, ω, P and $q = P + 2$ from (40) and (24).

if $n < n_*$ **then return** $\mathsf{M}(A, B)$ ▷ cutoff (46); M any $N(\log N)^{O(1)}$ multiplier
$x \leftarrow \mathsf{Pack}(A, B)$ ▷ radix-2^b digits as real and imaginary parts of one length-R array; Prop. 7.2
$\hat{y} \leftarrow \mathsf{Bridge}(x)$ ▷ $\hat{y} \approx F_R x$
$\hat{w} \leftarrow \overline{\mathcal{Q}_{p+2}(\hat{y} \odot \hat{y})}$ ▷ conjugated pointwise squaring, one rounding
$\hat{z} \leftarrow \mathsf{Bridge}(\hat{w})$ ▷ $\hat{z} \approx F_R \hat{w}$
return $\mathsf{Recover}(\hat{z})$ ▷ round $-2^{2b-1} R^2 \operatorname{Im} \hat{z}_j$ via (8), propagate radix-2^b carries; Prop. 7.2
procedure $\mathsf{Bridge}(x)$ ▷ approximates $F_R x$ on the grid; Theorem 4.7
  $U \leftarrow x$ scaled to $\widetilde{\mathbb{C}}_q$
  for $a = d$ **down to** 2 **do** ▷ Def. 4.1, Lems. 4.2, 4.8, §4.4
    Spread axis a of U with kernel g_a at offsets δ_a, folded mod t_a
  $Y \leftarrow \mathsf{TensorFFT}(U)$ ▷ direct complex $F_{\mathbf{t}}$, bit-exact Walsh layouts $L, (LB)^{-1}$; Thms. 5.8, 6.5
  for $a = 2$ **to** d **do** ▷ Lems. 4.4, 4.6
    Deconvolve axis a: solve $c_0 Z_f + c_1 Z_{(f - R/A_a) \bmod R} = Y_f$ on each cycle, set $Y \leftarrow Z$
  return Y rescaled ▷ approximates $F_R x$ to working precision

Inner products of $O((\log n)^2)$-bit integers are delegated to M.

2.2 Where the $n \log n$ barrier is broken, and what caps the saving

To see why our outer reduction improves both the structure and the exponent of [OAI26], we compare how the two algorithms connect the finite networks to integer multiplication. Because $R = \Theta(n/\log n)$ and working precision $P = \Theta(k) = \Theta(\log n)$ suffices to round the output coefficients

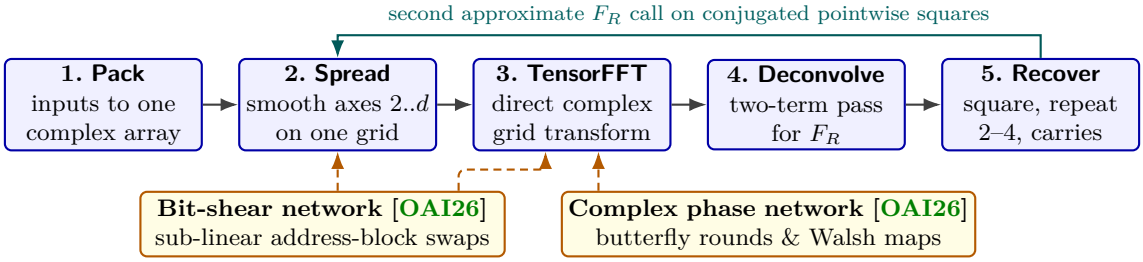


Figure 2: The five-step multiplication pipeline on one power-of-two grid. Steps 2–4 approximate the one-dimensional transform F_R by smoothing axes d down to 2 (Spread), running one direct complex transform (TensorFFT), and undoing the smoothing to working precision (Deconvolve). Step 5 (Recover) squares pointwise, repeats Steps 2–4 once, and propagates carries. The bit-shear network of [OAI26] accelerates axis swaps in Spread and internal chunk alignments in TensorFFT, while the complex phase network accelerates TensorFFT's butterfly rounds and bit-exact Walsh permutations.

to exact integers, the stored array has logical bit volume $V = \Theta(RP) = \Theta(n)$. On a multitape Turing machine, a standard one-dimensional FFT makes $k = \Theta(\log n)$ sequential passes over volume V, costing $\Theta(n \log n)$ steps from tape movement alone.

Harvey and van der Hoeven [HvdH21] reach $O(n \log n)$ by folding a convolution of odd-prime-product length $S = s_1 \cdots s_d$ into d fixed dimensions. Their reduction splits indices across coprime odd primes $s_1, \ldots, s_d$ via a Chinese remainder map, resamples onto larger powers of two t_i, and runs multidimensional FFTs.

In [OAI26], by contrast, the dimension $d = \lfloor k^{\varepsilon} \rfloor$ grows with n, and two fixed linear networks at ground-set parameter $h = 100$ act over block size $m = h^3 = 10^6$. Each network couples W matrix blocks, called *roles*, through sub-calls whose total matrix rank s, the *recursive edge-rank sum*, is strictly smaller than the trivial value Wm; see Fact 3.4.

(i) A *bit-shear network* of W_b roles with $s_b < W_b m^{\tau}$ and $\tau < 1$ interchanges two disjoint contiguous u-bit address fields in time $O(Vu^{\tau})$ rather than $O(Vu)$ [OAI26, Lemma 12]; see Lemma A.8.

(ii) A *complex phase network* of W_c roles with $s_c < W_c m^{\sigma}$ and $\sigma < 1$ executes the normalized butterfly $H_0(u, v) = ((u + v)/2, (u - v)/2)$ simultaneously across $D \le d$ consecutive K-bit address chunks, where $K = \lfloor d^c \rfloor$. Acting on one selected bit per chunk takes $O(Vd^{\lambda'})$ steps with $\lambda' < 1$ rather than $O(Vd)$ [OAI26, Prop. 14]; see Proposition A.11.

Since $\ell = \lceil k/d \rceil$ bit-levels across $d - 1$ outer axes cost $O(V\ell d^{\lambda'}) = O(Vkd^{\lambda'-1})$, one would hope for an exponent saving $\kappa \approx \varepsilon(1 - \lambda')$ linear in $1 - \sigma$.

Around those two primitives, however, [OAI26] retains the surrounding multi-grid reduction of [HvdH21] together with Bluestein's reduction [Blu70] and root-free Nussbaumer polynomial transforms [NQ78, Nus80] over $\mathbb{C}[y]/(y^r + 1)$. That surrounding stack incurs $O(d^2)$ adjacent axis-swap passes and amplifies middle-transform error by the record count $M = R/r$, as contrasted stage by stage in Table 1 and Figure 3.

The chunk-layout cap. Quantitatively, why does [OAI26] obtain $\kappa = 2^{-182}$ instead of a linear saving? Contiguous blocks of $r = t_d$ complex numbers form *records* indexed by $m_0 = \log_2(R/r)$ binary *address bits*. Before the simultaneous butterfly layer can act, those m_0 address bits, initially grouped axis by axis, must be rearranged by a *chunk layout* permutation L that interleaves K-bit chunks across axes. Executing L via $O(k/K)$ width-K chunk interchanges costs $O(V \cdot (k/K)K^{\tau}) = O(VPK^{\tau-1})$, whose saving factor $K^{1-\tau} = \Theta(P^{\varepsilon c(1-\tau)})$ needs the chunk exponent c to be large.

Inside the butterfly-layer recursion, however, recursive *non-leaf levels* subdivide chunk groups down to base *leaf levels* of fewer than d^{β} chunks, and aligning K-bit chunks forces $\tau(1 + c/\beta) < \lambda < 1$,

Table 1: Side-by-side comparison of the prior sub-$n \log n$ pipeline and our single-grid algorithm around the two shared $h = 100$ finite networks of [OAI26]. Every number-theoretic and polynomial-ring stage on the left is removed or replaced on the right, with formal locations listed in the third column.

[HvdH21] / [OAI26]	This paper	Where
CRT map onto d distinct odd primes; prime search by trial division	None: one power-of-two length $R = t_1 \cdots t_d = 2^k$	Thm. 1, §4.1
Gaussian resampling prime $\to 2^\ell$ ([HvdH21]; [OAI26, Lemmas 19–20])	Same-grid sinc-Gaussian spread; negative half-bin phase gives one uniform 2-term solver ($\|B_a^{-1}\|_\infty \le \frac{4}{7} t_a$; cf. [MS])	Lemmas 4.2, 4.4; App. B
$O(d^2)$ adjacent axis-swap passes ([OAI26, §9])	$O(d)$ direct axis exposures	Lemma 4.8
$O(k/K)$ chunk-swap layout; quadratic cap $\kappa < (1-\tau)^2/(C_1\tau)$ ([OAI26, Lemma 16])	Exact Walsh-layer positional permutations; linear cap $\kappa < (1-\sigma)/C_1$	Thm. 5.8, §7.4
Bluestein chirp + twist + ring convolution in $\mathbb{C}[y]/(y^r+1)$ ([OAI26, Prop. 18, Lemma 21])	Direct complex tensor FFT; no polynomial-ring multiplication	Thm. 6.5, Cor. 6.6
Middle error amplified by $M = R/r$; $\log_2 M$-bit output widening ([OAI26, Prop. 18])	Error $< 12\ell \cdot 2^{-q}$; no output widening ($\delta_{\mathrm{mid}} = 0$)	Thm. 6.5, Cor. 6.6
$\Theta(rP)$-bit packed products ([OAI26, Lemma 17], invoking [HvdH21])	$O(P^2)$-bit integer products; any $\mathsf{M}(N) = N(\log N)^{O(1)}$ suffices (e.g. [SS71])	Thm. 1, §7.3
Separate precision budgets for resampling, chirp, and ring convolution	Single contract $P = O(p + \log R)$ with one master margin	Thm. 4.7
Main-route chain uses [HvdH21] and [OAI26] Lemmas 3–4, 9–10, 12–13, 15–21, Props. 7–8, 11, 14, 18	Zero results of [HvdH21]; only [OAI26] Lemmas 3–4, Props. 7–8 as black boxes (9–10, 12–13, Props. 11, 14 re-derived in App. A; 15–21, Prop. 18 unused; none in Lean 4)	§3.4, §8, App. A

so $c < \beta(1-\tau)/\tau$. Meanwhile, each layer carries $\Delta = \lceil C_0 d^{C_1} \rceil$ extra *guard bits* internally to prevent overflow and intermediate rounding. Keeping Δ sublinear in P forces $\varepsilon < 1/C_1$. Multiplying these bounds traps any chunk-exchange layout beneath a *quadratic* ceiling in $1-\tau$:

$$\kappa < \varepsilon c (1-\tau) < \frac{\beta(1-\tau)^2}{C_1 \tau} < \frac{(1-\tau)^2}{C_1 \tau}. \tag{1}$$

Even with our certified $1-\tau = 2^{-40}$ and $C_1 = 14$, (1) caps κ below $2^{-83.8}$. At the schedule $\varepsilon = 2^{-75}$ and $c = 2^{-56}$ of [OAI26, (38)], the first term of (1) is 2^{-181}, giving $\kappa = 2^{-182}$, while the ceiling itself is about 2^{-104} under the printed constants.

2.3 A CRT-free front end: what the bridge removes and how

Our front end replaces prime-length splitting and Gaussian resampling by an algebraic bridge between F_R at $R = 2^k$ and a multidimensional transform on the same power-of-two grid.

Cooley–Tukey twiddles as fractional shifts. In dimension two [MS], with $R = t_1 r$, input indices $j = j_1 + t_1 j_2$, and reversed-digit frequencies $f = u_1 r + u_2$ for $0 \le j_1, u_1 < t_1$ and $0 \le j_2, u_2 < r$, expanding $(F_R x)_f = \frac{1}{R} \sum_{j=0}^{R-1} x_j e^{2\pi i j f / R}$ yields

$$(F_R x)_{u_1 r + u_2} = \frac{1}{t_1} \sum_{j_1=0}^{t_1-1} e^{2\pi i j_1 u_1 / t_1} \left(\frac{1}{r} \sum_{j_2=0}^{r-1} x_{j_1 + t_1 j_2} \, e^{2\pi i (j_2 + j_1/t_1) u_2 / r} \right). \tag{2}$$

The inner sum is the length-r DFT of row j_1 at the fractional shift $j_2 + \delta_2$ with $\delta_2 = j_1/t_1 \in [0,1)$, which Spread realizes in the index domain by convolving row j_1 with a localized kernel $g(s)$ folded modulo r at offset δ_2.

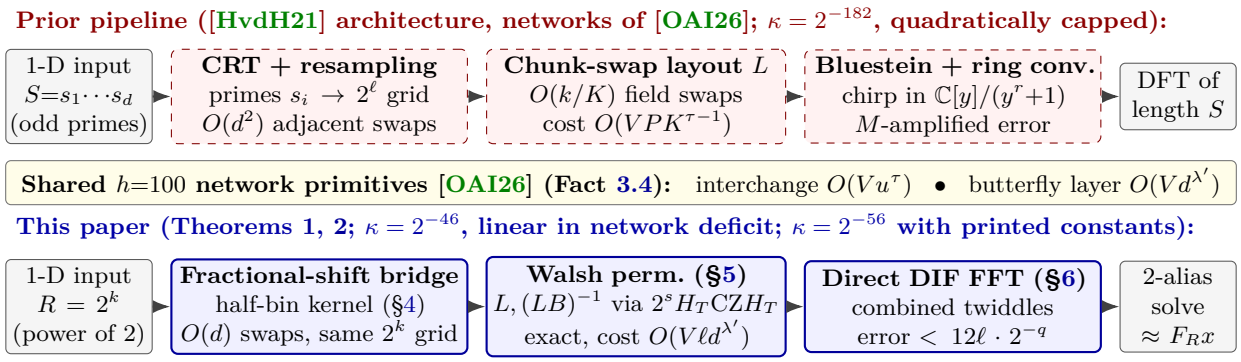


Figure 3: The prior sub-$n \log n$ multiplication pipeline compared with our single-grid pipeline. Both share the two fixed $h = 100$ network primitives of [OAI26], summarized in the center bar. Our pipeline replaces the top row's multi-grid and ring-convolution stages by Spread on one power-of-two grid, bit-exact Walsh permutations, a direct complex Fourier transform, and Deconvolve to working precision.

Poisson summation equates the transform of a periodized shifted kernel to a sum over integer-shifted frequency copies called *aliases*, expressing $F_{t_1,r}U$ on the *same* $t_1 \times r$ grid as

$$(F_{t_1,r}U)_{u_1,u_2} = \sum_{\nu \in \mathbb{Z}} \hat{g}\Big(\frac{u_2}{r} - \nu\Big)\,(F_R x)_{((u_1-\nu) \bmod t_1)\,r+u_2}.$$

For the *sinc-Gaussian kernel* $g_0(s) = \mathrm{sinc}(s)e^{-\pi s^2/\alpha^2}$, the product of $\mathrm{sinc}(s) = \sin(\pi s)/(\pi s)$ and a Gaussian of width $\alpha \asymp \sqrt{\omega}$, the transform $G(\xi) = \int_{\xi-1/2}^{\xi+1/2} \alpha e^{-\pi\alpha^2 v^2}\,dv$ smooths the indicator of $[-\frac{1}{2}, \frac{1}{2}]$. For $\xi \in (0,1)$, all aliases $\nu \notin \{0,1\}$ have Gaussian tail weight $O(2^{-\omega})$. Truncating the full alias sum to $\nu \in \{0,1\}$ leaves a *two-alias deconvolution* operator $B_a = c_0 I + c_1 T$ per cycle, where T is the unit cyclic shift $(Tz)_u = z_{u-1}$, undoing the smoothing to working precision.

The Nyquist singularity and negative half-bin modulation. If the spreading kernel is real-valued, conjugate symmetry $\hat{g}(-\xi) = \overline{\hat{g}(\xi)}$ forces $|c_0| = |\hat{g}(\frac{1}{2})| = |\hat{g}(-\frac{1}{2})| = |c_1|$ at the *Nyquist frequency* $u_2 = r/2$, where $\xi = \frac{1}{2}$. On an even cycle of length L, $c_0 I + c_1 T$ is singular for even g, and for any real Schwartz kernel its inverse has ℓ^∞ norm $\Omega(L)$ whenever invertible; see Proposition B.2. Whereas [MS] handles one Nyquist column at $d = 2$ via an alternating-sum identity, in d dimensions cycle lengths reach $L = A_a = \prod_{i<a} t_i \le R/r$, and losing $\Omega(L)$ conditioning on $d-1$ axes would cost $\sum_{a=2}^{d} \log_2 A_a = \Theta(dk)$ extra precision bits, destroying $V = \Theta(n)$.

We eliminate the Nyquist singularity uniformly across all dimensions and cycle lengths by multiplying the sinc-Gaussian kernel on each spread axis $a \in \{2, \ldots, d\}$ by a complex *carrier*, namely a negative half-bin phase:

$$g_a(s) = e^{-\pi i s/t_a}\ \mathrm{sinc}(s)\, e^{-\pi s^2/\alpha^2}, \qquad \hat{g}_a(\xi) = G\Big(\xi + \frac{1}{2t_a}\Big). \tag{3}$$

This carrier shifts every frequency sample from u_a/t_a to a half-bin midpoint $y = (u_a + \frac{1}{2})/t_a \in (0,1)$, as shown in Figure 4. Write $\mathcal{A}_a$ for the full alias operator on axis a over all $\nu \in \mathbb{Z}$, defined in (4) below. Because $t_a \ge 2$ is even, no integer $u_a \in [t_a]$ hits $y = \frac{1}{2}$, so $|y - \frac{1}{2}| \ge \frac{1}{2t_a}$ gives $|c_0 - c_1| = |G(y) - G(y-1)| \ge \frac{7}{4t_a}$, two-alias truncation error $\|\mathcal{A}_a - B_a\|_\infty = O(2^{-\omega})$, and inverse norm

$$\|B_a^{-1}\|_\infty \le \frac{1}{|c_0 - c_1|} \le \frac{4}{7}\,t_a \le t_a$$

independent of $L = A_a$; see Lemma 4.4. The negative sign in (3) is essential: a positive phase $e^{+\pi i s/t_a}$ shifts $u_a = 0$ to $y = -\frac{1}{2t_a} \notin (0,1)$, nearing the $\nu = -1$ passband and degrading the alias

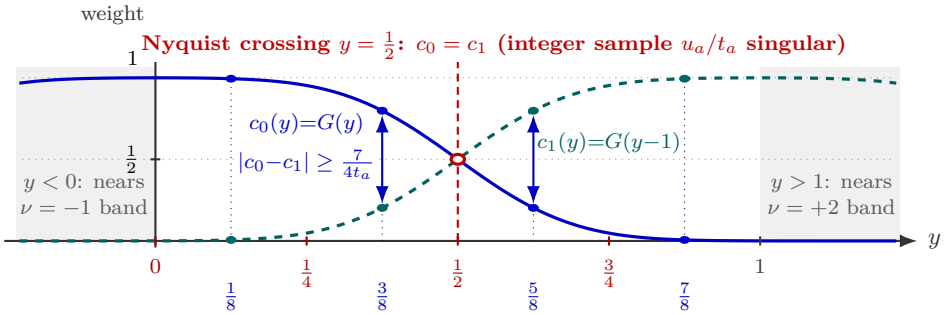


Figure 4: Why shifting frequency samples by half a bin keeps the two-term deconvolution system well conditioned, illustrated for axis length $t_a = 4$ and width parameter $\omega = 8$. Integer-bin samples u_a/t_a in red hit the Nyquist midpoint $y = \frac{1}{2}$ where $c_0 = c_1$, making the solver singular. Negative half-bin samples $y = (u_a + \frac{1}{2})/t_a$ shown as solid dots stay inside $(0, 1)$ at distance at least $\frac{1}{2t_a}$ from $\frac{1}{2}$, giving $|c_0 - c_1| \geq \frac{7}{4t_a}$ and $\|B_a^{-1}\|_\infty \leq \frac{4}{7}t_a$.

tail to $\Omega(\omega^{-1/2}2^{-\omega/4})$ at $t_a = 2$ in Proposition B.1. Because $e^{-\pi i s/t_a}$ is anti-periodic of period t_a, folding g_a modulo t_a alternates signs.

Triangular non-commuting factorization in d dimensions. For general $R = t_1 \cdots t_d = 2^k$, put $A_a = \prod_{i<a} t_i$, write time indices as $j = \sum_{i=1}^d j_i A_i$ with j_1 least significant, and write frequencies in reversed digit order $f(\mathbf{u}) = \sum_{i=1}^d u_i R/A_{i+1}$ with u_d least significant. On axis $a \geq 2$, the fractional shift $\delta_a(\mathbf{j}) = (\sum_{i<a} j_i A_i)/A_a \in [0, 1)$ depends on the untouched digits below a, so Spread processes axes in descending order $a = d, d-1, \ldots, 2$ to form $U = \mathcal{S}_2\mathcal{S}_3 \cdots \mathcal{S}_d x$. Dual to this in frequency space, shifting f by $-\nu R/A_a$ modifies only $u_1, \ldots, u_{a-1}$ while leaving $u_a, \ldots, u_d$ fixed, so Poisson summation factors into an exact triangular product in Lemma 4.2:

$$F_{\mathbf{t}} U = \mathcal{A}_2\mathcal{A}_3 \cdots \mathcal{A}_d (F_R x), \qquad (\mathcal{A}_a Z)_f = \sum_{\nu \in \mathbb{Z}} G\Big(\frac{u_a(f) + 1/2}{t_a} - \nu\Big) Z_{(f - \nu R/A_a) \bmod R}. \tag{4}$$

Because the alias operators $\mathcal{A}_a$ do not commute, Deconvolve applies the two-alias cyclic inverses $B_2^{-1}, B_3^{-1}, \ldots, B_d^{-1}$ in ascending order, where $\|(B_2 \cdots B_d)^{-1}\|_\infty \leq (4/7)^{d-1}(R/t_1) \leq R/t_1$. Precomputed tables need only be *faithful*, meaning certified within one or two grid units of true values. At working precision $P = O(p + \log R)$ and grid precision $q = P + 2$, the bridge precision theorem, Theorem 4.7, bounds the total error by $O(2^{-p} \max(1, \|x\|_\infty))$, while direct field interchanges with axis d cut axis exposures from $O(d^2)$ passes to $O(d)$ interchanges of cost $O(Vd(1+\ell^\tau))$ in Lemma 4.8.

2.4 Butterfly layers as exact positional permutations

The central new tool of our middle transform uses the simultaneous butterfly layer of Proposition A.11 to execute any *positional permutation* bit-exactly in a constant number of layer calls per bit-offset pass. Here a positional permutation means any reordering of the bit positions of a binary record address.

Selective Walsh layers via three-layer cancellation. Consider an array of 2^{m_0} records on the q-grid whose entries lie in the complex unit disk $|z| \leq 1$, where $m_0 = \log_2(R/r) = O(q)$ and a *slice* picks one bit position from each of $D \leq d$ consecutive K-bit address chunks. One black-box call to Proposition A.11 applies $H_0 = \frac{1}{2}\left(\begin{smallmatrix} 1 & 1 \\ 1 & -1 \end{smallmatrix}\right)$ to all D slice positions at once, written H_{all}; individual chunks cannot be turned off inside its recursive schedule. Instead, we use two 2×2 *Walsh identities* with the phase gate $S = \mathrm{diag}(1, i)$:

$$H_0^3 = \tfrac{1}{2} H_0, \qquad (H_0 S)^3 = \tfrac{1+i}{4} I. \tag{5}$$

Indeed, $H_0^2 = \frac{1}{2}I$ gives $H_0^3 = \frac{1}{2}H_0$, while cubing $H_0 S = \frac{1}{2}\left(\begin{smallmatrix} 1 & i \\ 1 & -i \end{smallmatrix}\right)$ gives $(H_0 S)^2 = \frac{1}{4}\left(\begin{smallmatrix} 1+i & 1+i \\ 1-i & i-1 \end{smallmatrix}\right)$ and $(H_0 S)^3 = \frac{1+i}{4}I$.

Suppose we want to apply H_0 only on a subset T of size $s = |T|$ of the slice and act as the identity on its complement U of size $q_{\text{sel}} = D - s$. We run three full layer calls H_{all}, preceding each call by a linear scan that multiplies every record at address x by $(S_U z)(x) = i^{\sum_{j \in U} x_j} z(x)$. Because operators on distinct address bits tensor and commute, $(H_{\text{all}} S_U)^3$ acts as $\frac{1}{2}H_0$ on every bit in T and collapses to $\frac{1+i}{4}I$ on every bit in U. Rescaling by $2^s[2(1-i)]^{q_{\text{sel}}}$ recovers the selective operator H_T exactly, as proved in Lemma 5.1 and illustrated in Figure 5(a).

Batched controlled flips without address movement. Next, conjugating $\mathrm{diag}(1, -1)$ by H_0 gives $2H_0 \,\mathrm{diag}(1, -1) H_0 = \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$, whereas $2H_0 I H_0 = I$. Take s pairwise disjoint control-target bit pairs (c_j, t_j) whose targets $T = \{t_1, \ldots, t_s\}$ lie in one slice while the controls c_j lie anywhere in the m_0-bit record address. Define the recordwise sign diagonal $(\mathrm{CZ}\, z)(x) = (-1)^{\sum_{j=1}^{s} x_{c_j} x_{t_j}} z(x)$. By Lemma 5.4, shown in Figure 5(b),

$$\mathrm{CNOT}^{\otimes s} = 2^s H_T \,\mathrm{CZ}\, H_T \tag{6}$$

moves every record from address x to the address with $x_{t_j} \mapsto x_{t_j} \oplus x_{c_j}$ for all $j = 1, \ldots, s$ using six full butterfly-layer calls and linear scans; see Proposition 5.5. Crucially, CZ never routes data between c_j and t_j. It computes the parity $\sum_j x_{c_j} x_{t_j} \bmod 2$ from each record's address in $q^{O(1)}$ steps, absorbed by the $\Theta(rq)$-bit record payload via Lemma A.5, and negates the record in place.

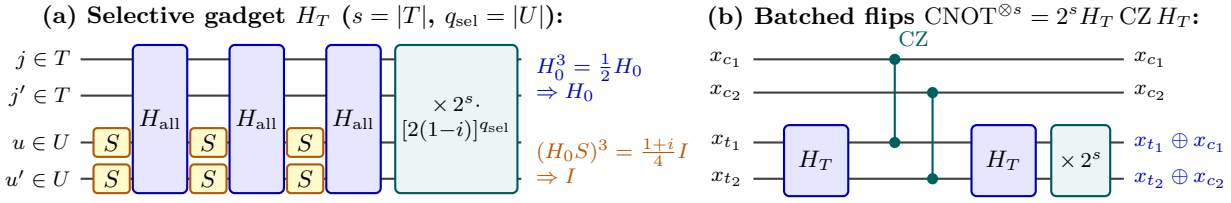


Figure 5: How simultaneous butterfly layers become bit-exact controlled address flips. In panel (a), inserting $S = \mathrm{diag}(1, i)$ on unselected bits U before three full layers cancels U to $\frac{1+i}{4}I$ while leaving $\frac{1}{2}H_0$ on selected bits T. In panel (b), sandwiching the recordwise sign diagonal CZ between two selective H_T gadgets flips all target address bits $t_j \in T$ by distant controls c_j in six layer calls.

Every positional permutation is $O(1)$ CNOT stages, and is bit-exact. Every permutation π of the m_0 record-address bit positions factors on its cycles as $\pi = I_1 \circ I_0$, a product of two involutions, and each involution is a set of disjoint two-bit swaps implemented by three stages of disjoint CNOTs; see Lemma 5.7. Partitioning the m_0 positions into $N_{\text{slice}} \le m_0/d + K$ slices of $D \le d$ chunks, plus fewer than K leftover positions handled by linear stream swaps, executes any positional permutation π in at most $36 N_{\text{slice}}$ layer calls. In Theorem 5.8, this costs $O(V[(m_0/d + K)d^{\lambda'} + K]) = O(V \ell d^{\lambda'})$ steps, equivalently $O(Vqd^{\lambda'-1})$ at $q = \Theta(k)$, matching the FFT butterfly rounds themselves.

Why does fixed-point truncation introduce zero error across complex linear combinations? Before each CNOT batch on a q-grid unit-disk array, Lemma 5.3 appends $8d$ zero fractional bits to lift to precision $q_* = q + 8d$. Each normalized D-bit layer divides by 2^D with $D \le d$, while S_U and CZ multiply by units in $\{\pm 1, \pm i\}$, so every intermediate entry has dyadic denominator exponent at most $q + s + 3D \le q + 4d < q_*$. Because Proposition A.11 evaluates each layer exactly internally using its guard Δ before one truncation Q_{q_*}, every truncation discards only trailing zeros by Lemma 5.2, and stripping the $8d$ zero bits afterward returns the exact permuted q-grid array.

2.5 A direct tensor FFT and where 2^{-46} comes from

With bit-exact positional permutations available at butterfly-layer speed, we evaluate the multi-dimensional Fourier transform directly over the complex numbers and remove the quadratic layout ceiling on κ.

Direct decimation-in-frequency tensor FFT. Bluestein's chirp reduction and polynomial-ring convolutions over $\mathbb{C}[y]/(y^r+1)$ disappear completely from TensorFFT. On the q-grid with $t_d = r = 2^\ell$ and $t_i \in \{r/2, r\}$, Theorem 6.5 computes $F_{\mathbf{t}}U$ in four steps.

(1) Run a standard length-r decimation-in-frequency FFT inside each contiguous record along axis d.
(2) Apply the bit-exact positional permutation L of Lemma 6.3 to interleave K-bit chunks of the $d-1$ record axes.
(3) Execute the ℓ decimation-in-frequency bit-levels across all $d-1$ record axes in parallel: at most K prefix or leftover levels by individual butterflies in $O(VdK)$, and the remaining chunk levels by one call to Proposition A.11 per level in $O(V\ell d^{\lambda'})$. Multiplying $d-1$ separately rounded twiddles per level would accumulate $O(d\ell \cdot 2^{-q})$ error. Instead, we sum all participating axis twiddle exponents modulo r into one exact table index and look up one $2r$-entry unit-disk phase table from Lemma 6.1, incurring error $< 6 \cdot 2^{-q}$ per level regardless of d.
(4) Apply the bit-exact positional permutation $(LB)^{-1} = B^{-1}L^{-1}$ via Theorem 5.8 to undo both the chunk layout L and the decimation-in-frequency bit-reversal B in one pass.

Every exact decimation-in-frequency round is an ℓ^∞ contraction, and the positional maps are bit-exact, so the ℓ last-axis levels and at most ℓ record-axis rounds accumulate error strictly below $12\ell \cdot 2^{-q}$ in Corollary 6.6, with no R/r amplification and no output bit-widening.

Cost balance and the exponent $\kappa = 2^{-46}$. Dividing the cost of one bridge call by $V = RP = \Theta(n)$ gives the five normalized terms of Table 5, where $P = \Theta(\log n)$, $d = \lfloor k^\varepsilon \rfloor = \Theta(P^\varepsilon)$, $\ell = \Theta(P^{1-\varepsilon})$, $K = \lfloor d^c \rfloor = \Theta(P^{\varepsilon c})$, and $\mathsf{m}(N) = \mathsf{M}(N)/N = (\log N)^{O(1)}$. Prefix butterflies cost $O(dK)$ with gap $1-\varepsilon(1+c)$ below P^1, while simultaneous layers and the exact maps $L, (LB)^{-1}$ cost $O(\ell d^{\lambda'})$ with gap $\varepsilon(1-\lambda')$. The last-axis FFT and combined twiddles cost $O(\ell\,\mathsf{m}(P))$ with gap ε up to $\log P$, direct axis exposures cost $O(d(1+\ell^\tau))$ with gap $(1-\varepsilon)(1-\tau)$, and Spread and Deconvolve cost $O(d(\mathsf{m}(P)+\mathsf{m}(P^2)))$ with gap $1-\varepsilon$ up to $\log P$. With $PK^{\tau-1}$ gone, choosing c tiny leaves the linear butterfly-layer gap $g_{\text{lay}} = \varepsilon(1-\lambda') < (1-\sigma)/C_1$ as the sole bottleneck.

Table 2: Comparison of the asymptotic saving exponent κ in $O(n(\log n)^{1-\kappa})$ across architectures and network-constant regimes. Row 1 is the bound of [OAI26], whose parameter choices [OAI26, §9.1] notes are deliberately conservative; Row 2 is proved in Appendix C, and Rows 3 and 4 in Section 7, with Row 4 formally verified in Lean 4 (Theorem 2). Additional comparison rows, including exposed-parameter Bluestein retunings (2^{-85}, 2^{-84}) and $C_1 = 20$ (2^{-47}), appear in Appendix C.

Algorithm	Front end	Middle transform	(β, C_1)	$(1-\tau, 1-\sigma)$	Binding gap	κ
[OAI26]	CRT + resampling	Bluestein + chunk swaps	$(\frac{1}{2}, 20)$	$(2^{-50}, 2^{-50})$	$\varepsilon c(1-\tau)$	2^{-182}
Bluestein, retuned (ε, c)	Frac.-shift bridge	Bluestein + chunk swaps	$(\frac{1}{2}, 20)$	$(2^{-50}, 2^{-50})$	$\varepsilon c(1-\tau)$	2^{-108}
Ours (printed)	Frac.-shift bridge	Direct FFT + Walsh perm.	$(\frac{1}{2}, 20)$	$(2^{-50}, 2^{-50})$	$\varepsilon(1-\lambda')$	2^{-56}
Ours (Thms. 1, 2)	Frac.-shift bridge	Direct FFT + Walsh perm.	$(\frac{1}{64}, 14)$	$(2^{-40}, 2^{-42})$	$\varepsilon(1-\lambda')$	2^{-46}

Table 2 separates the architectural gain from exposing internal layer parameters, with $k = \log_2 R$, $d = \lfloor k^\varepsilon \rfloor$, and $K = \lfloor d^c \rfloor$. Row 1 is the bound of [OAI26] at $\varepsilon = 2^{-75}$ and $c = 2^{-56}$. Retuning (ε, c) under the same printed constants with our bridge and $O(d)$ exposures raises the Bluestein route to 2^{-108} in Row 2, while (1) still traps any chunk-swap layout below $\beta(1-\tau)^2/(C_1\tau) \approx 2^{-105.3} < (1-\tau)^2/(C_1\tau) \approx 2^{-104.3}$. Comparing Rows 2 and 3 holds $(\beta, \tau, \sigma, C_1)$ fixed and replaces Bluestein convolutions and chunk swaps by Walsh permutations and TensorFFT, lifting κ from 2^{-108} to 2^{-56}.

Comparing Rows 3 and 4 isolates parameter exposure at $h = 100$ and $m = 10^6$, where the complex network of [OAI26] has exact relative deficit $\eta_c = (W_c m - s_c)/(W_c m) = 73/19{,}906{,}842{,}167{,}500$. Using $\ln 10^6 < 14$ via $e^7 > \sum_{j=0}^{11} 7^j/j! > 1000$ in place of $\ln m < 20$, Proposition A.2 certifies $s_c/W_c < m^\sigma$ at $\sigma = 1 - \xi$ with $\xi = 2^{-42}$, since $73 \cdot 2^{42} > 14 \cdot 19{,}906{,}842{,}167{,}500$, and $\tau = 1 - 4\xi = 1 - 2^{-40}$ for the bit network. Meanwhile, the network amplitude bound $B_{\text{net}} < 10^{66} = m^{11}$ gives guard exponent $C_1 = 11 + 3 = 14$ in Proposition A.11, matching [OAI26, §5.4]. Setting $\varepsilon = 1/15$, $\beta = 1/64$, $c = \xi/256$, $\lambda = 1 - \frac{63}{64}\xi$, and $\lambda' = 1 - \frac{31}{32}\xi$ in Section 7.5 gives $g_{\text{lay}} = \frac{31}{480}\xi > \frac{1}{16}\xi = 2^{-46}$, reaching the linear ceiling $(1 - \sigma)/C_1 \approx 2^{-45.8}$.

The following remark summarizes the exact dyadic ceilings of our direct route under printed constants and of the chunk-exchange Bluestein route under exposed constants.

Remark 2.1 (Printed constants and the Bluestein ceiling). Keeping the structural constants $\beta = \frac{1}{2}$, $\tau = \sigma = 1 - 2^{-50}$, and $C_1 = 20$ printed in [OAI26, Prop. 14] fixed and varying only the outer schedule exponents, our direct-FFT cost analysis already gives $\kappa = 2^{-56}$, the largest dyadic exponent 2^{-j} that our analysis admits with those constants (Section 7). Conversely, if the direct tensor FFT is replaced by a Bluestein middle transform whose layout is executed by chunk interchanges, the saving is capped quadratically at $\kappa < (1 - \tau)^2/(C_1\tau) \approx 2^{-83.8}$ even with our certified parameters (largest dyadic value $\kappa = 2^{-84}$). At the parameters of [OAI26], the same Bluestein analysis reproduces $\kappa = 2^{-182}$ (Appendix C).

A second remark notes how the small $O((\log n)^2)$-bit inner scalar products are handled in the written proof and in the Lean 4 formalization.

Remark 2.2 (Inner integer products without CRT). Theorem 1 requires only $\mathsf{M}(N) = N(\log N)^{O(1)}$ for the independent inner multiplier on $O((\log n)^2)$-bit operands, so classical Schönhage–Strassen multiplication [SS71] suffices as a black box. Instantiating M instead with the two-dimensional fractional-shift multiplier of [MS] eliminates Chinese remainder maps and prime lengths from the inner scalar products as well (Section 7). Its written analysis uses faithful phase and filter tables as in Section 4; Theorem 2 uses our independent Lean 4 formalization `IntMulTM.intMul_TM2_nlogn` (Section 8), separate from the Lean development of [MS].

2.6 Formal verification

Section 8 describes our Lean 4 formalization [Lean26] of Theorem 2, which verifies the complete $\kappa = 2^{-46}$ machine and its network certificates from Lean's three standard axioms alone. It constructs a concrete finite-alphabet machine in Mathlib's multi-stack model `Turing.FinTM2`, where each stack acts as a tape accessed at its top cell, wrapped by `Turing.TM2ComputableInTime`. On that machine it proves `SubNLogN.Challenge.IntMulSubNLogN`: exact multiplication of all $(a, b) \in \mathbb{N}^2$ within $C\,L\,(\log_2 L)^{1-2^{-46}}$ steps for input length $L \geq N_0$. Both $h = 100$ finite networks, their rank and endpoint certificates, all stream and parameter-transfer procedures, and the analytical bridge error bounds are proved from scratch.

The formal theorem statement records the asymptotic complexity bound; it does not formalize the structural claim that the construction uses no Chinese remainder map. The development formalizes the $\kappa = 2^{-46}$ Walsh machine directly and derives 2^{-47}, 2^{-56}, and $o(n \log n)$ by exponent monotonicity. It differs from the written text in two minor ways. First, the formalized machine fixes tie-upward nearest rounding, while our Lean bridge theorems hold for arbitrary nearest rounding. Second, it instantiates the inner $O(N \log N)$ multiplier via our independent Lean 4 proof `IntMulTM.intMul_TM2_nlogn` of the two-dimensional multiplier of [MS], kernel-checked within [Lean26] and separate from the Lean archive distributed with [MS].

3 Preliminaries

This section sets up the shared definitions and external facts needed to state and prove the bridge, permutation, and Fourier-transform bounds of Sections 4–7. Section 3.1 collects global notation in Table 3, and Section 3.2 specifies the multitape Turing machine model, tape array layout, and fixed-point rounding rules. Section 3.3 fixes our normalized discrete Fourier transform conventions and the two-call real convolution identity, while Section 3.4 records the two black-box facts borrowed from [OAI26]. A first-time reader may skim Table 3 and Section 3.4 and return to them when cited.

3.1 Notation

We begin by fixing basic conventions for logarithms, index ranges, and ℓ^∞ norms used across all sections. Logarithms are natural, written ln, unless written $\log_2$. Following [OAI26], we write $\lg n = \max\{\lceil \log_2 n \rceil, 1\}$, so $\lg R = \log_2 R$ whenever $R \geq 2$ is a power of two. For a positive integer N we write $[N] = \{0, \ldots, N-1\}$. For a complex vector x, $\|x\|_\infty = \max_j |x_j|$ denotes the largest complex modulus, and for a linear map A, $\|A\|_\infty$ is the induced ℓ^∞ operator norm, equal to the maximum absolute row sum. For an array of polynomial records in $\mathcal{R}_r = \mathbb{C}[y]/(y^r + 1)$, $\|\cdot\|_\infty$ denotes the largest modulus of any complex coefficient across the array. Table 3 summarizes global symbols.

3.2 Machine model and fixed-point arithmetic

This subsection defines the multitape Turing machine cost model, the sequential tape representation of multidimensional arrays, and the fixed-point complex grids on which the algorithm operates. Our complexity bounds refer to the standard deterministic multitape Turing machine model of [SS71, HvdH21, OAI26] with one fixed finite alphabet and a fixed finite number of one-dimensional tapes. In each step the machine reads and writes at most one cell per tape and moves each head by at most one cell. The running time is the total number of steps, so every scan, head movement, rewind, counter update, and workspace cleanup is charged. The input is $x\#y$ with $x, y \in \{0,1\}^n$, and the output is the $2n$-bit binary representation of their product [OAI26, §1].

Following [OAI26, §2], an array on $[a_1] \times \cdots \times [a_t]$ with $a_i \geq 1$ is stored in lexicographic order with the first coordinate most significant, so addresses are implicit in tape position rather than stored per entry. Adjacent spectator fields, meaning address fields that the current operation leaves untouched, may be collapsed into one field whose range is their product. An address permutation f moves the entry at address x to $f(x)$, giving $(fA)(f(x)) = A(x)$. The *logical bit volume* V of an array is its total number of stored data bits; temporary copies on work tapes are charged in running time without replacing V in recursive bounds. Integer parameters in procedure headers use the self-delimiting code $\Gamma(v) = 0^{b_v - 1}\mathrm{bin}_{b_v}(v)$ of length $2b_v - 1$, where $b_v = \lfloor \log_2 v \rfloor + 1$ for $v \geq 1$ [OAI26, (1)].

Independent integer multiplier. Ordinary products of N-bit integers are delegated to an independent deterministic multiplier of worst-case cost $\mathsf{M}(N) = N\,\mathsf{m}(N)$, such as Schönhage–Strassen multiplication [SS71] with $\mathsf{m}(N) = O(\lg N \lg\lg N)$. It runs as a black-box subroutine on dedicated work tapes that are clean on return, leaving the caller's other tapes untouched and making no recursive call to Algorithm 1.

Grids, rounding, and faithful tables. All stored complex numbers lie on dyadic fixed-point grids. The next definition fixes the grid $\widetilde{\mathbb{C}}_P$ and disk grid $\mathbb{D}_P$ at an arbitrary integer precision

Table 3: Reference summary of global symbols and parameters used throughout the paper. Bridge-specific quantities are defined in Section 4, and finite-network parameters in Section 3.4.

Symbol	Meaning
$n,\ \kappa$	bit length of each input integer; log-saving exponent in $O(n(\lg n)^{1-\kappa})$
$R = t_1 \cdots t_d,\ k = \lg R$	bridge transform length ($t_i \geq 2$ powers of two, d dimension, $r = t_d = 2^\ell$) and its binary logarithm
$M = R/r,\ \mathsf{M}(N),\ \mathsf{m}(N)$	number of records; cost of the independent N-bit integer multiplier; $\mathsf{m}(N) = \mathsf{M}(N)/N$
$p,\ P$	target precision of a bridge call (errors in units of 2^{-p}); internal working precision of the bridge
$q,\ v = 2^{-q}$	dyadic grid precision of Sections 5–6, instantiated by the bridge at $q = P + 2$; unit of that grid
w	two's-complement component width; $w = q + 2 = P + 4$ in the bridge schedule (24)
$\omega,\ \alpha$	alias width parameter and Gaussian width parameter of the bridge kernel
$a \in \{2, \ldots, d\},\ \nu \in \mathbb{Z}$	spread-axis index of the bridge; alias index in Poisson summation
$g_a,\ G$	negative half-bin carrier on axis a; Fourier transform of $\operatorname{sinc}(s)\, e^{-\pi s^2/\alpha^2}$
$f(\mathbf{u}),\ A_a$	scalar frequency index with digits $u_1, \ldots, u_d$; $A_a = \prod_{i<a} t_i$, the cycle length of B_a
$c_0,\ c_1,\ c_{\max}$	retained alias weights $G((u+\frac{1}{2})/t)$ and $G((u+\frac{1}{2})/t - 1)$; $\max(c_0, c_1)$
$\mathcal{A}_a,\ B_a$	full alias operator on axis a; two-term cyclic operator (its two-alias truncation)
$\mathcal{R}_r = \mathbb{C}[y]/(y^r + 1)$	record ring specifying the r-coefficient record layout
$\widetilde{\mathbb{C}}_q,\ \mathbb{D}_q$	fixed-point grid $2^{-q}(\mathbb{Z} + i\mathbb{Z})$; disk grid $\{z \in \widetilde{\mathbb{C}}_q : \lvert z\rvert \leq 1\}$
$Q_q,\ \mathcal{Q}_q$	componentwise truncation toward zero to $\widetilde{\mathbb{C}}_q$; componentwise nearest rounding to $\widetilde{\mathbb{C}}_q$
V	logical bit volume of a stored array
$K = \lfloor d^c \rfloor,\ D$	chunk width and number of active K-bit chunks ($1 \leq D \leq d$) in a simultaneous butterfly layer
$A,\ q_{\mathrm{rad}},\ \Gamma$	layer and stream interface symbols: prefix range (written P in [OAI26, Prop. 14]), fixed radix (written q in [OAI26]), and self-delimiting header code [OAI26, (1)]
$H_0,\ S$	normalized butterfly $(u, v) \mapsto ((u+v)/2, (u-v)/2)$ and phase gate $\operatorname{diag}(1, i)$
$h = 100,\ m = h^3 = 10^6$	ground-set size parameters of the fixed triple networks of [OAI26]
$W_b, s_b;\ W_c, s_c$	role counts and recursive call counts of the bit and complex networks ($s_b < W_b m,\ s_c < W_c m$)
$\eta_b,\ \eta_c;\ \tau,\ \sigma$	relative rank deficits $(Wm - s)/(Wm)$ and network exponents ($s_b/W_b < m^\tau,\ s_c/W_c < m^\sigma$)
$\beta,\ \nu_{\mathrm{net}},\ C_1$	leaf-stopping exponent, guard degree $\min\{j \geq 1 : m^j \geq B_{\mathrm{net}}\} = 11$, and guard exponent of the layer
$\varepsilon,\ c,\ \lambda,\ \lambda'$	schedule and recurrence exponents; $d = \lfloor k^\varepsilon \rfloor$, $K = \lfloor d^c \rfloor$
$S,\ L,\ B,\ K$	also reused locally (defined at use): suffix range or pre-scale/series width S, layout map or cycle/input length L, bit-reversal or suffix range B, general chunk width K (and K_0, K_*)

$P \geq 0$, and specifies how records of r complex numbers in $\mathbb{D}_q$ are stored in two's-complement binary at precision $q \geq 0$. It also introduces the two rounding operators, truncation toward zero Q_P and nearest rounding $\mathcal{Q}_P$, whose bounds (7) govern where each is used below.

Definition 3.1 (Fixed-point grids and rounding)**.** For an integer precision $P \geq 0$, define the *fixed-point grid* $\widetilde{\mathbb{C}}_P = \{2^{-P}(a + ib) : a, b \in \mathbb{Z}\}$ (also called the *P-grid*) and the *disk grid* $\mathbb{D}_P = \{z \in$

$\widetilde{\mathbb{C}}_P : |z| \le 1\}$. At precision $q \ge 0$, a *disk-grid record* is an element $\sum_{j<r} z_j y^j \in \mathcal{R}_r = \mathbb{C}[y]/(y^r+1)$ with every $z_j \in \mathbb{D}_q$, stored in coefficient order $j = 0, \dots, r-1$; a *disk array* is an array of records with $\|z\|_\infty \le 1$. Each real or imaginary component is stored as 2^{-q} times a signed w-bit two's-complement integer, most significant bit first, with the real component before the imaginary one [OAI26, Prop. 14]; any width $w \ge q+2$ represents every disk-grid component, including ± 1. For $z \in \mathbb{C}$ and any integer $P \ge 0$:

- $Q_P(z)$ truncates the real and imaginary parts of z toward zero to $2^{-P}\mathbb{Z}$ [OAI26, (26)];
- $\mathcal{Q}_P(z)$ rounds the real and imaginary parts of z to a nearest element of $2^{-P}\mathbb{Z}$ (for concreteness with ties to even, though every bound we prove holds for any nearest-rounding tie rule).

Both operators act coefficientwise on records and arrays.

For every $z \in \mathbb{C}$ and integer $P \ge 0$,

$$|Q_P(z) - z| < \sqrt{2} \cdot 2^{-P}, \qquad |Q_P(z)| \le |z|, \qquad |\mathcal{Q}_P(z) - z| \le \tfrac{\sqrt{2}}{2} \cdot 2^{-P}. \tag{7}$$

In particular, Q_q maps the closed unit disk into $\mathbb{D}_q$, so any exact disk-preserving operator followed by Q_q keeps every stored coefficient inside $\mathbb{D}_q$, whereas nearest rounding can step outside the unit disk near ± 1. By contrast, inside the bridge's spreading and deconvolution stages, where norms are tracked explicitly, $\mathcal{Q}_q$ halves the per-step rounding error. Precomputed transcendental entries for filter taps, solver parameters, and unit phases are never assumed to be exact nearest roundings. Instead, they are *faithful* approximations computed by truncated rational series with certified error at most 2^{-q} for real entries and $2 \cdot 2^{-q}$ for complex taps and disk-valued unit phases.

3.3 Discrete Fourier transforms

This subsection fixes the normalization and sign convention of our one-dimensional and multidimensional Fourier transforms and records the identity that recovers a real cyclic convolution from two forward transforms. Dividing by the transform length R in the definition below ensures that every exact transform is an ℓ^∞ contraction, keeping intermediate entries bounded without rescaling across dimensions.

Definition 3.2 (Normalized transforms)**.** For an integer $R \ge 1$, the normalized positive one-dimensional DFT $F_R \colon \mathbb{C}^R \to \mathbb{C}^R$ is $(F_R x)_u = \frac{1}{R}\sum_{j\in[R]} x_j e^{2\pi i j u/R}$. For $\mathbf{t} = (t_1, \dots, t_d)$ with $R = t_1 \cdots t_d$, the normalized positive tensor DFT $F_{\mathbf{t}} = F_{\mathbf{t}}^{+} = F_{t_1,\dots,t_d}$ acts on arrays indexed by $[t_1] \times \cdots \times [t_d]$ via

$$(F_{\mathbf{t}} U)_{u_1,\dots,u_d} = \frac{1}{R} \sum_{j_1,\dots,j_d} U_{j_1,\dots,j_d} \exp\Big(2\pi i \sum_{i=1}^{d} \frac{j_i u_i}{t_i}\Big).$$

By the triangle inequality, $\|F_R\|_\infty \le 1$ and $\|F_{\mathbf{t}}\|_\infty \le 1$. The cyclic convolution $(a \circledast b)_j = \sum_{l\in[R]} a_l b_{(j-l) \bmod R}$ satisfies $F_R(a \circledast b) = R\,(F_R a) \odot (F_R b)$, where $\odot$ denotes entrywise multiplication. For real vectors $a, b \in \mathbb{R}^R$ and $z = a + ib$, the *two-call identity* (cf. [MS]) recovers $a \circledast b$ from two forward transforms of length R:

$$a \circledast b = -\frac{R^2}{2} \operatorname{Im}\Big(F_R\big(\overline{(F_R z) \odot (F_R z)}\big)\Big). \tag{8}$$

Indeed, $\sum_{u\in[R]} e^{2\pi i(j-k)u/R}$ equals R for $j = k$ and 0 otherwise, so $F_R(\overline{F_R w}) = R^{-1}\overline{w}$ for every $w \in \mathbb{C}^R$. Applying this relation to $w = z \circledast z = (a \circledast a - b \circledast b) + 2i(a \circledast b)$, for which $(F_R z) \odot (F_R z) = R^{-1} F_R w$, gives $F_R(\overline{(F_R z) \odot (F_R z)}) = R^{-2}\overline{w}$, whose imaginary part is $-2R^{-2}(a \circledast b)$.

For completeness, and only for the Bluestein comparison in Appendix C, we also record the synthetic transforms of [OAI26, §6]; a first-time reader may skip this paragraph. Over the negacyclic ring $\mathcal{R}_r = \mathbb{C}[y]/(y^r + 1)$ with $r = 2^\ell \geq 2$, multiplication by y is an ℓ^∞ isometry. For any power of two $t \mid 2r$, setting $\omega_t = y^{2r/t} \in \mathcal{R}_r$ defines contractions $F_t^{\mp} \colon \mathcal{R}_r^t \to \mathcal{R}_r^t$ by $(F_t^{\mp} z)_j = t^{-1} \sum_{k<t} \omega_t^{\mp jk} z_k$ satisfying $F_t^+ F_t^- = t^{-1} I$. With B_t denoting $(\lg t)$-bit binary reversal on $[t]$, in-place decimation in frequency and its reverse compute $B_t F_t^-$ and $F_t^+ B_t^{-1}$. For $\mathbf{t}' = (t_1, \ldots, t_{d-1})$ with $t_i \mid 2r$, we write $F_{\mathbf{t}'}^{\pm} = \bigotimes_{i=1}^{d-1} F_{t_i}^{\pm}$ and $B = \bigotimes_{i=1}^{d-1} B_{t_i}$. Our direct complex tensor FFT in Section 6 uses no ring products or synthetic transforms in $\mathcal{R}_r$, keeping $\mathcal{R}_r$ only to specify the contiguous r-coefficient record format.

Motivation from two-dimensional fractional shifts. In dimension two ($R = t_1 r$), Section 2.3 read the Cooley–Tukey twiddle factor as a fractional shift of row j_1 by j_1/t_1 [MS] and spread each row by a sinc-Gaussian filter. Applying Poisson summation then relates $F_{t_1,r}$ of the spread array, on the same $t_1 \times r$ grid, to a cyclic alias mixture of $F_R x$ along each column in which two aliases dominate. With a real kernel the two retained alias weights coincide at the Nyquist column $u_2 = r/2$; [MS] handles this by a separate alternating-sum identity, and the coincidence prevents cycle-length-independent conditioning in higher dimensions. Section 4 removes both obstacles for general $d \geq 2$ via a negative half-bin carrier and descending axis spreading.

3.4 Ingredients from [OAI26]

This subsection states the two results from [OAI26] that our written proofs use as black boxes, and summarizes the general-parameter tape procedures derived from them in Appendix A. Appendix D restates the borrowed statements in full and records the symbol-renaming map from [OAI26]: its prefix range P becomes A, its fixed radix q becomes q_{rad}, and its box size T corresponds to R.

First, Fact 3.3 packages five linear-time sequential tape-stream primitives on arrays of bit volume V. These primitives cover digit permutations, stream splits and merges, cyclic row distributions into W role streams, prime-radix modular shifts, and exact fixed-width binary arithmetic.

Fact 3.3 (Elementary tape operations [OAI26, Lemmas 3–4])**.** *Fix an integer radix $q_{\mathrm{rad}} \geq 2$. On a stored array of logical bit volume V on a fixed number of finite-alphabet tapes, the following operations run in $O(V)$ steps (including counter initialization, head resets, and clearing visited workspace):*

(1) applying any fixed permutation to the pair of digit values at two specified radix-q_{rad} address positions, or moving one specified radix-q_{rad} digit position to another while preserving the order of intervening positions;

(2) splitting an array on a specified radix-q_{rad} digit, applying a fixed pointwise map to the aligned streams, and merging them back;

(3) within each prefix of an array on $[A] \times [R_{\mathrm{row}}] \times [B]$, distributing whole rows cyclically into a fixed number W of role streams and, when $W \mid R_{\mathrm{row}}$, merging them in any fixed order of row classes;

(4) when q_{rad} is prime and a target field $y \in [q_{\mathrm{rad}}^b]$ follows control fields of $O(b)$ digits each across arbitrary spectators, shifting $y \mapsto y + a(\mathrm{controls}) \pmod{q_{\mathrm{rad}}^b}$ for any offset computable in $b^{O(1)}$ time (in particular any fixed rational linear combination with denominators coprime to q_{rad}), or multiplying y modulo q_{rad}^b by a fixed rational unit prime to q_{rad};

(5) exact fixed-width signed binary addition, subtraction, sign change, bit-order reversal within a component, and multiplication or division by a power of two when guard bits prevent overflow or inexact truncation.

Second, Fact 3.4 states the finite algebraic certificates of the two fixed linear circuits at $h = 100$ and $m = h^3 = 10^6$. The strict inequalities $s_b < W_b m$ and $s_c < W_c m$, which say that each network's recursive rank sum falls short of the trivial product of its role count and the dimension m, are the sole source of the sub-$n \log n$ exponent saving.

Fact 3.4 (Finite linear networks with a rank deficit [OAI26, Props. 7–8])**.** *At ground-set parameter $h = 100$ and dimension $m = h^3 = 10^6$, there exist two fixed finite linear circuits (over the three-element subsets of $[h]$) with the following certified role counts, recursive rank sums, and endpoint actions for arbitrary initial values on all auxiliary scratch roles:*

(a) Bit shear network*: a fixed pointwise XOR circuit on W_b roles equipped with rational $m \times m$ terminal and gate matrices whose completed scalar action is a role permutation ρ_b satisfying $M_{\mathrm{out}(\rho_b(w))} - M_{\mathrm{in}(w)} = I_m$ for every input role w (implementing the common address shear $(H, D) \mapsto (H + D, D)$ on m-tuples of address fields). Its role count W_b and total edge-difference rank $s_b = \sum_e \mathrm{rank}_{\mathbb{Q}}(M_{\mathrm{head}(e)} - M_{\mathrm{tail}(e)})$ are*

$$W_b = 177\,176\,569\,091\,445\,000\,000, \qquad s_b = 177\,176\,569\,088\,785\,861\,287\,000\,000 < W_b m,$$

giving relative rank deficit $\eta_b = (W_b m - s_b)/(W_b m) = 339/22\,587\,335\,000\,000$.

(b) Complex phase network*: a fixed scalar linear circuit on W_c roles with coefficients in $\{0, \pm 1, \pm\frac{1}{2}\}$ and nested binary subspace labels in $\mathbb{F}_2^m$ whose edge-residual orthonormal bases have total dimension s_c, implementing $C^{\otimes m}$ with $C = \frac{1+i}{2} I + \frac{1-i}{2} X$ on every role after prescribed source/sink diagonal phases and bank restoration, with*

$$W_c = 1\,873\,807\,244\,643\,542\,670\,000, \qquad s_c = 1\,873\,807\,244\,636\,671\,267\,308\,000\,000 < W_c m,$$

giving relative rank deficit $\eta_c = (W_c m - s_c)/(W_c m) = 73/19\,906\,842\,167\,500$.

What the written proof depends on. No result of [HvdH21] is used anywhere in our proofs. Our written proof of Theorem 1 imports only Facts 3.3 and 3.4, corresponding to [OAI26, Lemmas 3–4 and Props. 7–8], as black boxes. Every other step is proved in Sections 4–7 or re-derived in Appendix A by following [OAI26, §§4–5] with all exponents and guard parameters exposed. Lemmas 15–21 and Proposition 18 of [OAI26], covering synthetic ring transforms, chunk-swap layout, packed products, ring convolution, Gaussian resampling, and Bluestein's chirp, are unused on the main route and appear only for comparison in Appendix C. The Lean 4 proof of Theorem 2 uses neither black box, constructing and verifying both $h = 100$ networks, their algebraic certificates, and all Turing machine stream procedures from scratch.

Parameter-exposed tape procedures. As printed, [OAI26, Lemma 12 and Prop. 14] fix $\tau = \sigma = 1 - 2^{-50}$, $\beta = \frac{1}{2}$, and $C_1 = 20$. In Appendix A we follow [OAI26, §§4–5] with all structural parameters exposed, deriving two procedures from Facts 3.3 and 3.4 together with Lemma A.5 for long-record address scans:

- *Arbitrary-width interchange* (Lemma A.8): for every fixed rational $\tau \in (0, 1)$ with $s_b/W_b < m^\tau$, two disjoint u-bit address fields are interchanged across arbitrary spectator fields in $O(Vu^\tau)$ steps, and in $O(V(1 + u^\tau))$ steps when their widths differ by one bit.
- *Simultaneous butterfly layer* (Proposition A.11, via packed selected-bit addition in Lemma A.9). Fix rational exponents $\tau, \sigma, \beta, c, \varepsilon, \lambda, \lambda'$ and an integer guard exponent $C_1 \geq \nu_{\mathrm{net}} + 3 = 14$ with $\varepsilon C_1 < 1$ satisfying the network bounds and admissibility constraints (51)–(52) of Proposition A.11, with $B_{\mathrm{net}}, \nu_{\mathrm{net}}, C_0$ as in (47). Under the size bands (53) with $K = \lfloor d^c \rfloor$, the layer applies H_0 simultaneously to one common bit offset of $D \leq d$ consecutive K-bit address chunks on a disk array

of $\mathcal{R}_r$-records at grid precision q. It evaluates the exact layer H inside $\Delta = \lceil C_0 d^{C_1} \rceil = o(q)$ internal guard bits, truncates once, and returns the disk array $Q_q(Hz)$ with $\|Q_q(Hz) - Hz\|_\infty < \sqrt{2} \cdot 2^{-q}$ in $O(Vd^{\lambda'})$ steps.

Using the rational bound $\ln 10^6 < 14$ in place of $\ln 10^6 < 20$, Proposition A.2 certifies the sharper network exponents $\tau = 1 - 2^{-40}$ and $\sigma = 1 - 2^{-42}$ from the integers in Fact 3.4. Independently, the base-10 comparison $m^{10} < B_{\text{net}} < m^{11}$ gives $\nu_{\text{net}} = 11$, certifying guard exponent $C_1 = \nu_{\text{net}} + 3 = 14$ as noted in [OAI26, §5.4].

4 The fractional-shift bridge

This section builds the fractional-shift bridge, the steps Spread, TensorFFT, and Deconvolve of Algorithm 1 (Section 2.1), which reduces a one-dimensional discrete Fourier transform F_R to a d-dimensional tensor transform $F_{\mathbf{t}}$ on the same power-of-two grid $R = t_1 \cdots t_d$. Before the tensor transform, each axis above the first is smoothed by a shifted complex sinc-Gaussian filter; afterwards, for each smoothed axis, a two-term cyclic recurrence in the frequency domain undoes that smoothing to working precision. For Theorem 1, this reduction avoids prime transform lengths and grid expansion, and sets the linear working-precision budget $P = O(p + \lg R)$ used throughout the multiplier. A first-time reader willing to treat the bridge as a black box needs only Definition 4.5 for the middle-transform contract, Theorem 4.7 for the bridge precision contract, and Lemma 4.8 for the axis-exposure bound, and may skip directly to Section 5.

Sections 4.1–4.3 carry out the continuous and exact Fourier analysis. We spread the input along axes $a = d, d-1, \ldots, 2$ in descending order using a negative half-bin phase carrier, and the triangular alias identity (Lemma 4.2) expresses the tensor DFT of the spread array as an ordered product $\mathcal{A}_2 \cdots \mathcal{A}_d F_R x$ of non-commuting alias operators. Lemma 4.4 then approximates each $\mathcal{A}_a$ to within $0.120 \cdot 2^{-\omega}$ by a two-term cyclic operator B_a whose inverse has ℓ^∞ norm at most $4t_a/7 \le t_a$, uniformly in its cycle length A_a. Applying $B_2^{-1}, \ldots, B_d^{-1}$ in that order therefore recovers $F_R x$ up to the omitted-alias error bounded in (23). Two converse results in Appendix B, Propositions B.1 and B.2, show that within this two-alias framework both the negative sign of the half-bin carrier and a complex kernel are required for uniform bounds.

Sections 4.4–4.5 turn this analytic reduction into a fixed-point algorithm and prove its error and bit-width bounds in Lemma 4.6 and Theorem 4.7. The rational-series evaluation of the filter and phase tables is deferred to Lemma B.3 in Appendix B. Finally, Section 4.6 gives the $O(d)$ direct axis-exposure schedule on multitape machines in Lemma 4.8.

4.1 Index conventions and the carrier

This subsection fixes the mixed-radix indexing of inputs and frequencies on the tensor grid and defines the shifted complex filter used on each spread axis.

Parameters and index conventions. Let $R = t_1 \cdots t_d = 2^k$ with every $t_i \ge 2$ a power of two, where k denotes $\lg R$ throughout the paper. Put $A_a = \prod_{i<a} t_i$ for $1 \le a \le d+1$, so that $A_1 = 1$ and $A_{d+1} = R$. We write input indices as $j = \sum_{i=1}^{d} j_i A_i$ with $0 \le j_i < t_i$, making j_1 the least significant digit, and identify $x \in \mathbb{C}^R$ with the tensor $x_{j_1,\ldots,j_d}$. Frequencies use the reversed digit order:

$$f(\mathbf{u}) = \sum_{i=1}^{d} u_i \frac{R}{A_{i+1}}, \qquad 0 \le u_i < t_i, \tag{9}$$

so u_d is the least significant digit. The map $\mathbf{u} \mapsto f(\mathbf{u})$ is a bijection onto $[R]$, and we write $u_a(f)$ for the a-th frequency digit of $f \in [R]$. For $2 \le a \le d$, the *fractional shift* of axis a is

$$\delta_a(\mathbf{j}) = \frac{\sum_{i<a} j_i A_i}{A_a} \in [0,1), \tag{10}$$

which depends only on the input digits below a. In a multi-axis Cooley–Tukey decomposition of F_R, the twiddle factor coupling axis a to the lower axes acts as a fractional shift by $\delta_a(\mathbf{j})$ along axis a, generalizing the two-dimensional observation of [MS].

As in [MS], our base smoothing kernel is a sinc-Gaussian whose Fourier transform G approximates the indicator of $[-\frac{1}{2}, \frac{1}{2}]$. On each spread axis $a \ge 2$, we multiply this base kernel by a negative half-bin phase so that frequency samples move from the grid points u/t_a to the midpoints $(u+\frac{1}{2})/t_a$. The next definition records this modulated carrier and its t_a-periodic folded filter, which supply the weights for axis spreading.

Definition 4.1 (Kernel and carrier)**.** Let $\omega \ge 8$ be the alias width, and fix α satisfying

$$2\sqrt{\omega \ln 2/\pi} \le \alpha \le 4\sqrt{\omega \ln 2/\pi}, \tag{11}$$

with default $\alpha = 2\sqrt{\omega \ln 2/\pi}$. Put $\alpha_\pi = \sqrt{\pi}\,\alpha$, so $\alpha_\pi \ge 2\sqrt{\omega \ln 2}$. With $\operatorname{sinc}(s) = \sin(\pi s)/(\pi s)$ for $s \ne 0$, $\operatorname{sinc}(0) = 1$, and Fourier transform $\hat{\phi}(\xi) = \int_{\mathbb{R}} \phi(s) e^{-2\pi i \xi s}\, ds$, the transform of $\operatorname{sinc}(s)\, e^{-\pi s^2/\alpha^2}$ is

$$G(\xi) = \int_{\xi-1/2}^{\xi+1/2} \alpha e^{-\pi \alpha^2 s^2}\, ds = \tfrac{1}{2}\big[\operatorname{erf}\big(\alpha_\pi(\xi+\tfrac{1}{2})\big) - \operatorname{erf}\big(\alpha_\pi(\xi-\tfrac{1}{2})\big)\big]. \tag{12}$$

On axis $a \in \{2, \dots, d\}$, the *negative half-bin carrier* is

$$g_a(s) = e^{-\pi i s/t_a}\, \operatorname{sinc}(s)\, e^{-\pi s^2/\alpha^2}, \qquad \hat{g}_a(\xi) = G\Big(\xi + \frac{1}{2t_a}\Big), \tag{13}$$

and its *folded filter* is the t_a-periodic function of $e \in \mathbb{Z}$ given by

$$H_\delta^{(a)}(e) = \sum_{s \in e + t_a\mathbb{Z}} g_a(s+\delta). \tag{14}$$

Because the carrier phase $e^{-\pi i s/t_a}$ is antiperiodic of period t_a rather than periodic, the modulated kernel itself must be folded as in (14). Since $e^{-\pi i(s+\delta)/t_a} = (-1)^{(e-s)/t_a} e^{-\pi i(e+\delta)/t_a}$ for $s \in e + t_a\mathbb{Z}$,

$$H_\delta^{(a)}(e) = e^{-\pi i(e+\delta)/t_a} \sum_{s \in e + t_a\mathbb{Z}} (-1)^{(e-s)/t_a}\, g_0(s+\delta), \qquad g_0(s) = \operatorname{sinc}(s)\, e^{-\pi s^2/\alpha^2}. \tag{15}$$

Periodizing g_0 first and multiplying by the carrier afterwards would drop the alternating signs $(-1)^{(e-s)/t_a}$. Identity (15) also shows that $H_\delta^{(a)}$ can be applied as a real negacyclic convolution between two diagonal phase scans. In Section 4.4, our algorithm evaluates the complex filter directly.

4.2 Triangular alias identity

This subsection shows that spreading the axes in descending order expresses the tensor transform of the spread array as an exact ordered product of one-axis alias operators applied to $F_R x$.

We spread one axis at a time. For $2 \le a \le d$, let $\mathcal{S}_a$ spread axis a while holding all other coordinates fixed:

$$(\mathcal{S}_a V)_{\dots,k_a,\dots} = \sum_{j_a \in [t_a]} V_{\dots,j_a,\dots}\, H^{(a)}_{\delta_a(\mathbf{j})}(j_a - k_a), \tag{16}$$

where $\delta_a(\mathbf{j})$ is read from the coordinates below a of the current array. We apply these operators in the descending order $a = d, d-1, \dots, 2$ and set $U = \mathcal{S}_2\mathcal{S}_3 \cdots \mathcal{S}_d x$. When axis a is spread, coordinates $1, \dots, a-1$ are still the original input digits $j_1, \dots, j_{a-1}$, because they are spread only at later stages. Hence

$$U_{j_1,k_2,\dots,k_d} = \sum_{j_2,\dots,j_d} x_{\mathbf{j}} \prod_{a=2}^{d} H^{(a)}_{\delta_a(\mathbf{j})}(j_a - k_a). \tag{17}$$

In the opposite order, δ_a would be evaluated after lower digits had already been replaced by spread indices, and (17) would fail.

For $2 \le a \le d$, the *alias operator* $\mathcal{A}_a$ on $\mathbb{C}^R$ collects the frequency aliases $f - \nu R/A_a$ for integer shifts $\nu \in \mathbb{Z}$ generated by Poisson summation along axis a, weighted by half-bin samples of G:

$$(\mathcal{A}_a Z)_f = \sum_{\nu \in \mathbb{Z}} G\Big(\frac{u_a(f) + 1/2}{t_a} - \nu\Big)\, Z_{(f - \nu R/A_a) \bmod R}. \tag{18}$$

The weights of $\mathcal{A}_a$ depend only on the digit $u_a(f)$, whereas shifting f by a multiple of R/A_a alters only the more significant digits $u_1, \dots, u_{a-1}$.

In the product $\mathcal{A}_2\mathcal{A}_3 \cdots \mathcal{A}_d$, each factor shifts only frequency digits that no factor to its right reads. The next lemma uses this triangular dependency to express the tensor transform of the spread array U directly in terms of the target transform $Z = F_R x$. Because the alias operators do not commute, their order dictates the left-to-right order in which deconvolution later inverts their truncations.

Lemma 4.2 (Triangular alias identity)**.** *With $U = \mathcal{S}_2\mathcal{S}_3 \cdots \mathcal{S}_d x$ and $Z = F_R x$,*

$$F_{\mathbf{t}} U = \mathcal{A}_2\mathcal{A}_3 \cdots \mathcal{A}_d\, Z, \tag{19}$$

where the operators compose from right to left. Explicitly, for every $\mathbf{u} \in [t_1] \times \cdots \times [t_d]$,

$$(F_{\mathbf{t}} U)_{\mathbf{u}} = \sum_{\boldsymbol{\nu} \in \mathbb{Z}^{d-1}} \Big[\prod_{a=2}^{d} G\Big(\frac{u_a + 1/2}{t_a} - \nu_a\Big)\Big]\, Z_{(f(\mathbf{u}) - \sum_{a=2}^{d} \nu_a R/A_a) \bmod R}, \tag{20}$$

where $\boldsymbol{\nu} = (\nu_2, \dots, \nu_d)$. The operators $\mathcal{A}_a$ do not commute in general, and (19) holds in the stated order.

Proof idea. We unfold the periodic filter on each spread axis and apply Poisson summation to express $(F_{\mathbf{t}} U)_{\mathbf{u}}$ as an absolutely convergent sum over alias vectors $\boldsymbol{\nu} \in \mathbb{Z}^{d-1}$. Comparing mixed-radix phases modulo 1 identifies each inner sum over the input grid with a shifted entry of $Z = F_R x$, yielding (20). Expanding $\mathcal{A}_2\mathcal{A}_3 \cdots \mathcal{A}_d Z$ from left to right then shows that earlier frequency shifts never modify the frequency digit read by a later operator.

Proof. Explicit formula. For any Schwartz function ϕ and real z, ξ, applying Poisson summation to $s \mapsto \phi(z - s)e^{2\pi i \xi s}$ yields

$$\sum_{e \in \mathbb{Z}} \phi(z - e) e^{2\pi i e \xi} = \sum_{\nu \in \mathbb{Z}} \hat{\phi}(\xi - \nu)\, e^{2\pi i z(\xi - \nu)}. \tag{21}$$

No symmetry of ϕ is required, so (21) applies directly to the complex carriers g_a. Substitute (17) into $F_{\mathbf{t}} U$. For fixed $\mathbf{j}$ and $a \in \{2, \dots, d\}$, unfolding the t_a-periodic filter $H^{(a)}_{\delta_a}$ and applying (21) with $\phi = g_a$, $z = j_a + \delta_a(\mathbf{j})$, and $\xi = u_a/t_a$ gives

$$\sum_{k_a \in [t_a]} H^{(a)}_{\delta_a}(j_a - k_a)\, e^{2\pi i k_a u_a / t_a} = \sum_{\nu_a \in \mathbb{Z}} \hat{g}_a\Big(\frac{u_a}{t_a} - \nu_a\Big)\, e^{2\pi i (j_a + \delta_a)(u_a/t_a - \nu_a)}.$$

By the Gaussian decay of g_a and $\hat{g}_a$, all sums converge absolutely and may be interchanged:

$$(F_{\mathbf{t}}U)_{\mathbf{u}} = \sum_{\boldsymbol{\nu}\in\mathbb{Z}^{d-1}} \Big[\prod_{a=2}^{d} \hat{g}_a\Big(\frac{u_a}{t_a} - \nu_a\Big)\Big] \frac{1}{R} \sum_{\mathbf{j}} x_{\mathbf{j}} \exp\Big(2\pi i\Big[\frac{j_1 u_1}{t_1} + \sum_{a=2}^{d} (j_a + \delta_a)\Big(\frac{u_a}{t_a} - \nu_a\Big)\Big]\Big).$$

Since $A_i/A_{a+1} \in \mathbb{Z}$ for $i > a$, we have $j/A_{a+1} \equiv (j_a + \delta_a)/t_a \pmod 1$ and $j/A_a \equiv \delta_a \pmod 1$. Moreover, $j/t_1 \equiv j_1/t_1 \pmod 1$ and $\nu_a j_a \in \mathbb{Z}$. The phase in brackets is therefore congruent modulo 1 to

$$\frac{j}{R}\Big(\sum_{i=1}^{d} u_i \frac{R}{A_{i+1}} - \sum_{a=2}^{d} \nu_a \frac{R}{A_a}\Big),$$

so the inner normalized sum over $\mathbf{j}$ equals $Z_{(f(\mathbf{u}) - \sum_{a=2}^{d} \nu_a R/A_a) \bmod R}$. Substituting $\hat{g}_a(u_a/t_a - \nu_a) = G((u_a + \frac{1}{2})/t_a - \nu_a)$ yields (20).

Product form. Subtracting $\nu R/A_a$ from f modifies only the digits $u_1, \dots, u_{a-1}$, because R/A_a is the positional weight of u_{a-1} in (9) and borrows propagate toward u_1. The lower digits $u_a, \dots, u_d$ remain unchanged. When $(\mathcal{A}_2\mathcal{A}_3 \cdots \mathcal{A}_d Z)_{f(\mathbf{u})}$ is expanded from left to right, every frequency shift accumulated before the weight of $\mathcal{A}_a$ is evaluated comes from factors $\mathcal{A}_{a'}$ with $a' < a$, which leave the digit u_a invariant. Every weight in the expansion is therefore evaluated at the original digit u_a of $f(\mathbf{u})$, matching the right-hand side of (20). In the reverse order a shift from $\mathcal{A}_{a+1}$ can alter u_a before $\mathcal{A}_a$ reads it, so the factors do not commute. □

Consequently, deconvolution must invert the approximate factors from left to right: first B_2^{-1}, then B_3^{-1}, and so on through B_d^{-1}, where B_a is the two-alias truncation of $\mathcal{A}_a$ analyzed in Section 4.3. The next remark illustrates why reversing this order fails already in dimension three, the smallest case with two alias operators.

Remark 4.3 (Non-commutativity in dimension three). Take $\mathbf{t} = (2, 2, 2)$, so $R = 8$, $A_2 = 2$, $A_3 = 4$, $j = j_1 + 2j_2 + 4j_3$, and $f = 4u_1 + 2u_2 + u_3$, with retained aliases $f - 4\nu_2 - 2\nu_3$ ($\nu_2, \nu_3 \in \{0, 1\}$). For $x = e_3$ ($(j_1, j_2, j_3) = (1, 1, 0)$), we have $\delta_2 = \frac{1}{2}$, $\delta_3 = \frac{3}{4}$, and $Z_f = \frac{1}{8} e^{2\pi i\, 3f/8}$. At $\mathbf{u} = (1, 0, 1)$ ($f = 5$), the four retained aliases $(\nu_2, \nu_3) = (0, 0), (1, 0), (0, 1), (1, 1)$ land at frequencies $5, 1, 3, 7$, with values $1, -1, i, -i$ times Z_5; with $c_0 = G(\frac{1}{4})$ and $c_1 = G(\frac{3}{4})$, axis 2 contributes weights (c_0, c_1) and axis 3 weights (c_1, c_0), so up to omitted aliases (20) gives $\frac{c_0 - c_1}{8\sqrt{2}}((c_0 + c_1) + i(c_0 - c_1))$. Similarly, writing $c_{a,\nu}(u) = G((u + \frac{1}{2})/t_a - \nu)$ for the weights of the two-term operator B_a in (22) below and taking Z to be the unit vector at frequency 6, evaluating $B_2 B_3 Z$ and $B_3 B_2 Z$ at frequency 0 gives $c_{2,0}(0)c_{3,1}(0)$ and $c_{3,1}(0)c_{2,0}(1)$, which differ since $c_{2,0}(0) = G(\frac{1}{4}) > G(\frac{3}{4}) = c_{2,0}(1)$ and $c_{3,1}(0) > 0$.

4.3 Two-term cyclic deconvolution

This subsection replaces each infinite alias operator $\mathcal{A}_a$ by a two-term cyclic operator B_a and proves that B_a is well conditioned uniformly in its cycle length.

Because each $\mathcal{A}_a$ sums over all $\nu \in \mathbb{Z}$, the algorithm inverts its two-alias truncation. For $2 \le a \le d$, the *two-term cyclic operator* B_a retains the terms $\nu \in \{0, 1\}$ of (18): with $y = (u_a(f) + \frac{1}{2})/t_a \in (0, 1)$,

$$(B_a Z)_f = c_0 Z_f + c_1 Z_{(f - R/A_a) \bmod R}, \qquad c_0 = G(y), \quad c_1 = G(y - 1). \tag{22}$$

Each orbit of $f \mapsto (f - R/A_a) \bmod R$ has length A_a, and the digit $u_a(f)$ is constant along every orbit. Thus B_a decomposes as a direct sum of cyclic bidiagonal operators $B\colon Z_j \mapsto c_0 Z_j + c_1 Z_{j-1}$ on cycles of length $L = A_a$, each with constant coefficients (c_0, c_1). In the balanced shape of Section 4.4, A_a grows up to R/r, far larger than t_a, so the conditioning of B^{-1} must be independent of the cycle length L.

Because the negative half-bin carrier places every sample $y = (u + \frac{1}{2})/t_a$ strictly inside $(0, 1)$, all omitted aliases lie at distance greater than $\frac{1}{2}$ from the Gaussian center. Since t_a is even, no sample hits the Nyquist midpoint $y = \frac{1}{2}$ where c_0 and c_1 would tie. The next lemma turns these two geometric margins into a uniform $O(2^{-\omega})$ truncation bound on $\mathcal{A}_a - B_a$ and a cycle-length-independent bound $\|B_a^{-1}\|_\infty \le 4t_a/7$, ensuring that inverting $B_2, \dots, B_d$ recovers $F_R x$ up to the omitted-alias error of order $(d-1)(R/t_1)2^{-\omega}\|x\|_\infty$ in (23).

Lemma 4.4 (Negative half-bin kernel and conditioning)**.** *Let $\omega \ge 8$, let α satisfy (11), and let $t = t_a \ge 2$ be a power of two ($2 \le a \le d$). For every frequency digit $0 \le u < t$, with $y = (u + \frac{1}{2})/t$, $c_0 = G(y)$, and $c_1 = G(y - 1)$:*

(i) $\|\mathcal{A}_a\|_\infty \le 1$ *and* $\|B_a\|_\infty \le 1$*;*

(ii) (omitted aliases) $\|\mathcal{A}_a - B_a\|_\infty < 0.120 \cdot 2^{-\omega}$*;*

(iii) (passband) $c_{\max} := \max(c_0, c_1) \ge \frac{1}{2} - 0.060 \cdot 2^{-4\omega} > 0.49999999$*;*

(iv) (separation) $|c_0 - c_1| \ge 7/(4t)$*;*

(v) (conditioning) *on a cycle of any length $L \ge 1$, the operator $B\colon Z_j \mapsto c_0 Z_j + c_1 Z_{j-1}$ is invertible with $\|B^{-1}\|_\infty \le 1/|c_0 - c_1| \le 4t/7$ (with equality in the first bound when L is even); consequently $\|B_a^{-1}\|_\infty \le 4t_a/7 \le t_a$;*

(vi) (product) $\|(B_2 \cdots B_d)^{-1}\|_\infty \le (4/7)^{d-1} R/t_1 \le R/t_1$*, and for $Y = F_{\mathbf{t}} U$,*

$$\|(B_2 \cdots B_d)^{-1} Y - F_R x\|_\infty \le 0.120\,(d-1)\,\frac{R}{t_1}\,2^{-\omega}\,\|x\|_\infty. \tag{23}$$

Proof idea. Parts (i)–(iii) bound the row sums, omitted Gaussian tails, and passband weight of G by tiling $\mathbb{R}$ with unit intervals and using the convexity of the complementary error function. Part (iv) lower-bounds the weight gap $|c_0 - c_1|$ by differentiating $G(\frac{1}{2} - e) - G(\frac{1}{2} + e)$ and using the half-bin distance $|y - \frac{1}{2}| \ge 1/(2t)$ from the midpoint. Parts (v)–(vi) invert each cyclic bidiagonal block via a finite Neumann series whose row sum cancels the cycle-closure denominator, and then telescope the product error across axes.

Proof. (i) Norms. The intervals $[\xi - \nu - \frac{1}{2}, \xi - \nu + \frac{1}{2}]$ for $\nu \in \mathbb{Z}$ tile $\mathbb{R}$, so $\sum_{\nu \in \mathbb{Z}} G(\xi - \nu) = \int_{\mathbb{R}} \alpha e^{-\pi\alpha^2 s^2}\, ds = 1$ with all terms positive. The absolute row sums of $\mathcal{A}_a$ and B_a are at most $\sum_{\nu \in \mathbb{Z}} G(y - \nu) = 1$ and $c_0 + c_1 < 1$, respectively.

(ii) Omitted aliases. Removing the intervals for $\nu \in \{0, 1\}$ leaves $(-\infty, y - \frac{3}{2}] \cup [y + \frac{1}{2}, \infty)$, so the omitted alias weight at y is $\Psi(y) = \frac{1}{2}\operatorname{erfc}(\alpha_\pi(y + \frac{1}{2})) + \frac{1}{2}\operatorname{erfc}(\alpha_\pi(\frac{3}{2} - y))$, which bounds every absolute row sum of $\mathcal{A}_a - B_a$. Since $\operatorname{erfc}'' > 0$ on $(0, \infty)$, Ψ is convex on $[0, 1]$ and symmetric about $\frac{1}{2}$, whence $\Psi(y) \le \Psi(0)$ for all $y \in (0, 1)$. Using $\operatorname{erfc} z \le e^{-z^2}/(\sqrt{\pi} z)$ for $z > 0$ and $\alpha_\pi/2 \ge \sqrt{\omega \ln 2}$, we obtain for $\omega \ge 8$ that

$$\Psi(0) \le \frac{2^{-\omega}}{2\sqrt{\pi\omega \ln 2}} + \frac{2^{-9\omega}}{6\sqrt{\pi\omega \ln 2}} < 0.120 \cdot 2^{-\omega}.$$

Because every half-bin sample $y = (u + \frac{1}{2})/t$ lies in $(0, 1)$, this bound is uniform in $t \ge 2$.

(iii) Passband. Since G is even and strictly decreasing on $[0, \infty)$ and $\min(y, 1 - y) \le \frac{1}{2}$, we have $c_{\max} \ge G(\frac{1}{2}) = \frac{1}{2}\operatorname{erf}(\alpha_\pi) = \frac{1}{2} - \frac{1}{2}\operatorname{erfc}(\alpha_\pi)$, and $\frac{1}{2}\operatorname{erfc}(\alpha_\pi) \le e^{-\alpha_\pi^2}/(2\sqrt{\pi}\,\alpha_\pi) < 0.060 \cdot 2^{-4\omega}$ for $\omega \ge 8$.

(iv) Separation. For $0 \le e \le \frac{1}{2}$, put $\Delta_G(e) = G(\frac{1}{2} - e) - G(\frac{1}{2} + e)$. Differentiating (12) and using $\pi\alpha^2 = \alpha_\pi^2$ gives

$$\Delta_G'(e) = \alpha\big[2e^{-\alpha_\pi^2 e^2} - e^{-\alpha_\pi^2(1-e)^2} - e^{-\alpha_\pi^2(1+e)^2}\big] = 2\alpha e^{-\alpha_\pi^2 e^2}\big(1 - e^{-\alpha_\pi^2}\cosh(2\alpha_\pi^2 e)\big) > 0$$

on $[0, \frac{1}{2}]$, so Δ_G is strictly increasing on $[0, \frac{1}{2}]$. For $0 \le e \le \frac{1}{4}$, we have $e^{-\alpha_\pi^2}\cosh(2\alpha_\pi^2 e) \le e^{-\alpha_\pi^2(1-2e)} \le e^{-\alpha_\pi^2/2} \le 2^{-2\omega} \le 2^{-16}$, so $\Delta_G'(e) \ge 2\alpha(1 - 2^{-16})e^{-\alpha_\pi^2 e^2}$, and integrating from 0 yields $\Delta_G(e) \ge (1 - 2^{-16})\,\mathrm{erf}(\alpha_\pi e)$. Because $t \ge 2$ is even, the midpoint $(t-1)/2$ is a half-integer, so every $u \in [t]$ satisfies $e = |y - \frac{1}{2}| = |u - \frac{t-1}{2}|/t \ge \frac{1}{2t}$, with $\frac{1}{2t} \le \frac{1}{4}$. By concavity of erf on $[0, \infty)$ and $2/t \le 1$,

$$\mathrm{erf}\Big(\frac{\alpha_\pi}{2t}\Big) \ge \frac{2}{t}\,\mathrm{erf}\Big(\frac{\alpha_\pi}{4}\Big) \ge \frac{2}{t}\,\mathrm{erf}(\sqrt{2\ln 2}),$$

since $\alpha_\pi/4 \ge \frac{1}{2}\sqrt{\omega \ln 2} \ge \sqrt{2\ln 2}$. The tail bound of part (iii) gives $\mathrm{erf}(\sqrt{2\ln 2}) > 1 - \frac{1}{4\sqrt{2\pi \ln 2}} > \frac{22}{25}$, and therefore

$$t\,|c_0 - c_1| \ge t\,\Delta_G\Big(\frac{1}{2t}\Big) > 2\,(1 - 2^{-16})\,\frac{22}{25} > \frac{7}{4}.$$

(v) Conditioning. Set $\rho = \min(c_0, c_1)/c_{\max} < 1$ and let T be the cyclic shift $(TZ)_j = Z_{(j-1) \bmod L}$. If $c_0 > c_1$, then $BZ = Y$ reads $(I + \rho T)Z = Y/c_0$. If $c_1 > c_0$, it reads $(I + \rho T^{-1})Z = T^{-1}Y/c_1$. For $S \in \{T, T^{-1}\}$, the Neumann identity gives $(I + \rho S)^{-1} = (1 - (-\rho)^L)^{-1}\sum_{i<L}(-\rho S)^i$. Since the L powers of S occupy distinct columns in each row, every absolute row sum of $(I + \rho S)^{-1}$ equals $(1 + \rho + \cdots + \rho^{L-1})/|1 - (-\rho)^L|$, which is $1/(1-\rho)$ for even L and $\frac{1-\rho^L}{1+\rho^L} \cdot \frac{1}{1-\rho} \le \frac{1}{1-\rho}$ for odd L. Because $c_{\max}(1-\rho) = |c_0 - c_1|$, part (iv) gives $\|B^{-1}\|_\infty \le 1/|c_0 - c_1| \le 4t/7$. Since B_a is a direct sum of such operators on cycles of even length A_a, we have $\|B_a^{-1}\|_\infty \le 4t_a/7 \le t_a$.

(vi) Product. By submultiplicativity of $\|\cdot\|_\infty$ and part (v), we obtain $\|(B_2 \cdots B_d)^{-1}\|_\infty = \|B_d^{-1} \cdots B_2^{-1}\|_\infty \le \prod_{a=2}^{d}(4t_a/7) = (4/7)^{d-1}R/t_1 \le R/t_1$. Telescoping

$$\mathcal{A}_2 \cdots \mathcal{A}_d - B_2 \cdots B_d = \sum_{a=2}^{d} B_2 \cdots B_{a-1}(\mathcal{A}_a - B_a)\mathcal{A}_{a+1} \cdots \mathcal{A}_d$$

and applying the product bound together with parts (i) and (ii), $\|F_R x\|_\infty \le \|x\|_\infty$, and Lemma 4.2 yields (23). □

Notice that the spread array has the same shape $t_1 \times \cdots \times t_d$ as the input: the bridge expands no axis. Appendix B proves two converse bounds within this two-alias framework. With the positive carrier $e^{+\pi i s/t_a} g_0(s)$ and retained pair $\nu \in \{0, 1\}$, the sample $u = 0$ lands at $y = -\frac{1}{2t_a} \notin (0, 1)$ and loses uniform $O(2^{-\omega})$ alias decay, as shown in Proposition B.1. Likewise, for any real-valued kernel, $|c_0| = |c_1|$ at the Nyquist sample $\xi = \frac{1}{2}$, forcing $\|B^{-1}\|_\infty \ge \Omega(L)$ on cycles of length $L = A_a$ by Proposition B.2.

4.4 The fixed-point bridge algorithm and middle-transform contract

This subsection specifies the five-step fixed-point bridge algorithm on dyadic grids and states the interface contract required of its middle transform.

Parameters. Throughout the remainder of this section, $R = t_1 \cdots t_d = 2^k$ has *balanced shape*: $d \ge 2$, $t_d = r = 2^\ell$ with $\ell \ge 2$, and $t_i \in \{r/2, r\}$ for $1 \le i < d$. For the input x, we write $X = \max(1, \|x\|_\infty)$. We denote by P the internal working precision of the bridge, by $q = P + 2$ its dyadic grid precision, and by $v = 2^{-q}$ the unit of the q-grid $\widetilde{\mathbb{C}}_q$.

The bridge interacts with the multidimensional transform at its core solely through a black-box accuracy and bit-width interface. The next definition states this contract: given a bounded spread tensor on the q-grid, the middle transform must return an approximation to $F_{\mathbf{t}}U$ on the q-grid within $E_{\mathrm{mid}}v$ in natural frequency order.

Definition 4.5 (Middle-transform contract)**.** A *middle transform at precision q with error factor E_{mid} and widening δ_{mid}* is a procedure that accepts a complete tensor U in natural axis order $(j_1, \dots, j_d)$, with the last axis contiguous, entries on $\widetilde{\mathbb{C}}_q$, and $\|U\|_\infty \le 1/8$, and returns an array $\widehat{Y}$ on $\widetilde{\mathbb{C}}_q$ in natural frequency order $(u_1, \dots, u_d)$, with component width at most $q+2+\delta_{\mathrm{mid}}$ $(0 \le \delta_{\mathrm{mid}} \le P)$ and

$$\|\widehat{Y} - F_{\mathbf{t}}U\|_\infty \le E_{\mathrm{mid}}\, v, \qquad E_{\mathrm{mid}} \le 16R(k+1).$$

In our main algorithm, the Walsh middle-transform corollary (Corollary 6.6 in Section 6) supplies a middle transform at precision q with $E_{\mathrm{mid}} = 12\ell$ and $\delta_{\mathrm{mid}} = 0$. It applies to every parameter tuple (q, d, r, K) satisfying the hypotheses of the direct tensor FFT theorem (Theorem 6.5) at precision q. Lemma 7.1 verifies those hypotheses along the root parameter family of Section 7.1, the grid sizes and precisions that the top-level multiplier uses as functions of n. For comparison, the Bluestein middle transform analyzed in Lemma C.4 of Appendix C also satisfies Definition 4.5, with $E_{\mathrm{mid}} = \sqrt{2}\,M(3\ell+1) + 12d + 16$ and $\delta_{\mathrm{mid}} = \lg M$, where $M = R/r$.

Given $x \in \widetilde{\mathbb{C}}^R_{p+2}$ with $\|x\|_\infty \le 2^p$, the bridge computes $\widehat{F}_R x$ in five steps:

(1) *Pre-scaling.* Scan x once to find an integer H with $X \le 2^H < 4X$, set $S = 2^{H+\lceil(\lg(R/t_1))/2\rceil+4}$, and form $x' = x/S$ without rounding on the q-grid.

(2) *Spreading.* Reorder the input digits $(j_d, \dots, j_1)$ into natural tensor order $(j_1, \dots, j_d)$ and spread the axes $a = d, d-1, \dots, 2$ in descending order via (16), exposing each axis a as the innermost axis by Lemma 4.8. Along each exposed line of length t_a, spreading is a complex cyclic convolution with the reflected filter $e \mapsto H^{(a)}_{\delta_a(\mathbf{j})}(-e)$ truncated to unfolded offsets $|s| \le P$, whose taps are generated once per line.

We evaluate each line by overlap-add with signed Kronecker packing (cf. [MS]). First, partition the line into blocks of $W_t = \min(2P+1, t_a)$ entries. For each block, pack its signed real and imaginary numerators $(u_j)_{j<W_t}$, and those of the W_t-tap reflected filter, each into one signed integer $\sum_{j<W_t} u_j B_0^j$ in base $B_0 = 2^{4P+16}$ via a one-pass borrow scan across $(4P+16)$-bit slots. Second, compute four exact real integer products per block, and extract the exact convolution coefficients sequentially in the balanced range $[-B_0/2, B_0/2)$ by subtracting each balanced residue modulo B_0 and shifting right by $4P+16$ bits. Third, accumulate the overlaps at fractional scale $2q$, wrapping positions modulo t_a, and round each completed line entry once to $\widetilde{\mathbb{C}}_q$ by $\mathcal{Q}_q$.

(3) *Middle transform.* Apply a middle transform at precision q satisfying Definition 4.5 to the computed spread array $\widehat{U}$, producing $\widehat{Y}$.

(4) *Deconvolution.* Apply two-pass cyclic streaming solvers for $B_2^{-1}, B_3^{-1}, \dots, B_d^{-1}$ in that order on the array in natural one-dimensional frequency order, as analyzed in parts (iv) and (v) of Lemma 4.6 and Section 4.6.

(5) *Output.* Scale the deconvolved array by the power-of-two factor S without rounding, and round once to $\widetilde{\mathbb{C}}_{p+2}$ by $\mathcal{Q}_{p+2}$.

All precomputed tables of folded taps, solver parameters, and unit phases are faithful approximations in the sense of Section 3.2, certified in part (iii) of Lemma 4.6.

4.5 Component error bounds and the precision contract

This subsection bounds the numerical error and bit cost of each finite-precision bridge stage and combines them into the bridge precision contract (Theorem 4.7).

Before assembling the full bridge, we bound each local source of finite-precision error on the q-grid ($q = P + 2$, $v = 2^{-q}$) for $P \geq 20$, $8 \leq \omega \leq P$, and rational $\lambda_G \in [1, 4]$ in $\alpha^2 = 4\lambda_G \omega \ln 2/\pi$. The next lemma shows that truncating the unfolded filter to $|e| \leq P$ leaves an exponentially small spatial tail. With taps certified within $2v$, Kronecker-packed spreading of a q-grid array of norm at most 1 loses at most $4(P+1)v$ per axis. Rational series evaluate all kernel and phase tables without a transcendental oracle. Moreover, on even cycles with faithfully stored parameters and $v \leq 1/(64t_a)$, the two-pass cyclic solver for B_a^{-1} incurs error at most $32t_a^2 v \max(1, \|Y\|_\infty)$, independently of the cycle length.

Lemma 4.6 (Finite-precision spreading and deconvolution)**.** *Let $P \geq 20$, $q = P + 2$, $v = 2^{-q}$, and $8 \leq \omega \leq P$. Let $\alpha^2 = 4\lambda_G \omega \ln 2/\pi$ with a fixed rational $\lambda_G \in [1, 4]$ (so α satisfies (11)), and let $t = t_a$ ($2 \leq a \leq d$).*

(i) (Filter norms and spatial tail) *For every $\delta \in [0, 1)$, $\|H_\delta^{(a)}\|_2 \leq 1$ and $\|H_\delta^{(a)}\|_1 \leq \sqrt{t}$ over one period $[t]$. Truncating the unfolded kernel to $|e| \leq P$ leaves a spatial tail $\sum_{|e|>P} |g_a(e + \delta)| < 0.10 \cdot 2^{-P} = 0.40\, v$.*

(ii) (Spreading) *With complex taps certified to error at most $2v$, step (2) of Section 4.4 maps any q-grid array z with $\|z\|_\infty \leq 1$ to a q-grid array within $4(P+1)v$ of $\mathcal{S}_a z$. Each line of length t uses at most $2t/W_t$ blocks ($W_t = \min(2P + 1, t)$), with four exact products of integers of at most $W_t(4P + 16) + 1$ bits per block and $O(tP)$ bit operations for packing, extraction, and exact accumulation.*

(iii) (Tables and phases) *For every integer $b \geq \max(8, P)$, each unfolded kernel sample, alias weight c_0, c_1, solver parameter $c_{\max}^{-1}$ and $\rho = \min(c_0, c_1)/c_{\max}$, and rational-angle unit phase used by the bridge can be computed deterministically with certified error at most 2^{-b} in $O((b + k)^3)$ bit operations, without any transcendental oracle. In particular, every truncated folded tap can be stored on $\widetilde{\mathbb{C}}_q$ with certified error below v (in $O(P(P + k)^3)$ bit operations per tap), the solver parameters $\hat{\mu}, \hat{\rho}$ on $2^{-q}\mathbb{Z}$ with $\hat{\rho} \geq 0$ and error below v, every unit phase e as a disk-grid entry $\hat{e} \in \mathbb{D}_q$ with $|\hat{e} - e| < 2v$, and a d-factor tensor chirp in $\mathbb{D}_q$ with error at most $4dv$.*

(iv) (Cyclic solver) *Let $L \geq 2$ be an even cycle length (every bridge cycle length A_a, $a \geq 2$, is even). Suppose $v \leq 1/(64t)$ and the stored parameters satisfy $|\hat{\mu} - c_{\max}^{-1}| \leq v$, $|\hat{\rho} - \rho| \leq v$, $\hat{\rho} \geq 0$, and $|\hat{\gamma} - (1 - \hat{\rho}^L)^{-1}| \leq v$. Then the two-pass cyclic sweep, with every product rounded by $\mathcal{Q}_q$, returns an approximation $\widehat{Z}$ to $Z = B^{-1}Y$ satisfying*

$$\|\widehat{Z} - Z\|_\infty \leq 32t^2\, v \max(1, \|Y\|_\infty),$$

and every intermediate state has modulus strictly below $7t \max(1, \|Y\|_\infty)$.

(v) (Closure parameter) *Under $v \leq 1/(64t)$ and even $L \geq 2$, computing $\hat{\rho}^L$ from the stored $\hat{\rho}$ by binary powering with truncation at precision $Q = q + \lceil \lg L \rceil + 2\lceil \lg(4t) \rceil + 10$, followed by one guarded division and rounding to $2^{-q}\mathbb{Z}$, produces $\hat{\gamma}$ with $|\hat{\gamma} - (1 - \hat{\rho}^L)^{-1}| \leq v$ using $O(P + \lg L + \lg t)$-bit arithmetic (which is $O(P + k)$ for $L = A_a \leq R$).*

(vi) (Product norm) $\|(B_2 \cdots B_d)^{-1}\|_\infty \leq R/t_1$.

Proof idea. Part (i) bounds the filter norms via Parseval's identity and estimates the truncated Gaussian tail by an exponential integral. Part (ii) combines this tail with tap approximation error and checks that the Kronecker base $B_0 = 2^{4P+16}$ prevents carry wrap during signed overlap-add. Parts (iv) and (v) analyze the two-pass cyclic recurrence for $(I+\hat{\rho}T)^{-1}$. For even cycle length L, the geometric sum $S_L = \sum_{j<L} \hat{\rho}^j$ multiplies the per-step rounding error in the prefix pass and cancels against the closure denominator $1-\hat{\rho}^L$, yielding a bound independent of L.

Proof. (i) Filter norms and spatial tail. For $u \in [t]$, unfolding $H_\delta^{(a)}$ and applying (21) gives $\sum_{e\in[t]} H_\delta^{(a)}(e)e^{-2\pi i e u/t} = \sum_{\nu\in\mathbb{Z}} G(\frac{u+1/2}{t}-\nu)e^{2\pi i\delta(u/t-\nu)}$, whose modulus is at most $\sum_{\nu\in\mathbb{Z}} G(\frac{u+1/2}{t}-\nu) = 1$. Parseval's identity over $\mathbb{Z}_t$ yields $\|H_\delta^{(a)}\|_2 \le 1$, and Cauchy–Schwarz gives $\|H_\delta^{(a)}\|_1 \le \sqrt{t}$. For the spatial tail, $|g_a(s)| \le e^{-\pi s^2/\alpha^2}/(\pi|s|)$. Since $\alpha \le 4\sqrt{\omega \ln 2/\pi}$ and $\omega \le P$, we have $\alpha\sqrt{P\ln 2/\pi} \le (4\ln 2/\pi)\sqrt{\omega P} < P$. Thus on each side of $[-P,P]$, the first omitted sample contributes less than $2^{-P}/(\pi P)$. Substituting $z = \pi s^2/\alpha^2$ and using $\int_b^\infty e^{-z}z^{-1}\,dz \le e^{-b}/b$ for $b \ge P\ln 2$ bounds the remaining samples by $\int_P^\infty e^{-\pi s^2/\alpha^2}(\pi s)^{-1}\,ds \le 2^{-P}/(2\pi P\ln 2)$. Summing both sides gives $\sum_{|e|>P}|g_a(e+\delta)| \le (\frac{2}{\pi P}+\frac{1}{\pi P\ln 2})2^{-P} < 0.06\cdot 2^{-P} < 0.10\cdot 2^{-P} = 0.40\,v$ for $P \ge 20$.

(ii) Spreading. Truncating the unfolded kernel to $|s| \le P$ leaves at most $2P+1$ effective taps, folded modulo t when $t < 2P+1$. Each tap is certified to error at most $2v$, while the omitted spatial tail adds at most $0.40\,v$ by part (i). Hence the effective filter error has ℓ^1 norm at most $(4P+2.4)v$. Since $\|z\|_\infty \le 1$ and the single final rounding $\mathcal{Q}_q$ adds at most $\frac{\sqrt{2}}{2}v < v$, each output entry differs from $\mathcal{S}_a z$ by at most $(4P+3.4)v < 4(P+1)v$.

For the packing width, $\|z\|_\infty \le 1$, and part (i) gives $\|H_\delta^{(a)}\|_\infty \le \|H_\delta^{(a)}\|_2 \le 1$, so every certified tap has modulus below 2. At base $B_0 = 2^{4P+16}$, each numerator satisfies $|u_j| < 2\cdot 2^q < B_0/2$, so $|\sum_{j<W_t} u_j B_0^j| < B_0^{W_t}/2 < 2^{W_t(4P+16)}$ fits in at most $W_t(4P+16)+1$ signed bits and is formed by a linear borrow scan in $O(W_tP)$ bit operations. Likewise, every real product coefficient c_m and overlap sum at fractional scale $2q$ has magnitude at most $16(2P+1)2^{2q} < 2^{4P+15} = B_0/2$, so $c_m \in (-B_0/2, B_0/2)$ is the unique balanced residue of $\sum_{k\ge m} c_k B_0^{k-m}$ modulo B_0. Reading $(4P+16)$-bit slots from low to high, subtracting c_m, and shifting right by $4P+16$ bits recovers all c_m and accumulates overlaps at scale $2q$ with no wrap or intermediate rounding. Partitioning the line into $\lceil t/W_t\rceil \le 2t/W_t$ blocks gives four exact integer products of width at most $W_t(4P+16)+1$ bits per block, plus $O(tP)$ bit operations for packing, extraction, and accumulation.

(iii) Tables and phases. This is Lemma B.3, proved in Appendix B.

(iv) Cyclic solver. Assume $c_0 > c_1$; the opposite case $c_1 > c_0$ reduces to the same forward recurrence via the reorientation $z_i = Z_{(-i) \bmod L}$ and $y'_i = Y_{(1-i) \bmod L}$, encoding the one-step shift from part (v) of the proof of Lemma 4.4. Let T be the cyclic shift $(TZ)_j = Z_{(j-1) \bmod L}$. Starting from $z = 0$, the first pass computes $b_j = \mathcal{Q}_q(\hat{\mu}Y_j)$ and updates $z \leftarrow b_j + \mathcal{Q}_q(-\hat{\rho}z)$ for $j = 0,\dots,L-1$. It then closes the cycle via $\widehat{Z}_{L-1} = \mathcal{Q}_q(\hat{\gamma}z)$, and a second pass from $z = \widehat{Z}_{L-1}$ computes $\widehat{Z}_0,\dots,\widehat{Z}_{L-2}$ by the same recurrence.

By parts (iii) and (iv) of Lemma 4.4 and $v \le 1/(64t)$, we have $\rho \le 1-7/(4t)$, which implies $0 \le \hat{\rho} \le 1-1/(2t)$, $\|(I+\rho T)^{-1}\|_\infty \le t$, and $\|(I+\hat{\rho}T)^{-1}\|_\infty \le 2t$. Moreover $c_{\max}^{-1} < 2.0000001$, so $|\hat{\mu}| < 2.01$. By the resolvent identity, the exact solution $\tilde{Z} = \hat{\mu}(I+\hat{\rho}T)^{-1}Y$ for the stored parameters $(\hat{\mu},\hat{\rho})$ satisfies $\|\tilde{Z}-Z\|_\infty \le (2t+2.01\cdot 2t^2)v\|Y\|_\infty \le 6t^2 v\|Y\|_\infty$. Each step of the first pass introduces at most two nearest roundings of $\frac{\sqrt{2}}{2}v$ each, adding at most $2v$ in total. Because earlier steps are damped by powers of $\hat{\rho}$, the accumulated prefix error at the end of the pass is at most $2vS_L$ with $S_L = \sum_{j<L}\hat{\rho}^j$. Since L is even, $(I+\hat{\rho}T)^{-1}$ closes with denominator $1-(-\hat{\rho})^L = 1-\hat{\rho}^L$, giving the exact cancellation

$$\frac{S_L}{1-\hat{\rho}^L} = \frac{1}{1-\hat{\rho}} \le 2t.$$

Relative to $\tilde{Z}_{L-1}$, the prefix rounding error therefore contributes at most $2vS_L/(1-\hat{\rho}^L) \le 4tv$ to $\widehat{Z}_{L-1}$. Since the prefix states satisfy $|z| \le S_L(\max_j |b_j| + 2v) < 4.1t \max(1, \|Y\|_\infty)$, the closure error $|\hat{\gamma} - (1-\hat{\rho}^L)^{-1}| \le v$ adds at most $v|z| < 4.1tv \max(1, \|Y\|_\infty)$, and the final $\mathcal{Q}_q$ adds at most v, so $|\widehat{Z}_{L-1} - \tilde{Z}_{L-1}| \le 12tv \max(1, \|Y\|_\infty)$ independently of L. The second pass adds at most $2v/(1-\hat{\rho}) \le 4tv$, giving

$$\|\widehat{Z} - Z\|_\infty \le (6t^2 + 16t)v \max(1, \|Y\|_\infty) \le 32t^2 v \max(1, \|Y\|_\infty).$$

Finally, $\|\tilde{Z}\|_\infty \le |\hat{\mu}| \, \|(I + \hat{\rho}T)^{-1}\|_\infty \|Y\|_\infty < 4.02t\|Y\|_\infty$, and every second-pass state lies within $16tv \max(1, \|Y\|_\infty) \le \frac{1}{4} \max(1, \|Y\|_\infty)$ of $\tilde{Z}$, so all states have modulus strictly below $7t \max(1, \|Y\|_\infty)$.

(v) Closure parameter. Binary powering of $\hat{\rho} \in [0, 1)$ with truncation toward zero at precision Q incurs error at most $2L \cdot 2^{-Q}$ and never exceeds $\hat{\rho}^L$. Since $1 - \hat{\rho}^L \ge 1 - \hat{\rho} \ge 1/(2t)$, both exact and truncated denominators are at least $1/(2t)$, where the derivative of $x \mapsto (1-x)^{-1}$ is at most $4t^2$. The powering error therefore perturbs $(1-\hat{\rho}^L)^{-1}$ by at most $4t^2 \cdot 2L \cdot 2^{-Q} \le v/2048$, and one guarded division followed by nearest rounding to $2^{-q}\mathbb{Z}$ stays within error v. Because $\hat{\gamma}$ is computed from the stored dyadic $\hat{\rho}$ without any exact power oracle, the cancellation $S_L/(1-\hat{\rho}^L) = 1/(1-\hat{\rho})$ in part (iv) holds with no condition relating L to 2^P.

(vi) Product norm. This is part (vi) of Lemma 4.4. □

We now combine the stage bounds of Lemma 4.6 with the truncation estimate of Lemma 4.4 to state the main precision interface of the bridge. For $p \ge \max(8, k)$ and an input x on the $(p+2)$-grid of norm at most 2^p, assume the precision schedule (24) (which makes $P = O(p + \lg R)$), default α, faithful tables, guarded closure parameters, and any middle transform at precision q satisfying Definition 4.5. Then the fixed-point bridge computes $F_R x$ within ℓ^∞ error below $0.18 \cdot 2^{-p} \max(1, \|x\|_\infty)$, with the constant made explicit in (25). Every stored and intermediate number stays within $O(P)$ bits. Later sections invoke the bridge solely through this schedule P, the grid precision $q = P + 2$, and the component width $w = q + 2$.

Theorem 4.7 (Bridge precision contract)**.** *Let $p \ge \max(8, k)$ and let $x \in \widetilde{\mathbb{C}}^R_{p+2}$ satisfy $\|x\|_\infty \le 2^p$, with $X = \max(1, \|x\|_\infty)$. Set*

$$\omega = p+k+\lceil \lg d \rceil + 20, \qquad P = 2p+12k+32\lceil \lg(p+k+2) \rceil + 256, \qquad q = P+2, \qquad w = q+2, \tag{24}$$

take $\alpha = 2\sqrt{\omega \ln 2/\pi}$, certify real table entries to error $v = 2^{-q}$ and complex taps and unit phases to $2v$, and compute closure parameters at precision $Q = q + \lceil \lg L \rceil + 2\lceil \lg(4t) \rceil + 10$. Then for every middle transform at precision q satisfying Definition 4.5, the algorithm of Section 4.4 returns $\widehat{F}_R x \in \widetilde{\mathbb{C}}^R_{p+2}$ satisfying

$$\|\widehat{F}_R x - F_R x\|_\infty < \Big(\frac{\sqrt{2}}{8} + 2^{-12} + 2^{-23}\Big) 2^{-p} X < 0.177021 \cdot 2^{-p} X. \tag{25}$$

Specifically, before the final rounding the arithmetic error is at most $2^{-p-12}X$ and the omitted-alias error is at most $2^{-p-23}X$, while the final rounding adds at most $\frac{\sqrt{2}}{8} 2^{-p}$. Every stored and intermediate component has width $O(P)$ bits (Table 4), with $P = O(p + \lg R)$.

Proof idea. From the linear precision schedule (24) we first derive a single master margin inequality (26), which simultaneously discharges all smallness hypotheses of Lemma 4.6. Pre-scaling shrinks the input by $\Theta(\sqrt{R/t_1})$ so that every intermediate spread array stays within norm $1/8$, meeting the input bound of Definition 4.5. Propagating the spreading, middle-transform, and cyclic-solver errors through $B_2^{-1}, \ldots, B_d^{-1}$ yields bounds that do not depend on the cycle lengths A_a, and adding the omitted-alias bound and final rounding gives (25).

Proof. Write $U = \mathcal{S}_2 \cdots \mathcal{S}_d x'$ for the exact spread array of $x' = x/S$ and $\widehat{U}$ for the computed spread array passed to the middle transform.

Margins. Put $z = p + k + 2 \geq 12$ and $b = \lceil \lg z \rceil$. From (24), $P + 1 \leq 44z + 233 \leq 64z$, so $\lg(P+1) \leq b + 6$; also $d \leq k$. Consequently $\log_2(2^{14} d(P+1)R^3 v) \leq 18 + 4k + b - P \leq -p - 12$, since $P - (p + 4k + b + 30) = p + 8k + 31b + 226 > 0$. This master margin

$$2^{14} d(P+1)R^3 v \leq 2^{-p-12} \tag{26}$$

implies $16d(P+1)\sqrt{R/t_1}\, v < \frac{1}{16}$, $E_{\mathrm{mid}} v \leq 16R(k+1)v < \frac{1}{100}$, $v \leq 1/(64 \max_a t_a)$, and $64v \sum_{a=2}^{d} t_a < 1$, while $8 \leq \omega \leq P$, verifying all hypotheses of Lemma 4.6.

Pre-scaling. Let $C = \max(1, \max_j |\mathrm{Re}\, x_j|, \max_j |\mathrm{Im}\, x_j|)$ and $H = 1 + \lceil \log_2 C \rceil$; then $X \leq 2^H < 4X$ and $H \leq p + 1$. With $S = 2^{H + \lceil (\lg(R/t_1))/2 \rceil + 4}$ we obtain $16\sqrt{R/t_1}\, X \leq S < 128\sqrt{R/t_1}\, X$, whence $\|x'\|_\infty \leq 1/(16\sqrt{R/t_1})$. Moreover x' has at most $(p+2) + H + \lceil (\lg(R/t_1))/2 \rceil + 4 \leq 2p + \lceil (\lg(R/t_1))/2 \rceil + 7 \leq q$ fractional bits, so $x' \in \widetilde{\mathbb{C}}_q^R$ is exact.

Spreading. By part (i) of Lemma 4.6, $\|\mathcal{S}_a\|_\infty \leq \sqrt{t_a}$, so every exact partial spread has ℓ^∞ norm at most $\|x'\|_\infty \prod_{a=2}^{d} \sqrt{t_a} = \|x'\|_\infty \sqrt{R/t_1} \leq 1/16$. By part (ii) of Lemma 4.6 and induction over the $d-1$ stages $a = d, \ldots, 2$, the spreading error $E_{\mathrm{spread}} = \|\widehat{U} - U\|_\infty$ satisfies

$$E_{\mathrm{spread}} \leq 4(P+1)v \sum_{a=2}^{d} \prod_{j=2}^{a-1} \sqrt{t_j} \leq 16d(P+1)\sqrt{R/t_1}\, v < \frac{1}{16}$$

by (26). Every computed partial spread therefore has norm at most $\frac{1}{16} + \frac{1}{16} = \frac{1}{8} \leq 1$, maintaining the input bound of part (ii) of Lemma 4.6 at each stage and ensuring $\|\widehat{U}\|_\infty \leq 1/8$ for Definition 4.5.

Middle transform. Applying Definition 4.5 to $\widehat{U}$ and using $\|F_{\mathbf{t}}\|_\infty \leq 1$, $E_{\mathrm{mid}} \leq 16R(k+1)$, and $k + 1 \leq P + 1$, the middle-transform output $\widehat{Y}$ satisfies $\|\widehat{Y} - F_{\mathbf{t}} U\|_\infty \leq E_Y := E_{\mathrm{spread}} + E_{\mathrm{mid}} v \leq 32d(P+1)Rv$ and $\|\widehat{Y}\|_\infty < \frac{1}{8} + \frac{1}{100} < 1$.

Deconvolution. Put $R_a = \prod_{j=2}^{a} t_j$ for $2 \leq a \leq d$ and $R_1 = 1$, so $R_d = R/t_1$. Because $32v \sum_{a=2}^{d} t_a < \ln 2$, induction with part (iv) of Lemma 4.6 and $\|B_a^{-1}\|_\infty \leq t_a$ shows that the input to solver stage a has norm at most $2R_{a-1}$. Stage a therefore introduces local error at most $32 t_a^2 v \cdot 2R_{a-1}$, which the subsequent exact inverses $B_d^{-1} \cdots B_{a+1}^{-1}$ amplify by at most R_d/R_a. Since $t_a^2 R_{a-1}(R_d/R_a) = t_a(R/t_1)$, which does not depend on the cycle lengths A_a, the total solver error satisfies

$$E_{\mathrm{solver}} \leq 64 \frac{R}{t_1} v \sum_{a=2}^{d} t_a \leq 64 d R^2 v.$$

Meanwhile, part (vi) of Lemma 4.6 bounds the amplification of E_Y by $\|(B_2 \cdots B_d)^{-1}\|_\infty \leq R/t_1 \leq R$. The normalized arithmetic error is thus at most $(R/t_1)E_Y + E_{\mathrm{solver}} \leq 96d(P+1)R^2 v$, and after exact scaling by $S < 128\sqrt{R/t_1}\, X \leq 128RX$,

$$E_{\mathrm{arith}} \leq 2^{14} d(P+1)R^3 v X \leq 2^{-p-12} X$$

by (26).

Aliases and output. Applying (23) to x' and rescaling by S bounds the omitted-alias error by $0.120(d-1)(R/t_1)2^{-\omega}X < 0.120 \cdot 2^{-p-20}X < 2^{-p-23}X$, since $(d-1)2^{-\lceil \lg d \rceil} < 1$ and $R/t_1 < 2^k$. Because $X \geq 1$, nearest rounding $\mathcal{Q}_{p+2}$ adds at most $\frac{\sqrt{2}}{2} 2^{-p-2} = \frac{\sqrt{2}}{8} 2^{-p} \leq \frac{\sqrt{2}}{8} 2^{-p} X$. Since $1.4142136^2 > 2$ gives $\sqrt{2} < 1.4142136$, rational arithmetic yields $\frac{\sqrt{2}}{8} + 2^{-12} + 2^{-23} < 0.17702096 < 0.177021$, proving (25). All component widths match Table 4; any middle-transform widening δ_{mid} is stripped only after verifying $\|\widehat{Y}\|_\infty < 1$, which alters no value. □

Table 4: Bit widths of all stored and intermediate quantities inside the fixed-point bridge. Widths follow the precision schedule of Theorem 4.7 at grid precision $q = P + 2$; on the direct tensor FFT route of Section 6, the middle-transform widening is $\delta_{\mathrm{mid}} = 0$.

Object	Width or scale
normalized input x', completed spread arrays	$q + 2 = P + 4$ bits per real/imaginary component
table evaluation (Lemma B.3)	$S = 2b + 8\lceil \lg b \rceil + 64$ fractional bits ($b \le q + O(\lg P)$) plus $O(b + k)$ integer bits; temporaries $\le 3S + O(k)$ fractional bits
exact overlap-add accumulators	fractional scale $2q$, plus $O(\lg P)$ integer bits
packed spread operands	$O(P)$ slots of $4P + 16$ bits
middle-transform output $\widehat{Y}$	$q + 2 + \delta_{\mathrm{mid}}$ bits ($0 \le \delta_{\mathrm{mid}} \le P$)
deconvolution states and prefixes	$q + k + 6 = P + k + 8$ bits
closure powering (Lemma 4.6(v))	$Q = q + \lceil \lg L \rceil + 2\lceil \lg(4t) \rceil + 10$ fractional, $O(\lg t)$ integer bits
final exact scaling by S and output	add $\log_2 S$ to exponent; one rounding $\mathcal{Q}_{p+2}$ to $\widetilde{\mathbb{C}}_{p+2}$

4.6 Axis exposures and streaming layout

This subsection shows how to expose each spread axis on a multitape Turing machine and stream the cyclic deconvolutions without full array transpositions.

Spreading operates on the lines of one axis a at a time, which lie contiguously on the tape only when that axis is innermost. Bubbling each axis inward through $O(d)$ adjacent swaps would cost $O(Vd^2\ell^\tau)$ steps per sweep over the d axes, more than the cost analysis of Section 7 can absorb at our dimension exponent ε. Instead, we exchange axis a directly with the innermost axis in a single call to the arbitrary-width interchange (Lemma A.8). The Walsh positional permutations of Section 5 do not apply here because bridge line operations carry $O(P)$-bit scalar entries rather than length-r polynomial payloads.

To run a line operation along axis a, we swap axis a directly with axis d and swap it back afterwards. The next lemma bounds the tape cost of each exposure and restoration by $O(V(1 + \ell^\tau))$ steps even when the two axis bit-widths differ by one bit, so sweeping across all d axes costs $O(Vd(1 + \ell^\tau))$ steps rather than $O(Vd^2\ell^\tau)$.

Lemma 4.8 (Axis exposures via chunk interchanges)**.** *Let $\tau \in (0, 1)$ be any rational interchange exponent satisfying $s_b/W_b < m^\tau$ in Lemma A.8 (for instance $\tau = 1 - 2^{-40}$ by Proposition A.2, or $\tau = 1 - 2^{-50}$ as in [OAI26, Lemma 12]). Let a complete tensor of logical bit volume V have axis bit-widths in $\{\ell - 1, \ell\}$ ($\ell \ge 2$) and a nonempty record payload. Given the selected axis and the lengths of the collapsed prefix, gap, and suffix fields as a fixed number of binary integers (their computation from the axis list is charged to the caller), any axis can be exposed as the innermost axis, and the original axis order restored afterwards, in $O(V(1 + \ell^\tau))$ steps. A sweep of line operations over all axes uses at most $2(d - 1)$ such exchanges, and reversing the axis order uses at most $\lfloor d/2 \rfloor$ more, so a constant number of sweeps costs $O(Vd(1 + \ell^\tau))$ steps.*

Proof idea. We collapse the axes before the chosen axis a into a prefix field, the axes strictly between a and the innermost axis d into a gap field, and the record payload into a suffix field, as Lemma A.8 requires. When the two axes have equal bit-width, one interchange suffices; when their widths differ by one bit, we split off the extra bit and combine one equal-width interchange with one linear-time single-bit move.

Proof. Collapse all axes preceding the chosen axis a into the prefix field A, all axes strictly between a and the innermost axis d into the gap field G, and the scalar payload into the suffix B of Lemma A.8.

If axes a and d have equal bit-width ℓ or $\ell - 1$, one call to Lemma A.8 exchanges them in $O(V\ell^\tau)$ steps, independently of the size of G.

Now suppose their widths differ by one bit. If the earlier field is longer, write it as (x_0, X) with $|X| = |Y|$, where Y is the later field, and execute $(x_0, X, G, Y) \mapsto (x_0, Y, G, X) \mapsto (Y, G, x_0, X)$. This uses one equal-width interchange of cost $O(V\ell^\tau)$ followed by one single-bit move of cost $O(V)$ from item (1) of Fact 3.3 and item (2) of Lemma A.4, totaling $O(V(1 + \ell^\tau))$. If the later field is longer, execute $(X, G, y_0, Y) \mapsto (y_0, X, G, Y) \mapsto (y_0, Y, G, X)$. Running the inverse sequence after the line operation restores the layout. Axis d requires no exchange, so a full sweep uses at most $2(d - 1)$ exchanges, and reversing the d axes factors into $\lfloor d/2 \rfloor$ disjoint exchanges of the same form. □

Two streaming observations complete the tape implementation of Section 4.4:

- *Deconvolution without transposition.* In natural one-dimensional frequency order, write any frequency $f \in [R]$ as $f = j\,(R/A_a) + s$ with $0 \le j < A_a$ and $0 \le s < R/A_a$. For each fixed s, the entries $j = 0, 1, \ldots, A_a - 1$ form a single cycle of B_a along which $u_a(f)$ is constant, while the tape stores, for $j = 0, 1, \ldots, A_a - 1$ in order, the contiguous row of all $s \in [R/A_a]$.

  On a sequential state tape, we maintain one $O(P)$-bit solver state record for each $s \in [R/A_a]$, holding the current state z and the column parameters $\hat{\mu}, \hat{\rho}, \hat{\gamma}$. Because these parameters depend only on a and u_a, they are replayed rather than recomputed. We scan each data row j in lockstep with the state tape and rewind only the state tape after each row. This costs $O(V)$ tape movement per pass and per axis a, independently of A_a, while the $O(1)$ multiplications per entry are charged in Section 7.3. Columns with $c_0 > c_1$ and $c_1 > c_0$ are processed in two masked passes. For $c_1 > c_0$, linear-time row-block reversals implement the index reflections $j \mapsto (-j) \bmod A_a$ and $j \mapsto (1 - j) \bmod A_a$ from the proof of part (iv) of Lemma 4.6 and undo them afterwards. This lockstep scan relies on Definition 4.5 returning $\widehat{Y}$ in natural frequency order.

- *Amortized tap and table generation.* Each exposed axis a has $R/t_a \le 2R/r$ lines, and generating the $O(P)$ folded taps of a line takes $\mathrm{poly}(P, k)$ bit operations by part (iii) of Lemma 4.6. Even regenerating taps for every line therefore costs $O(dR(P+k)^{O(1)}/r)$ steps, which is $o(RP)$ whenever r exceeds every fixed polynomial in P for large P, as holds along the root family of Lemma 7.1. The same bound applies to the solver and phase tables.

5 Walsh-layer permutations

This section builds an exact multitape procedure that permutes the bit positions of a binary record address without altering any record's payload. Instead of moving records through physical chunk exchanges, we synthesize address permutations algebraically from three ingredients: simultaneous 2×2 Walsh–Hadamard butterfly layers $H_0 = \frac{1}{2}\left(\begin{smallmatrix} 1 & 1 \\ 1 & -1 \end{smallmatrix}\right)$, recordwise fourth-root and sign phases, and dyadic scalings. The fewer than K leftover address positions outside the chunks are handled by elementary stream swaps. For Theorem 1, these exact positional permutations prepare the chunked address layout and restore natural frequency order for the direct tensor FFT of Section 6, eliminating the external chunk-exchange bottleneck of [OAI26]. A first-time reader willing to treat address reordering as a black box needs only the positional-permutation theorem (Theorem 5.8) and the lifting lemma it relies on (Lemma 5.3), and may skip directly to Section 6.

The construction proceeds in three steps. Section 5.1 interleaves three full butterfly layers with recordwise fourth-root phases to apply H_0 selectively to any chosen subset of active bits while acting as the identity on the rest. Section 5.2 sandwiches a recordwise sign diagonal between two selective

layers to execute a batch of disjoint controlled bit flips with zero rounding error. Section 5.3 then factors every positional permutation into six stages of such batched flips.

Throughout this section and Section 6, stored arrays lie on the q-grid $\widetilde{\mathbb{C}}_q = 2^{-q}(\mathbb{Z} + i\mathbb{Z})$, which the bridge of Section 4 instantiates at $q = P + 2$. For arrays of records, $\|\cdot\|_\infty$ denotes the largest complex modulus of a coefficient, and a *disk array* is one with $\|z\|_\infty \le 1$. By (7), componentwise truncation Q_q toward zero to $\widetilde{\mathbb{C}}_q$ satisfies

$$\|Q_q z\|_\infty \le \|z\|_\infty, \qquad \|Q_q z - z\|_\infty < \sqrt{2} \cdot 2^{-q}. \tag{27}$$

We invoke the simultaneous butterfly layer (Proposition A.11), proved in Appendix A from the elementary stream primitives of Fact 3.3 and the fixed triple networks at $h = 100$ and $m = h^3 = 10^6$ of Fact 3.4. The next two paragraphs restate its parameter family and its tape interface at grid precision q.

Parameters of the simultaneous butterfly layer. Fix rational network exponents $0 < \tau < 1$ and $0 \le \sigma < 1$ satisfying $s_b/W_b < m^\tau$ and $s_c/W_c < m^\sigma$. Fix rational parameters $0 < \beta < 1$, $c > 0$, $0 < \varepsilon < 1$, λ, and λ' satisfying $\max\{\tau, \sigma, \tau(1 + c/\beta)\} < \lambda < 1$ and $\max\{\lambda, \sigma + \beta(1 - \sigma)\} < \lambda' < 1$, together with a guard exponent $C_1 \ge 14$ satisfying $\varepsilon C_1 < 1$. Fix comparison-band constants $a_d \le b_d$, $a_r \le b_r$, C_M, C_{desc}, and $C_w > 1$. We consider all grid precisions q above a fixed cutoff such that

$$a_d q^\varepsilon \le d \le b_d q^\varepsilon, \qquad 2^{a_r q/d} \le r \le 2^{b_r q/d}, \qquad K = \lfloor d^c \rfloor,$$

together with integers $1 \le D \le d$, $A, S \ge 1$, and $0 \le \rho < K$ satisfying $M = AS\,2^{KD}$, $\lg(2M) \le C_M q$, and $q + 2 \le w \le C_w q$.

Layer interface and cost. The layer procedure accepts a disk-grid array in lexicographic order on $[A] \times [2^K]^D \times [S]$ together with the self-delimiting header $\Gamma(q)\Gamma(d)\Gamma(r)\Gamma(D)\Gamma(A)\Gamma(S)\Gamma(K)\Gamma(\rho + 1)\Gamma(w)$ of length at most $C_{\mathrm{desc}} q$. Each address carries a length-r record in $\mathcal{R}_r$ stored in coefficient order, where each real or imaginary component is 2^{-q} times a signed w-bit two's-complement integer written most significant bit first. Let H apply H_0 simultaneously to bit ρ, counted from the least significant end, of each of the D active chunks. Proposition A.11 evaluates Hz *exactly* over $\mathbb{Z}[i, 1/2]$ using $\Delta = \lceil C_0 d^{C_1} \rceil = o(q)$ internal guard bits, namely $q + \Delta$ fractional and $\Delta + 3$ integer and sign bits per real component, before applying one final truncation Q_q. It returns $Q_q(Hz)$ in the original record and component format as a disk array satisfying $\|Q_q(Hz) - Hz\|_\infty < \sqrt{2} \cdot 2^{-q}$, in $O(Vd^{\lambda'})$ steps, where $V = 2Mrw$.

5.1 Selective Walsh operations

This subsection shows how three full simultaneous butterfly layers, interleaved with recordwise fourth-root phases, implement H_0 on an arbitrary subset of a slice while leaving the remaining positions unchanged.

Controlled flips require applying H_0 on a chosen subset of a *slice*, the set of D positions selected by one call of the butterfly layer; Definition 5.6 fixes the slices used for positional maps. Positions in a binary record address are numbered from the most significant end, while within each K-bit chunk the offset ρ is counted from the least significant end. Let $S = \mathrm{diag}(1, i)$, and for any set U of record-address positions define the recordwise diagonal

$$(S_U z)(x) = i^{\sum_{j \in U} x_j}\, z(x), \tag{28}$$

which multiplies every coefficient of the record at address x by the same fourth-root phase.

A single butterfly-layer call applies H_0 to all D active positions of a slice at once, whereas controlled flips need H_0 on a subset T and the identity on the complement U. The next lemma achieves this separation algebraically: alternating H_0 three times with the fourth-root gate S collapses $(H_0S)^3$ to a scalar multiple of the identity on U, while H_0^3 remains a scalar multiple of H_0 on T.

Lemma 5.1 (Three-layer selection)**.** *Let a slice consist of D selected positions ($1 \le D \le d$) and let T be any subset of them, with $s = |T|$, U the selected positions of the slice not in T, and $q_{\mathrm{sel}} = |U| = D - s$. Let H_{all} apply H_0 on all D positions and H_T only on those in T. Then*

$$H_0^3 = \tfrac{1}{2}H_0, \qquad (H_0S)^3 = \tfrac{1+i}{4}I, \qquad H_T = 2^s\,[2(1-i)]^{q_{\mathrm{sel}}}\,(H_{\mathrm{all}}S_U)^3. \tag{29}$$

If $q_{\mathrm{sel}} = 2f + e$ with $e \in \{0, 1\}$, then $2^s[2(1-i)]^{q_{\mathrm{sel}}} = 2^{s+3f+e}(-i)^f(1-i)^e$.

Proof. The first identity follows from $H_0^2 = \frac{1}{2}I$, and cubing $H_0S = \frac{1}{2}\left(\begin{smallmatrix} 1 & i \\ 1 & -i \end{smallmatrix}\right)$ gives the second. Operators on different positions commute and tensor independently, so $(H_{\mathrm{all}}S_U)^3$ acts position by position: as $\frac{1}{2}H_0$ on each position in T and as $\frac{1+i}{4}I$ on each position in U. Hence $(H_{\mathrm{all}}S_U)^3 = 2^{-s}(\frac{1+i}{4})^{q_{\mathrm{sel}}}H_T$, and $(\frac{1+i}{4})^{-1} = 2(1-i)$. Finally, $[2(1-i)]^2 = -8i$ gives the last formula. □

In chronological order, the *gadget* of Lemma 5.1 applies $S_U, H_{\mathrm{all}}, S_U, H_{\mathrm{all}}, S_U, H_{\mathrm{all}}$ and then the scalar $2^s[2(1-i)]^{q_{\mathrm{sel}}}$; there are three S_U diagonals, including the initial one. The scalar is applied in one scan: a fourth-root phase, at most one multiplication by $1-i$ via $(\Re z, \Im z) \mapsto (\Re z + \Im z, \Im z - \Re z)$, and one binary shift. Every external diagonal and scalar correction is applied only at completed layer boundaries in the ordinary record layout, after the layer call has restored its role routing and removed its padding. Position masks are kept as caller data on separate tapes.

Each call to Proposition A.11 truncates its output to the active grid precision, which would ordinarily introduce rounding error. On a slice of $1 \le D \le d$ positions, suppose we first lift a disk array on the q-grid to precision $q_* = q + 8d$ by adding $8d$ extra fractional zero bits. The next lemma shows that across two consecutive selective gadgets at precision q_* with the same target set T, even with a sign or fourth-root diagonal between them, every layer output has denominator dividing 2^{q+8d}. All layer truncations therefore discard only zeros, and both gadgets act without error.

Lemma 5.2 (Exact boundaries)**.** *Let the input be a disk array on the q-grid and let a slice have $1 \le D \le d$ selected positions. Put $q_* = q + 8d$, lift the representation exactly to precision q_*, and execute the gadget of Lemma 5.1 by completed calls of Proposition A.11 at precision q_*, each with its internal guard Δ. Then the gadget is exact: every final truncation Q_{q_*} of a layer call discards only zeros, and the output is $H_T z$, on a grid of denominator exponent at most $q + s$. After any diagonal with entries in $\{1, -1, i, -i\}$ (which preserves both the grid and the disk), a second gadget with the same T is again exact, with output on a grid of denominator exponent at most $q + 2s$. All completed outputs are disk arrays.*

Proof idea. Because each D-bit butterfly layer increases the binary denominator exponent by at most $D \le d$ and fourth-root diagonals add no denominator bits, $8d$ extra fractional bits accommodate two three-layer gadgets back to back with room to spare. Inside a gadget, every full layer and every S_U is a contraction, so each layer call again receives a disk array. After the final scalar scan, the disk-array property follows from the exact operator identity H_T on disk inputs rather than from a bound on the intermediate scalar.

Proof. A full normalized D-bit layer has Gaussian dyadic entries with denominator dividing 2^D, and S_U adds no denominator bits. The three layer outputs of the first gadget therefore lie on grids

of denominator exponent at most $q + D$, $q + 2D$, and $q + 3D$, all strictly below $q_* = q + 8d$, so each Q_{q_*} is the identity. The internal computation of each layer is exact by Proposition A.11, whose internal denominators are controlled by Δ. After the exact scalar scan, the value equals $H_T z$, which has denominator exponent at most $q + s$.

A diagonal with entries in $\{1, -1, i, -i\}$ multiplies every coefficient by a unit of $\mathbb{Z}[i]$, preserving both this denominator exponent and the modulus of every coefficient. The second gadget therefore again starts from a disk array, now on denominator exponent at most $q + s$; its three layer outputs have denominator exponents at most $q + s + 3D \le q + 4D < q_*$, and its completed output $H_T(\cdot)$ has exponent at most $q + 2s$. Each full layer and each S_U is a contraction in $\|\cdot\|_\infty$. Moreover, the completed gadget output equals H_T applied to a disk array and is therefore itself a disk array: this operator identity, rather than a bound on the intermediate scalar, verifies the disk-input condition of the next layer call. The scalar scan requires only a constant number of temporary integer bits. □

Before running the lifted gadgets, we pad every stored numerator with $8d$ trailing zero bits and check that the butterfly layer's size bands still hold at the lifted precision $q_* = q + 8d$. The next lemma verifies that this lift costs linear time $O(V)$ and stays inside fixed widened comparison bands.

Lemma 5.3 (Lifting)**.** *Lifting a q-grid array to precision $q_* = q+8d$ costs $O(V)$: append $8d$ fractional zero bits to every numerator and use component width $w_* = w+8d$, so that $q_*+2 \le w_* \le C_w q_*$, and rebuild the header with q_*, w_* and the same d, r, K. Since $d = O(q^\varepsilon)$ with $\varepsilon < 1$, eventually $q_* \le 2q$, and the comparison bands $a_d q^\varepsilon \le d \le b_d q^\varepsilon$ and $2^{a_r q/d} \le r \le 2^{b_r q/d}$ at q imply $(a_d/2) q_*^\varepsilon \le d \le b_d q_*^\varepsilon$ and $2^{(a_r/2) q_*/d} \le r \le 2^{b_r q_*/d}$. Hence one fixed set of widened comparison constants serves both q and q_*.*

Proof. Because $C_w > 1$ and $w \le C_w q$, we have $w_* = w + 8d \le C_w(q + 8d) = C_w q_*$. Appending $8d$ low zero bits represents the exact same dyadic values at scale 2^{-q_*}, whereas changing only the precision field in the header would rescale the array. The internal guarded width $q_* + 2\Delta + 3$ remains $O(q)$, and all re-encoding scans run in time $O(V)$. □

5.2 Exact controlled flips

This subsection combines two selective Walsh gadgets with a recordwise sign diagonal to execute a batch of disjoint controlled address-bit flips without error.

For disjoint pairs (c_j, t_j), $1 \le j \le s$, of distinct record-address positions, let $T = \{t_1, \dots, t_s\}$ and define

$$(\mathrm{CZ}\, z)(x) = (-1)^{\sum_{j=1}^{s} x_{c_j} x_{t_j}}\, z(x). \tag{30}$$

Conjugating a controlled sign change by H_0 on the target bit turns a phase flip into a bit flip. The next lemma records the tensor-product form of this identity across s disjoint control-target pairs, moving whole records between addresses without modifying their coefficients.

Lemma 5.4 (Controlled flips)**.** *As array operators, $\mathrm{CNOT}^{\otimes s} = 2^s H_T\, \mathrm{CZ}\, H_T$, where the left-hand side moves the record at address x to the address obtained by $x_{t_j} \mapsto x_{t_j} \oplus x_{c_j}$ for all $1 \le j \le s$ and changes no coefficient.*

Proof. For a single pair (c_1, t_1), fix the control bit x_{c_1}. If $x_{c_1} = 0$, the target diagonal is I and $2H_0^2 = I$. If $x_{c_1} = 1$, the target diagonal is $\mathrm{diag}(1, -1)$ and $2H_0\, \mathrm{diag}(1, -1) H_0 = \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$. Tensoring over the s disjoint pairs gives the identity. Because the address map is an involution, the move and pull conventions coincide. □

When all target bits of a disjoint controlled-flip batch belong to one slice and the controls lie anywhere else in the record address, both selective H_T operators act on that single slice. Under the hypotheses of Proposition A.11 at grid precision q, the next proposition shows that on disk arrays on the q-grid, six butterfly-layer calls at lifted precision $q_* = q + 8d$ and a constant number of linear scans execute the entire batch. They return an exact permutation of the input records in the original q-grid format in $O(Vd^{\lambda'})$ steps.

Proposition 5.5 (Slice-batched controlled flips)**.** *Assume the hypotheses of Proposition A.11 at grid precision q. Suppose all targets $t_1, \dots, t_s$ of a batch of disjoint controlled flips lie among the selected positions of one slice, while the controls $c_1, \dots, c_s$ lie anywhere else in the record address (including selected positions of the same slice outside T). On disk arrays on the q-grid, the batch is performed exactly in $O(Vd^{\lambda'})$ steps by six completed layer calls at precision $q_* = q + 8d$ and a constant number of linear scans, returning the output in the original q-grid format.*

Proof idea. We lift to precision $q_* = q + 8d$, run the two selective H_T gadgets around the sign diagonal CZ, rescale by 2^s, and strip the $8d$ extra low bits. Lemma 5.2 guarantees that all six layer calls are exact, and Lemma 5.4 guarantees that the completed output is a permutation of the original q-grid records, so stripping the low bits discards only zeros.

Proof. Lift the input from q to $q_* = q + 8d$ by Lemma 5.3, apply the first selective gadget for H_T, the sign diagonal CZ, and the second selective gadget for H_T, multiply without rounding by 2^s, and lower from q_* back to q. Since CZ has entries in $\{\pm 1\}$, Lemma 5.2 applies: all six layer truncations discard only zeros. If a control c_j lies in the same slice outside T, the internal layers of a gadget mix that bit temporarily, but the completed gadget acts on it as the identity before CZ is applied.

By Lemma 5.4, the value after multiplying by 2^s is an exact permutation of the input q-grid records, so lowering from q_* to q discards only zero low bits; this relies on the permutation identity rather than denominator counting alone. No control-before-target ordering is required because CZ is a recordwise value diagonal rather than a controlled address rotation. For each record, the phases of (28) and the parity of (30) are computed once from its address and the caller's mask or pair list in time polynomial in q. By Lemma A.5, that overhead is absorbed by the record length r. Meanwhile, lifting and lowering are elementary streams by Lemma A.4, and the lifted volume remains $O(V)$. Thus Proposition A.11 gives total time $O(Vd^{\lambda'})$. □

Because a completed batch is an exact record permutation on $\widetilde{\mathbb{C}}_q$, it neither amplifies inherited numerical error nor accumulates denominator bits across batches.

5.3 Positional permutations

This subsection partitions the record-address bits into slices and factors every positional address permutation into six stages of slice-batched controlled flips.

To feed arbitrary positional maps into the simultaneous butterfly layer, we divide the m_0 record-address bits into K-bit chunks and group those chunks into consecutive groups of d chunks, the last group possibly shorter. Picking the same bit offset ρ across every chunk in a group defines a slice, while fewer than K leftover positions remain at the end of the address. The next definition fixes this slice geometry and the move convention for positional maps.

Definition 5.6 (Positional maps and slices)**.** A *positional map* on m_0 record-address positions is a permutation π of $\{0, \dots, m_0 - 1\}$ under the move convention: the bit at position j is placed at position $\pi(j)$. It induces an array operator that permutes whole records while leaving each record's coefficient list unchanged; $f \circ g$ applies g first. Put $J = \lfloor m_0/K \rfloor$, partition the first JK positions

into consecutive K-bit chunks, and group those chunks into $\lceil J/d \rceil$ consecutive groups of d chunks each, the last group possibly shorter. For a group starting at position j_0 with D chunks ($1 \le D \le d$) and an offset $0 \le \rho < K$, the corresponding *slice* consists of the positions $j_0 + jK + (K - 1 - \rho)$ for $0 \le j < D$, with complete rectangular layer shape $[2^{j_0}] \times [2^K]^D \times [2^{m_0 - j_0 - DK}]$. The fewer than K remaining address positions are *leftover* positions.

The number of slices is

$$N_{\text{slice}} = K\lceil J/d \rceil \le \frac{m_0}{d} + K. \tag{31}$$

The inequality holds since $\lceil J/d \rceil \le J/d + 1$ and $JK \le m_0$.

Any permutation of bit positions factors into two involutions made of disjoint two-position swaps, and each swap factors into three XOR-based controlled flips. The next lemma records this six-stage decomposition so that within each stage all active control-target pairs are pairwise disjoint.

Lemma 5.7 (Two involutions)**.** *Every positional map is a composition $I_1 \circ I_0$ of two involutions. Each involution is a set of disjoint positional swaps, and each set of disjoint swaps is implemented by three stages of disjoint controlled flips.*

Proof idea. Reflecting each cycle of π across two adjacent axes of symmetry expresses π as the composition $I_1 \circ I_0$ of two involutions made of disjoint transpositions. Each two-bit transposition then expands into the standard three-step XOR swap, keeping distinct pairs disjoint in every stage.

Proof. On a cycle $(i_0, \dots, i_{n_c-1})$ with $\pi(i_j) = i_{j+1}$ and indices taken modulo n_c, define $I_0(i_j) = i_{-j}$ and $I_1(i_j) = i_{1-j}$. Both maps are involutions and $I_1(I_0(i_j)) = i_{j+1}$, so defining them independently on every cycle of π gives $\pi = I_1 \circ I_0$. A swap of two positions b_1, b_2 factors into three controlled flips, $x_{b_1} \leftarrow x_{b_1} \oplus x_{b_2}$, $x_{b_2} \leftarrow x_{b_2} \oplus x_{b_1}$, and $x_{b_1} \leftarrow x_{b_1} \oplus x_{b_2}$. Within each of the three stages, distinct swaps act on disjoint pairs of positions and therefore commute. □

Grouping the controlled flips of each stage by the slice containing their target bit turns Lemma 5.7 into an exact multitape permutation algorithm. The next theorem is the main export of Section 5. Under the parameter family of Proposition A.11 at grid precision q, consider a complete m_0-bit record-address cube of disk-valued q-grid records of r complex coefficients, with $m_0 = O(q)$. When $dK = o(q)$, every positional permutation of the m_0 record-address bits is executed without error in $O(Vqd^{\lambda'-1})$ steps, returning the permuted array in the original q-grid format.

Theorem 5.8 (Positional permutations)**.** *Assume the parameter family of Proposition A.11 at grid precision q. Let a complete m_0-bit record-address cube, with $m_0 = O(q)$, carry disk-valued q-grid records of r complex coefficients, so that $V = 2^{m_0} \cdot 2rw$. For every positional map π on $\{0, \dots, m_0 - 1\}$, a fixed multitape procedure performs the induced record permutation exactly in*

$$O\Big(V\Big[\Big(\frac{m_0}{d} + K\Big)d^{\lambda'} + K\Big]\Big) \tag{32}$$

steps, which is $O(Vqd^{\lambda'-1})$ when $dK = o(q)$. The output is returned in the original q-grid format.

Proof idea. We execute the six disjoint controlled-flip stages of Lemma 5.7 in sequence. Within each stage, flips targeting slice positions are grouped into at most N_{slice} slice batches and run via Proposition 5.5, while the fewer than K flips targeting leftover positions are run directly by linear-time two-digit stream swaps. Because each record has length r, all polynomial-time header and mask bookkeeping is absorbed into $O(V)$.

Proof. Schedule. Execute the three stages of I_0 followed by the three stages of I_1 from Lemma 5.7. Within each stage, partition the controlled-flip pairs according to the slice that contains their target bit; reordering batches within a stage is valid because the pairs of a stage have pairwise disjoint supports. Each slice batch satisfies Proposition 5.5, including pairs whose control bit lies in a leftover position or in another slice. Any pair whose target bit is a leftover position is executed directly by a two-digit stream permutation from item (1) of Lemma A.4. Since there are fewer than K leftover positions, each of the six stages performs fewer than K such permutations, costing $O(VK)$ in total.

Bookkeeping. Cycle decompositions, involution tables, slice masks, and nine-field headers have size polynomial in q and are constructed by polynomially many scans of polynomial-size tables, absorbed into $O(V)$ by the record length via Lemma A.5. Every slice acts on the complete cube of 2^{m_0} records, its two spectator ranges contribute at most $2m_0 + 2$ header bits, and the record length contributes $2 \lg r + 1 = O(q)$ bits, so a single pair of constants C_M, C_{desc} serves every slice call.

Cost. Six stages with at most N_{slice} slice batches per stage and six layer calls per nonempty batch use at most $36N_{\mathrm{slice}}$ completed layer calls. Combining (31) with Proposition 5.5 yields (32). Each layer completes its own padding and cleanup, and each batch returns to the q-grid before the next begins. Internal packed basis changes from Lemmas A.8 and A.9 are already accounted for in $O(Vd^{\lambda'})$. When $m_0 = O(q)$ and $dK = o(q)$, we have $(m_0/d + K)d^{\lambda'} + K = O(qd^{\lambda'-1})$. □

Note that Theorem 5.8 permutes the bit positions of a complete binary record address carrying a long contiguous payload via intermediate linear combinations on $\widetilde{\mathbb{C}}_{q_*}$. It does not execute arbitrary permutations of the 2^{m_0} records or apply to short payloads without a record factor r.

6 Direct tensor FFT

This section implements the middle transform of the fractional-shift bridge directly as a complex tensor FFT $F_{\mathbf{t}}$ on the q-grid. We store the last coordinate as a contiguous record of $r = 2^\ell$ complex coefficients and transform that last axis inside each record. We then run decimation-in-frequency rounds across all $d - 1$ record-address axes in parallel, combining the simultaneous butterfly layers of Proposition A.11 with the exact positional permutations of Theorem 5.8.

For Theorem 1, computing this tensor transform directly over $\mathbb{C}$ achieves additive error below $12\ell \cdot 2^{-q}$ with zero output widening. It needs no Bluestein chirp and no polynomial-ring convolution in $\mathcal{R}_r$, and its error does not grow with the number $M = R/r$ of records, as it does on the Bluestein route of Appendix C. A first-time reader willing to take the middle transform as a black box needs only Theorem 6.5 for the direct tensor FFT and Corollary 6.6 for its bridge interface, and may skip directly to Section 7.

Parameters and conventions. Throughout this section, let

$$t_d = r = 2^\ell, \qquad t_i \in \{2^{\ell-1}, 2^\ell\}\ (1 \le i < d), \qquad R = t_1 \cdots t_d, \qquad M = R/r, \qquad m_0 = \lg M.$$

The last coordinate is stored as the contiguous length-r coefficient list of each record; here $\mathcal{R}_r$ specifies only the record format, and no multiplication in that ring is performed. The exact transform is the normalized positive tensor DFT $F^+_{\mathbf{t}} = F_{\mathbf{t}}$ of Definition 3.2. Stored arrays lie on the q-grid $\widetilde{\mathbb{C}}_q$ with component width $q + 2 \le w \le C_w q$. Scalar integer products are delegated to an independent multiplier of cost $\mathsf{M}(N) = N\,\mathsf{m}(N)$ on N-bit integers, such as Schönhage–Strassen [SS71], and never to a recursive call of the algorithm of this paper.

6.1 Phases and scalar products

This subsection builds the two scalar primitives needed in every FFT round, ensuring that both keep all intermediate complex numbers inside the closed unit disk.

Because the simultaneous butterfly layer requires disk-array inputs at every round, approximate roots of unity cannot exceed modulus 1 by even a single grid unit. The next lemma constructs a $2r$-entry table of $2r$-th roots of unity on the q-grid by rational series evaluation followed by slight inward shrinking. Each entry has modulus at most 1, error below $2 \cdot 2^{-q}$, and deterministic evaluation cost $O(q^3)$ bit operations.

Lemma 6.1 (Disk-valued phases)**.** *For every precision $q \geq 8$ and $r = 2^\ell$ with $\ell \leq q$, the $2r$ phases $\theta_j = e^{\pi i j/r}$, $0 \leq j < 2r$, have deterministically computable approximations $\tilde{\theta}_j \in \widetilde{\mathbb{C}}_q$ satisfying*

$$|\tilde{\theta}_j| \leq 1, \qquad |\tilde{\theta}_j - \theta_j| < 2 \cdot 2^{-q}.$$

The fourth roots of unity are returned exactly. Each entry costs $O(q^3)$ bit operations on dyadic numbers of $O(q)$ bits, on a fixed number of tapes.

Proof idea. Using trigonometric symmetries, we reduce the angle to $[-\pi/4, \pi/4]$, approximate π via Machin's formula, and sum alternating Taylor series for cosine and sine at $S = 2(q+5) + O(\lg q)$ fractional bits. On non-axis angles, multiplying by $1 - 2^{-q-3}$ before truncating toward zero keeps the modulus at most 1 and the total error under $2 \cdot 2^{-q}$.

Proof. Put $u_0 = 2^{-q}$ and $b = q + 5$. We compute with dyadic numbers of $S = 2b + 8\lceil \lg b \rceil + 64$ fractional bits, rounding each product and quotient once with error at most 2^{-S}, as in part (iii) of Lemma 4.6 and Appendix B. Exact products before a rounding use at most $2S$ fractional bits. We evaluate θ_j by rational series to within $u_0/16$, then shrink the result slightly and truncate it into the disk.

Reduction. Reduce the dyadic rational $\rho = j/r$ without error modulo 2 and then, by the symmetries of sine and cosine, to $|\rho| \leq \frac{1}{4}$. The symmetries contribute only a sign, a conjugation, or a factor $\pm i$, which act without error; the four fourth roots of unity are recognized directly from the index.

The angle. Machin's identity $\pi = 16 \arctan \frac{1}{5} - 4 \arctan \frac{1}{239}$, summed to $N = 4S$ terms with one rounded division per term, gives $\tilde{\pi}$ with $|\tilde{\pi} - \pi| \leq 21N2^{-S}$, since the omitted series tails lie below $20 \cdot 5^{-2N-1}$. Because ρ is dyadic with denominator dividing $r = 2^\ell$, the product $x = \tilde{\pi}\rho$ is formed without error on the grid of $S + \ell$ fractional bits and rounded once to S fractional bits, adding at most 2^{-S}. Thus $|x| < 1$ and $|x - \pi\rho| \leq (21N + 1)2^{-S}$.

Series. Sum $\cos x$ and $\sin x$ as the alternating Taylor polynomials $\sum_{n<N} (-1)^n T_n$, with $T_0 = 1$ and $T_n = T_{n-1}x^2/((2n)(2n-1))$ for the cosine, and $T_0 = x$ and $T_n = T_{n-1}x^2/((2n)(2n+1))$ for the sine. Because the step multipliers satisfy $x^2/2 < \frac{1}{3}$, every rounding residue is damped, so each sum has rounding error below $4N2^{-S}$ and tail below $2(1.6\,\mathrm{e}/N)^N$. Since cos and sin are 1-Lipschitz, the resulting complex approximation a satisfies

$$|a - \theta_j| \leq 64N2^{-S} + 4(1.6\,\mathrm{e}/N)^N \leq 2^{-b-4} < \frac{u_0}{16},$$

as $S - b - 4 = b + 8\lceil \lg b \rceil + 60$.

Into the disk. Set $\tilde{\theta}_j = Q_q((1 - u_0/8)a)$. Before truncation, the modulus is at most $(1 - u_0/8)(1 + u_0/16) < 1$ and the error from θ_j is at most $u_0/16 + (u_0/8)(1 + u_0/16) < u_0/4$, so by (27) we obtain $|\tilde{\theta}_j| \leq 1$ and $|\tilde{\theta}_j - \theta_j| < (\sqrt{2} + \frac{1}{4})u_0 < 2u_0$.

Cost. All operands have $O(S) = O(q)$ bits and there are $O(N) = O(q)$ arithmetic steps, so schoolbook arithmetic takes $O(q^3)$ bit operations per entry without any transcendental oracle. □

Multiplying a disk-grid coefficient by a disk-valued phase must likewise remain inside the unit disk without intermediate rounding drift. The next lemma shows that forming the complex product without rounding at precision $2q$ and truncating once toward zero preserves $\mathbb{D}_q$ with error below $\sqrt{2} \cdot 2^{-q}$.

Lemma 6.2 (Disk-preserving products)**.** *If $a, b \in \widetilde{\mathbb{C}}_q$ satisfy $|a|, |b| \le 1$, then $Q_q(ab)$ is computed by four exact products of $O(q)$-bit integers and $O(q)$ further work, lies in the disk $\mathbb{D}_q$, and satisfies $|Q_q(ab) - ab| < \sqrt{2} \cdot 2^{-q}$.*

Proof. Write $a = a_0 + ia_1$ and $b = b_0 + ib_1$, form $(a_0b_0 - a_1b_1) + i(a_0b_1 + a_1b_0)$ exactly on the $2q$-bit fractional grid without rounding the four real products separately, and truncate both components toward zero by Q_q. Since $|ab| \le 1$, (27) gives both claims. □

6.2 The decimation-in-frequency schedule and its positional maps

This subsection defines the two positional maps L and U of the direct tensor FFT and checks that the chunk rounds between them have the layer shape required by Proposition A.11.

The decimation-in-frequency (DIF) recursion runs on all $d-1$ record axes in parallel, one bit level per round. A positional map L groups their address slots into K-bit chunks so that each bulk chunk round becomes a single call to Proposition A.11, and a second positional map U restores natural frequency order afterwards. For a transform of even length u, one positive normalized DIF step applies H_0 followed, on the branch $b = 1$, by multiplication by $e^{2\pi ij/u}$:

$$(F_u^+ z)_{2v+b} = \frac{1}{u/2} \sum_{j<u/2} e^{2\pi ivj/(u/2)}\, e^{2\pi ibj/u}\, \frac{z_j + (-1)^b z_{j+u/2}}{2}, \qquad b \in \{0,1\}. \tag{33}$$

Successive address slots from high to low thus receive frequency bits from low to high, so the completed DIF recursion produces bit-reversed frequency order.

Named slots and chunked layout. Following the named-slot arrangement of [OAI26, Lemma 16], put $a_i = \lg t_i$ for $1 \le i \le d-1$, and name the address slots of record axis i from most to least significant by $(i, a_i - 1), \dots, (i, 0)$. Let $b_0 = (\ell - 1) \bmod K$ and $J_{\mathrm{blk}} = (\ell - 1 - b_0)/K$. The original slot list is $\mathcal{O} = ((i,v) : i = 1, \dots, d-1,\ v = a_i - 1, \dots, 0)$. The reordered list begins with the prefix

$$\mathcal{P} = ((i, \ell - 1) : a_i = \ell,\ i \text{ increasing}) \parallel ((i,v) : v = b_0 - 1, \dots, 0 \text{ (outer)},\ i \text{ increasing (inner)})$$

and continues with the chunked list

$$\mathcal{C} = \mathop{\Big\|}_{j=0}^{J_{\mathrm{blk}}-1} \mathop{\Big\|}_{i=1}^{d-1} ((i,v) : v = \ell - 2 - jK, \dots, \ell - 1 - (j+1)K),$$

empty sublists being omitted. Let L be the positional map moving each named slot from its position in $\mathcal{O}$ to its position in $\mathcal{P}\|\mathcal{C}$, and let B be the per-axis bit reversal $(i,v) \mapsto (i, a_i - 1 - v)$ on $\mathcal{O}$.

Once the address slots are reordered into $\mathcal{P}\|\mathcal{C}$, every bulk DIF round across the $d-1$ record axes acts on a common bit offset ρ of $d-1$ consecutive K-bit chunks, matching the input layout of Proposition A.11. The next lemma verifies this layer shape and shows that applying $U = (LB)^{-1}$ after the rounds untangles both the layout permutation L and the DIF bit reversal B.

Lemma 6.3 (Layout and natural-order restoration)**.** *In a round at named slot v with $b_0 \le v \le \ell - 2$, the selected positions in $\mathcal{P}\|\mathcal{C}$ are bit $\rho = (v - b_0) \bmod K$, counted from the least significant end,*

of $D = d - 1$ consecutive K-bit chunks. With n_P and n_S denoting the numbers of record-address bits before and after these $d - 1$ chunks, the round has layer shape $[2^{n_P}] \times [2^K]^{d-1} \times [2^{n_S}]$ for Proposition A.11 (unlike a slice of Definition 5.6, whose chunks start at position 0, here they start after $\mathcal{P}$). The DIF rounds on the $d - 1$ record axes, executed in layout L on input in the original order, have exact counterpart $L\,B\,F^{+}_{\mathbf{t}_{<d}}$, where $F^{+}_{\mathbf{t}_{<d}}$ is the tensor DFT over the first $d - 1$ axes. Natural frequency order is restored by the positional map

$$U = (LB)^{-1} = B^{-1}L^{-1}. \tag{34}$$

Both L and U are compiled in time polynomial in q and executed exactly by Theorem 5.8.

Proof idea. Stripping the top bit of length-2^ℓ axes and the lowest b_0 bits into $\mathcal{P}$ leaves an exact multiple of K bits per record axis. The list $\mathcal{C}$ interleaves those remaining bits block by block so that slot v lands at a common chunk offset ρ, and conjugating the per-axis DIF recursion by L produces $LBF^{+}_{\mathbf{t}_{<d}}$, which $U = (LB)^{-1}$ inverts.

Proof. The layout. The prefix $\mathcal{P}$ removes the extra top slot of each axis of length 2^ℓ and the lowest b_0 slots of all $d-1$ record axes. The remaining $\ell - 1 - b_0 = J_{\mathrm{blk}}K$ slots of each record axis split into J_{blk} chunks of width K, which $\mathcal{C}$ orders first by chunk block j and then by axis $i \in \{1, \dots, d-1\}$.

The rounds. In a round at named slot v with $b_0 \le v \le \ell - 2$, every record axis uses chunk block $j = \lfloor (\ell - 2 - v)/K \rfloor$, whose least significant slot is $\ell - 1 - (j+1)K = (J_{\mathrm{blk}} - j - 1)K + b_0$. Its offset from the least significant end of that chunk is therefore $(v - b_0) \bmod K$, while the preceding and following address bits act as spectators of lengths n_P and n_S.

The output order. Applying (33) inductively along each record axis yields the bit-reversed output order B. Relocating the named slots by L conjugates each physical round by L, so executing the rounds in layout L after applying L to the original-order input computes $LBF^{+}_{\mathbf{t}_{<d}}$, proving (34).

Compilation. The slot lists have $m_0 = O(q)$ entries, so computing ranks, compositions, and inverses to form U as $(L \circ B)^{-1}$ takes time polynomial in q. □

The next remark gives a small example showing that the two factors of U in (34) do not commute, and records that executing L by chunk exchanges, as in [OAI26], would reintroduce the cost term that Theorem 5.8 avoids.

Remark 6.4. In general $B^{-1}L^{-1} \neq L^{-1}B^{-1}$, and (34) requires the former: for two record axes of bit widths $(4, 3)$ and $K = 2$ ($\ell = 4$, $b_0 = 1$), in zero-based destination-list notation $L = (0, 3, 4, 1, 5, 6, 2)$ and $B = (3, 2, 1, 0, 6, 5, 4)$, so $L \circ B$ sends position 0 to 1 whereas $B \circ L$ sends it to 3. Executing L via width-K chunk exchanges as in [OAI26, Lemma 16] (Appendix C) would reintroduce the normalized cost term $qK^{\tau-1}$; here L and U are executed instead by Theorem 5.8.

6.3 One combined twiddle per record

This subsection combines the up to $d - 1$ per-axis twiddle factors of a parallel DIF round into a single root-of-unity phase per record, so that rounding error does not grow with the dimension d.

Adding their angles exactly, as one integer index into the phase table, before touching the record coefficients ensures that a round incurs only one phase approximation and one complex multiplication error per coefficient, regardless of d.

At named slot v, each participating record axis i has current subproblem length $u_i = 2^{v+1}$. For a record address x, reading the named-slot table gives its current branch bit $b_i = x_{(i,v)}$ and its unprocessed lower index $j_i^{<v} = \sum_{0 \le s < v} 2^s x_{(i,s)}$. Because the twiddle of axis i depends only on the

bits of axis i, it commutes with the butterflies on all other axes, so one parallel round has exact counterpart

$$z \mapsto D_\zeta H z, \qquad \zeta(x) = \exp\Big(2\pi i \sum_{i \text{ participating}} \frac{b_i j_i^{<v}}{u_i}\Big), \tag{35}$$

where H applies H_0 to slot v of each participating axis and D_ζ multiplies record x by $\zeta(x)$. Since every u_i divides $r = 2^\ell$, the integer

$$J(x) = \sum_{i \text{ participating}} \frac{r}{u_i}\, b_i j_i^{<v} \bmod r$$

is computed without error, and the round multiplies every coefficient of record x by the single phase entry $\tilde{\theta}_{2J(x)}$ from Lemma 6.1.

6.4 The direct tensor FFT

This subsection assembles the intra-record transform, the layout permutations, and the parallel record-axis rounds into the complete direct tensor FFT and bounds its error and running time.

Under the parameter regime below, on a tensor z of the shape fixed above ($t_d = r,\ t_i \in \{r/2, r\}$) with disk-grid entries in natural tensor order, four sequential stages approximate $F_{\mathbf{t}}^{+} z$ directly on the q-grid without widening the component width. First comes an intra-record DIF transform on the last axis, then the layout permutation L, then at most ℓ parallel record-axis butterfly-and-twiddle rounds, and finally the restoring permutation $U = (LB)^{-1}$. Each of the ℓ last-axis levels and at most ℓ record-axis rounds adds error below $6 \cdot 2^{-q}$, and the two positional permutations add none, so the total error stays under $12\ell \cdot 2^{-q}$.

Theorem 6.5 (Direct complex tensor FFT)**.** *Assume the parameter family of Proposition A.11 at grid precision q. Suppose $q \geq 8$, $d \geq 2$, $r = 2^\ell$ with $2 \leq \ell < q$, $d\ell \leq C_\ell q$, $1 \leq K \leq \ell - 1$, $t_i \in \{r/2, r\}$ for $1 \leq i < d$, and $q + 2 \leq w \leq C_w q$. On a disk array on the q-grid in natural tensor order, a fixed multitape procedure returns a disk array on the q-grid in natural frequency order on every axis, in component width w (with no $\lg M$ widening), satisfying*

$$\|\widehat{F}_{\mathbf{t}}^{+} z - F_{\mathbf{t}}^{+} z\|_\infty < 12\ell \cdot 2^{-q}, \tag{36}$$

in

$$O(V[dK + \ell d^{\lambda'} + \ell\, \mathsf{M}(q+2)/q]) = O(V[dK + \ell d^{\lambda'} + \ell\, \mathsf{m}(q+2)]) \tag{37}$$

steps, where $V = 2Rw = \Theta(Rq)$.

Proof idea. The last-axis transform acts inside each length-r record and commutes with record-level operations, so Lemma 6.3 identifies the exact composition of the four stages with $F_{\mathbf{t}}^{+}$. One precomputed $2r$-entry phase table supplies all twiddles via sequential tape scans, while bulk record-axis rounds call Proposition A.11 and the few remaining prefix rounds apply H_0 sequentially at extra precision before a single truncation. Because every stage keeps entries in the unit disk $\mathbb{D}_q$ and each exact round contracts $\|\cdot\|_\infty$, local errors below $6 \cdot 2^{-q}$ per level add linearly across ℓ last-axis levels and at most ℓ record-axis rounds.

Proof. The procedure executes four stages sequentially:

(i) Apply a normalized positive DIF FFT on every length-r contiguous record, followed by bit-reversal restoration inside each record.

(ii) Apply the positional map L on the record addresses via Theorem 5.8.

(iii) Run record-axis rounds at named slots $v = \ell - 1, \dots, 0$, where at $v = \ell - 1$ only axes of length r participate. Chunk rounds with $b_0 \le v \le \ell - 2$ call Proposition A.11 with $D = d - 1$, while prefix rounds with $v = \ell - 1$ or $v < b_0$ apply exact individual butterflies before a single truncation. Each completed butterfly round is followed by the combined twiddle multiplication of Section 6.3.

(iv) Apply the restoring positional map $U = (LB)^{-1}$ on the record addresses via Theorem 5.8.

Because the last-axis transform acts within records and commutes with record-axis operations and record permutations, (33) and Lemma 6.3 show that the exact counterpart of the four stages is $F_{\mathbf{t}}^{+}$.

Phase table. One master $2r$-entry phase table from Lemma 6.1 serves all rounds. Building it once costs $rq^{O(1)} = o(V)$, since $M \ge r/2$ and r exceeds every fixed polynomial in q under the band $r \ge 2^{a_r q/d}$ with $d \le b_d q^{\varepsilon}$. Every subsequent table lookup is a charged sequential scan, so no random access is assumed.

Last axis. At a DIF level of block length u, stream the two halves of each block onto aligned tapes, form each half-sum and half-difference without error, and truncate to $\widetilde{\mathbb{C}}_q$ by Q_q. Multiply the $b = 1$ branch by its stage phase via Lemma 6.2. One scan of the master table extracts the stage table $(\tilde{\theta}_{(2r/u)j})_{j<u/2}$, which is replayed with a local rewind across the blocks of each record. Splits, merges, and the final bit reversal on the ℓ intra-record bit positions are elementary streams by Lemma A.4; this intra-record reversal is separate from U. Each level costs $O(rq + r\,\mathsf{M}(q+2))$ per record, and the intra-record bit reversal costs $O(rq\ell)$ per record.

Record axes. In each chunk round, Lemma 6.3 gives the layer shape, and setting $C_M = C_\ell + 1$ and $C_{\mathrm{desc}} = 32(C_\ell + C_w + 1)$ satisfies the header hypotheses of Proposition A.11. Indeed, there are 2^{m_0} records with $m_0 \le d\ell \le C_\ell q$, so $\lg(2 \cdot 2^{m_0}) \le (C_\ell + 1)q$, and the nine-field header has length at most $2(C_\ell + 1)q + O(\lg q) \le C_{\mathrm{desc}} q$ for large q. In each prefix round, the at most $d - 1$ participating H_0 factors are applied sequentially without rounding before one truncation Q_q. Intermediate values lie in the disk on denominator exponent at most $q + d - 1$, and temporary sums have modulus at most 2, requiring $q + d + O(1) = O(q)$ bits.

In both kinds of record-axis round, for each record x, the index $J(x)$ is computed in time polynomial in q, and $\tilde{\theta}_{2J(x)}$ is fetched by one scan of the master table. Every coefficient of the record is then multiplied by $\tilde{\theta}_{2J(x)}$ via Lemma 6.2, with total address and scan overhead $O(V)$ by Lemma A.5. Every record-axis round thus implements

$$z \mapsto Q_q(\tilde{D}\, Q_q(Hz)), \tag{38}$$

where $\tilde{D}$ is the recordwise diagonal of stored phases.[1]

Error. Put $u_0 = 2^{-q}$, let z be a disk array, and set $z' = Q_q(Hz)$. By (27), Lemma 6.1, and Lemma 6.2,

$$\begin{aligned}\|Q_q(\tilde{D}z') - D_\zeta Hz\|_\infty &\le \|Q_q(\tilde{D}z') - \tilde{D}z'\|_\infty + \|(\tilde{D} - D_\zeta)z'\|_\infty + \|D_\zeta(z' - Hz)\|_\infty \\ &< (\sqrt{2} + 2 + \sqrt{2})u_0 < 6u_0.\end{aligned}$$

The same $6u_0$ bound holds for each last-axis level, whose $b = 0$ branch incurs one truncation and whose $b = 1$ branch incurs two truncations and one phase error. Every computed intermediate array

[1] Unlike monomial twists in $\mathcal{R}_r$, general complex twiddles do not commute with componentwise truncation Q_q, and the rounded map need not be a contraction between arbitrary inputs. The error analysis below therefore compares each computed round directly against its exact contraction $D_\zeta H$ on disk inputs.

entering a round is a disk array, because H_0 layers preserve the disk, Q_q does not increase modulus, $|\tilde{\theta}_j| \le 1$, and positional maps permute values. Since every exact round A is a contraction in $\|\cdot\|_\infty$, for computed input x and ideal input x_0 we have $\|\widehat{A}x - Ax_0\|_\infty \le \|\widehat{A}x - Ax\|_\infty + \|x - x_0\|_\infty$. Summing the local bound $6u_0$ over ℓ last-axis levels and at most ℓ record-axis rounds, while L and U act without error by Theorem 5.8, gives (36).

Time. The at most $b_0 + 1 \le K$ prefix rounds cost $O(VdK)$, and the at most ℓ chunk rounds cost $O(V\ell d^{\lambda'})$. Scalar complex products across the last-axis FFT and the record-axis twiddles cost $O(V\ell + R\ell\,\mathsf{M}(q+2)) = O(V\ell\,\mathsf{M}(q+2)/q)$. By Theorem 5.8, the positional maps L and U cost $O(V[(m_0/d + K)d^{\lambda'} + K]) = O(V\ell d^{\lambda'})$, since $m_0/d \le \ell$ and $K \le \ell - 1$. All subroutines run sequentially on a fixed number of tapes and clean their workspaces on return. □

6.5 Interface with the bridge

This subsection specializes the direct tensor FFT to the grid precision of the fractional-shift bridge and verifies that it fulfills the middle-transform contract.

Instantiating Theorem 6.5 at the bridge's grid precision $q = P + 2$ and component width $w = q + 2$, where P is the internal working precision of Theorem 4.7, supplies the middle transform used in Algorithm 1. The corollary below records that its error factor is $E_{\mathrm{mid}} = 12\ell$ and its output widening is $\delta_{\mathrm{mid}} = 0$, both well within the limits of Definition 4.5.

Corollary 6.6 (Walsh middle transform)**.** *Assume the hypotheses of Theorem 6.5 at the bridge's grid precision $q = P + 2$ with component width $w = q + 2$ (verified for the root family in Lemma 7.1, with Lemma 5.3 covering the lifted precision $q + 8d$ of the internal controlled-flip batches). Applied to a complete tensor U in natural axis order $(j_1, \dots, j_d)$ with the last axis contiguous and entries in $\mathbb{D}_q$ (in particular the bridge's prepared spread array, which satisfies $\|U\|_\infty \le 1/8$), Theorem 6.5 returns a disk array on the q-grid in natural frequency order $(u_1, \dots, u_d)$, in component width $q + 2$, satisfying*

$$\|\widehat{F}^{+}_{\mathbf{t}} U - F^{+}_{\mathbf{t}} U\|_\infty < 12\ell \cdot 2^{-q}. \tag{39}$$

In particular, it satisfies the middle-transform contract (Definition 4.5) of Theorem 4.7 with $E_{\mathrm{mid}} = 12\ell$ and output widening $\delta_{\mathrm{mid}} = 0$.

Proof. Immediate from Theorem 6.5 and Proposition 5.5 at grid precision q and width $w = q + 2$; internal guard allocations and temporary lifts to $q + 8d$ return to width $q + 2$ at completed boundaries. □

The middle transform uses Theorem 5.8 for L and U, where every record carries r coefficients. By contrast, Lemma 4.8 implements the bridge's line-spreading axis exposures by the chunk interchanges of Lemma A.8 at cost $O(Vd(1 + \ell^\tau))$, since a single-line operation has no long retained record payload.

7 Multiplier, cost analysis, and proof of the main theorem

This section assembles the complete integer multiplication algorithm from the fractional-shift bridge of Section 4 and the direct tensor FFT of Section 6, balances its parameters, and proves Theorem 1. Two n-bit integers are cut into $(\log n)$-bit digits, packed into a single complex vector of power-of-two length R, and multiplied via two bridge calls, one pointwise squaring, nearest-integer rounding, and carry propagation. Summing the normalized tape costs of all stages shows that each stage falls short of $n \log n$ by a positive power of $\log n$. These five exponent gaps are collected in (45), and the

explicit parameter tuple of Table 6 yields the saving exponent $\kappa = 2^{-46}$ with strict positive slack. A first-time reader interested primarily in how the components fit together and why $\kappa = 2^{-46}$ emerges can read Sections 7.1–7.5 and skim the symbolic cutoff calculation in Section 7.6.

7.1 The algorithm and its root parameter family

This subsection defines the power-of-two grid dimensions as functions of the input bit length n and summarizes how the top-level multiplier connects its subroutines.

Parameters of the root family. We consider two n-bit integers with n at least the threshold n_* defined in Section 7.6; for $n < n_*$, the machine invokes the independent inner multiplier directly. Let $b = \lfloor \log_2 n \rfloor$ and $N_{\mathrm{dig}} = \lceil n/b \rceil$, and let $R = 2^k$ be the least power of two satisfying $R \geq 2N_{\mathrm{dig}}$. Given fixed rational exponents ε and c, chosen in Section 7.5, define

$$p = 2b + 2k + 6, \qquad d = \lfloor k^{\varepsilon} \rfloor, \qquad \ell = \lceil k/d \rceil, \qquad r = 2^{\ell}, \qquad K = \lfloor d^{c} \rfloor. \tag{40}$$

Because $0 \leq d\ell - k < d$, we set the first $d\ell - k$ axis lengths to $r/2$ and the remaining axis lengths, including t_d, to r, so that $\prod_{i=1}^{d} t_i = R$. The alias width ω, internal working precision P, grid precision $q = P + 2$, and component width $w = q + 2$ follow the bridge schedule (24); we call these sizes the *root family*. Fixed rational powers are computed by exact integer comparisons: for $\varepsilon = \varepsilon_1/\varepsilon_2$, the integer $d = \lfloor k^{\varepsilon} \rfloor$ is the largest $y \in \mathbb{N}$ satisfying $y^{\varepsilon_2} \leq k^{\varepsilon_1}$. Every transform length in this root family is a power of two. As in Section 6, $\mathsf{M}(N) = N\,\mathsf{m}(N)$ denotes the cost of the independent inner multiplier on N-bit integers.

Top-level pipeline. The multiplier packs the radix-2^b digits of the two inputs into one complex vector of length R and recovers the product polynomial from two one-dimensional transforms F_R via the root-packing proposition (Proposition 7.2). Each F_R is evaluated by the fractional-shift bridge under the bridge precision contract (Theorem 4.7); the axis exposures of Lemma 4.8 bring each axis $a = d, \ldots, 2$ innermost for spreading with the negative half-bin carrier. Inside the bridge, the direct tensor FFT of Corollary 6.6 evaluates the tensor transform. Its two address permutations, the layout map L and the restoring map $U = (LB)^{-1}$ of Section 6.2, are executed without error by the positional-permutation theorem (Theorem 5.8). Finally, chronological deconvolutions $B_2^{-1}, \ldots, B_d^{-1}$ undo the spreading to working precision.

The butterfly layer, the Walsh permutation routines, and the direct tensor FFT of Sections 5–6 assume that the dimension d, the last-axis length r, the chunk width K, and the grid precision stay inside fixed comparison bands. When $0 < \varepsilon < \frac{1}{14}$, $c > 0$, and $\varepsilon(1 + c) < 1$, the next lemma verifies all of these size and growth conditions simultaneously for all sufficiently large n, both at the bridge grid precision q and at the lifted precision $q + 8d$ of Lemma 5.3.

Lemma 7.1 (Uniform root family)**.** *Fix any rational exponents $0 < \varepsilon < \frac{1}{14}$ and $c > 0$ with $\varepsilon(1 + c) < 1$ (in particular the primary parameter tuple of Table 6, as well as the comparison rows of Appendix C). For all sufficiently large n: $RP = \Theta(n)$, $P = \Theta(q) = \Theta(k) = \Theta(\log n)$, $d = \Theta(P^{\varepsilon})$, $K = \Theta(P^{\varepsilon c})$, and $\ell = \Theta(P^{1-\varepsilon})$. At both precisions $q' \in \{q,\ q + 8d\}$, the comparison bands of Proposition A.11 hold with the fixed constants*

$$a_d = \tfrac{1}{4}, \quad b_d = 1, \quad a_r = \tfrac{1}{128}, \quad b_r = \tfrac{1}{8}, \quad C_\ell = 1, \quad C_M = 2, \quad C_w = 2, \quad C_{\mathrm{desc}} = 128, \tag{41}$$

that is, $a_d (q')^{\varepsilon} \leq d \leq b_d (q')^{\varepsilon}$ and $2^{a_r q'/d} \leq r \leq 2^{b_r q'/d}$. Moreover $dK = o(P)$, $K = o(\ell)$, $K/\log P \to \infty$, and $r/P^j \to \infty$ for every fixed j.

Proof idea. From $R = \Theta(n/\log n)$ and $b = \Theta(\log n)$, the linear bridge schedule (24) pins both q and $q + 8d$ within constant multiples of $k = \lg R$. Substituting $16k \le q' \le 64k$ and $\varepsilon < 1/14$ into the definitions of d, ℓ, and K directly verifies each band constant in (41) and the asymptotic growth ratios.

Proof. Sizes. From $2N_{\mathrm{dig}} \le R < 4N_{\mathrm{dig}}$ we have $k \le b \le 2k$ for large n and $b/k \to 1$, so the bridge schedule (24) gives $P/k \to 20$ and $q/k \to 20$. In particular, $16k \le q \le 32k$ for all $k \ge 4096$, and $q + 8d \le 64k$. Since $R = \Theta(n/\log n)$, we also have $RP = \Theta(n)$.

Bands. Because $0 < \varepsilon < \frac{1}{14}$, once $k^\varepsilon \ge 2$ we have $\frac{1}{2}k^\varepsilon \le d \le k^\varepsilon$. Since $64^{1/14} = 2^{3/7} < 2$, for $q' \in \{q, q + 8d\}$ we obtain

$$d \ge \tfrac{1}{2}\Big(\frac{q'}{64}\Big)^{\varepsilon} \ge \tfrac{1}{2}\,64^{-1/14}(q')^{\varepsilon} > \tfrac{1}{4}(q')^{\varepsilon},$$

while $d \le k^\varepsilon \le (q')^\varepsilon$. Because $k/d \le \ell \le 2k/d$ and $16k \le q' \le 64k$, we obtain $q'/(128d) \le \ell \le q'/(8d)$. The remaining constants of (41) hold as well: $d\ell \le k + d \le 2k < q'$ and $\lg(2 \cdot R/r) \le k + 1 < 2q'$. The nine-field headers have length $O(k) \le C_{\mathrm{desc}}q'$, and the component widths $w = q + 2$ and $w_* = q + 2 + 8d$ satisfy $q' + 2 \le w_{q'} \le 2q'$.

Growth. Finally, $dK/k = O(k^{\varepsilon(1+c)-1}) \to 0$, which implies $K = o(\ell)$ since $\ell = \Theta(k/d)$. The limits $K/\log P \to \infty$ and $r/P^j \to \infty$ follow from $\varepsilon c > 0$ and $\ell \ge k^{1-\varepsilon}$. □

7.2 Exact recovery

This subsection proves that two approximate bridge transforms and one rounded pointwise squaring determine every integer convolution coefficient without error.

Packing the radix-2^b digits of the two inputs as $x_j = 2^{-b}(x_j^{(0)} + ix_j^{(1)})$ lets the two-call identity (8) express their cyclic convolution through two forward transforms F_R. The next proposition tracks the bridge error bound (25) through the squaring and second transform to show that, at target precision $p = 2b + 2k + 6$, every scaled output entry lies within $\frac{1}{2}$ of the true integer convolution coefficient. Nearest-integer rounding and carry propagation then recover the exact $2n$-bit product.

Proposition 7.2 (Two-call root packing)**.** *Let $x_j^{(0)}, x_j^{(1)} \in [0, 2^b)$ $(0 \le j < N_{\mathrm{dig}})$ be the radix-2^b digits of the two n-bit inputs (with $x_j^{(0)} = x_j^{(1)} = 0$ for $N_{\mathrm{dig}} \le j < R$), and set $x_j = 2^{-b}(x_j^{(0)} + ix_j^{(1)})$ for $0 \le j < R$. Compute $\hat{y} \approx F_R x$ by the bridge, $\hat{w} = \overline{\mathcal{Q}_{p+2}(\hat{y} \odot \hat{y})}$, and $\hat{z} \approx F_R \hat{w}$ by the bridge. Then*

$$c_j = \mathrm{round}(-2^{2b-1}R^2 \operatorname{Im} \hat{z}_j)$$

equals the exact convolution coefficient $\sum_{u+v=j} x_u^{(0)} x_v^{(1)}$, and radix-$2^b$ carry propagation returns the exact $2n$-bit product. Outside the two bridge calls, the cost is $O(R\,\mathsf{M}(P) + n) = O(RP\,\mathsf{m}(P) + n)$.

Proof idea. Because $R \ge 2N_{\mathrm{dig}}$, the cyclic convolution of length R coincides with the non-cyclic digit convolution. Applying the bridge error bound $0.177021 \cdot 2^{-p}X$ of (25) twice, with $X = \max(1, \|\cdot\|_\infty)$ evaluated on each bridge input, and accounting for the intermediate squaring and rounding bounds the unscaled error by $1.2391 \cdot 2^{-p}$. Multiplying by $2^{2b-1}R^2 = 2^{p-7}$ keeps the final error below $1.2391 \cdot 2^{-7} < \frac{1}{2}$.

Proof. The identity. Applying (8) to $2^{-b}x^{(0)}$ and $2^{-b}x^{(1)}$ gives $x^{(0)} \circledast x^{(1)} = -2^{2b-1}R^2 \operatorname{Im} F_R(\overline{(F_R x) \odot (F_R x)})$, which has no cyclic wrap because the product polynomial has degree at most $2N_{\mathrm{dig}} - 2 < R$.

The error. Put $u_0 = 2^{-p}$. Each x_j lies on the $(p+2)$-grid with $\|x\|_\infty < \sqrt{2}$, so Theorem 4.7 gives $\|\hat{y} - y\|_\infty < 0.177021\sqrt{2}\,u_0 < 0.25035\,u_0$, where $y = F_R x$ and $\|y\|_\infty \le \|x\|_\infty < \sqrt{2}$. Since $p \ge 14$, we have $\|\hat{y} \odot \hat{y} - y \odot y\|_\infty \le (\|\hat{y}\|_\infty + \|y\|_\infty)\|\hat{y} - y\|_\infty < 0.7082\,u_0$, and rounding to $\widetilde{\mathbb{C}}_{p+2}$ adds at most $\frac{\sqrt{2}}{8}u_0 < 0.17678\,u_0$. Hence $\|\hat{w} - \overline{y \odot y}\|_\infty < 0.8850\,u_0$ and $\|\hat{w}\|_\infty < 2.0001$. The second bridge call adds less than $0.177021 \cdot 2.0001\,u_0 < 0.35406\,u_0$, and $\|F_R\|_\infty \le 1$, so $\|\hat{z} - F_R(\overline{y \odot y})\|_\infty < 1.2391\,u_0$. Multiplying by $2^{2b-1}R^2 = 2^{2b+2k-1} = 2^{p-7}$ yields an error below $1.2391 \cdot 2^{-7} < \frac{1}{2}$, so rounding to the nearest integer recovers every c_j without error. On $\widetilde{\mathbb{C}}_{p+2}$, if $n_j \in \mathbb{Z}$ is the numerator of $\operatorname{Im} \hat{z}_j$, then $-2^{2b-1}R^2 \operatorname{Im} \hat{z}_j = -n_j/512$.

Carries and cost. Each coefficient satisfies $0 \le c_j \le N_{\mathrm{dig}}(2^b - 1)^2$. Setting $s_0 = 0$, $\gamma_j = (c_j + s_j) \bmod 2^b$, and $s_{j+1} = \lfloor (c_j + s_j)/2^b \rfloor$ produces the radix-2^b digits γ_j of the product, which is strictly below 2^{2n}, emitting $2n$ bits. Pointwise squaring uses R complex multiplications on $O(P)$-bit integers, and all remaining steps are linear scans. □

By Proposition 7.2, the total running time $\mathsf{M}_{\mathrm{W}}(n)$ of the multiplier satisfies

$$\mathsf{M}_{\mathrm{W}}(n) \le 2\,\mathsf{T}_{\mathrm{W}}(R, P) + O(R\,\mathsf{M}(P) + n), \tag{42}$$

where $\mathsf{T}_{\mathrm{W}}(R, P)$ is the cost of one bridge call on the Walsh route, with no recursive call to M_{W} itself.

7.3 Cost analysis of a bridge call

This subsection tallies all setup work, inner integer multiplications, and tape-routing steps in one bridge call, normalizing each contribution by the stored bit volume $V = RP$.

Put $V = RP$. Because $\varepsilon C_1 < 1$, all internal and external component widths, including the layer guard $\Delta = \lceil C_0 d^{C_1} \rceil = o(P)$ and the $8d$-bit Walsh lift, remain $O(P)$, so V is the stored bit volume up to constant factors. One-time and per-line or per-record setup work that is not repeated per coefficient covers cycle decompositions, masks, headers, named-slot lists, filter taps, solver parameters, phase tables, and recursion descriptors. Each such object is built in time polynomial in P across at most $(R/r + r)P^{O(1)}$ objects. Hence, for fixed constants $C_{\mathrm{set}}, D_{\mathrm{set}}$, the setup cost satisfies

$$\mathcal{S}(R, P) \le C_{\mathrm{set}}\big((R/r)P^{D_{\mathrm{set}}} + rP^{D_{\mathrm{set}}} + P^{D_{\mathrm{set}}}\big) = o(RP), \tag{43}$$

by Lemma 7.1 and $R/r \ge r/2$.

Operand widths of all inner integer products. Per-coefficient multiplications are delegated to an independent integer multiplier of cost $\mathsf{M}(N) = N\,\mathsf{m}(N)$ on N-bit operands, assuming only $\mathsf{m}(N) = (\lg N)^{O(1)}$, as satisfied by Schönhage–Strassen [SS71] with $\mathsf{m}(N) = O(\lg N \lg\lg N)$. Its operands and output occupy dedicated work tapes, leaving the caller's other tapes untouched and cleaning its workspace on return. Fixing a regular majorant $\overline{\mathsf{M}}(N) = A\,N(\lg(2N))^J$ of M gives $\overline{\mathsf{M}}(c'N) = O(\overline{\mathsf{M}}(N))$ for every constant $c' > 0$. Every ordinary integer product on the Walsh route occurs at one of four sites and has at most $O(P^2) = O((\log n)^2)$ bits:

(1) *Root packing* in Proposition 7.2: R complex squarings on $\widetilde{\mathbb{C}}_{p+2}$, using $O(P)$-bit integer products of total cost $O(R\,\mathsf{M}(P)) = O(V\,\mathsf{m}(P))$.

(2) *Direct tensor FFT* in Theorem 6.5 at $q = P + 2$: ℓ last-axis levels and at most ℓ record-axis combined twiddles, each performing four exact $(q+2)$-bit integer products per complex multiplication by Lemma 6.2, of total cost $O(V\ell\,\mathsf{m}(P))$. Neither the butterfly layers of Proposition A.11 nor the positional permutations of Theorem 5.8 perform general multiplications.

(3) *Cyclic deconvolution* in part (iv) of Lemma 4.6: $O(1)$ multiplications per entry on operands of at most $P + k + 8 = O(P)$ bits across $d - 1$ axes, of total cost $O(Vd\,\mathsf{m}(P))$. Truncated binary powering on $O(P + k)$-bit integers for the closures in part (v) of Lemma 4.6 and rational-series evaluations on $O(P)$-bit integers in part (iii) of Lemma 4.6 and Lemma 6.1 belong to the setup term (43).

(4) *Line spreading* in part (ii) of Lemma 4.6: on each of $d - 1$ axes, overlap-add blocks of $W_t = \min(2P + 1, t_a) = \Theta(P)$ entries are packed via signed Kronecker substitution into $(4P + 16)$-bit slots. Each block performs four exact products of integers of at most $W_t(4P + 16) + 1 = O(P^2)$ bits across $O(R/P)$ blocks per axis, costing $O(Vd\,\mathsf{m}(P^2))$.

In particular, because no polynomial-ring multiplication in $\mathcal{R}_r$ is performed, no $\Theta(rP)$-bit packed product arises, and both $\mathsf{m}(P)$ and $\mathsf{m}(P^2)$ are $(\lg P)^{O(1)}$.

Under Theorem 5.8, both L and $U = (LB)^{-1}$ cost $O(V[(m_0/d + K)d^{\lambda'} + K]) = O(V\ell d^{\lambda'})$. Combining this bound with Theorem 6.5, Lemma 4.8, and Lemma 4.6 yields the normalized cost breakdown in Table 5.

Table 5: Normalized running time of each stage in one bridge call on the Walsh route. Costs are divided by the bit volume $V = RP$, with $d = \Theta(P^{\varepsilon})$, $\ell = \Theta(P^{1-\varepsilon})$, and $K = \Theta(P^{\varepsilon c})$; a row with gap g has normalized cost $O(P^{1-g})$ up to polylogarithmic factors in P where indicated (Section 7.4).

operation	normalized cost	gap below P^1
individual prefix butterfly rounds	dK	$1 - \varepsilon(1 + c)$
butterfly layers and positional layouts L, U	$\ell d^{\lambda'}$	$\varepsilon(1 - \lambda')$
complex twiddles and last-axis FFT	$\ell\,\mathsf{m}(P)$	ε, up to logarithms
axis exposures and restorations	$d(1 + \ell^{\tau})$	$(1 - \varepsilon)(1 - \tau)$
line spreading and cyclic solves	$d(\mathsf{m}(P) + \mathsf{m}(P^2))$	$1 - \varepsilon$, up to logarithms

Summing the rows of Table 5 and absorbing (43) gives

$$\mathsf{T}_{\mathrm{W}}(R, P) = O(RP\,[dK + \ell d^{\lambda'} + \ell\,\mathsf{m}(P) + d(1 + \ell^{\tau}) + d(\mathsf{m}(P) + \mathsf{m}(P^2))]). \tag{44}$$

Here the direct axis exposures of Lemma 4.8 contribute $d(1 + \ell^{\tau})$ in place of the $d^2(1 + \ell^{\tau})$ adjacent-axis schedule of [OAI26, §9]. Executing L and $U = (LB)^{-1}$ via Theorem 5.8 eliminates the external chunk-exchange term $PK^{\tau-1}$, while the internal packed basis changes of Proposition A.11 are already included in $d^{\lambda'}$.

7.4 Gaps and slack

This subsection expresses the condition for saving a power of $\log n$ in terms of five exponent gaps. It also shows why replacing chunk exchanges by Walsh permutations makes the exponent depend linearly, rather than quadratically, on the network deficit introduced in Section 2.

By Table 5, a row with gap g contributes $O(V\,P^{1-g}(\lg P)^{O(1)})$ steps. An admissible parameter tuple therefore yields the saving exponent κ whenever $0 < \kappa < g_{\min}$, where $g_{\min}$ is the minimum of the five gaps

$$g_{\mathrm{pre}} = 1 - \varepsilon(1 + c), \quad g_{\mathrm{lay}} = \varepsilon(1 - \lambda'), \quad g_{\mathrm{last}} = \varepsilon, \quad g_{\mathrm{axis}} = (1 - \varepsilon)(1 - \tau), \quad g_{\mathrm{short}} = 1 - \varepsilon. \tag{45}$$

Any strict positive slack $s = g_{\min} - \kappa > 0$ absorbs fixed powers of $\lg P$: for all integers $j \geq 0$ and $P \geq 2$,

$$(\lg(2P))^j \leq 4^j (\ln P)^j \leq j!\,(4/s)^j\,P^s,$$

by the j-th term of the exponential series for $e^{s \ln P}$. The first inequality holds at $P = 2$ since $\lg 4 = 2 < 4 \ln 2$, and for $P \geq 3$ since $\lg(2P) = \lceil 1 + \log_2 P \rceil < 2 + (\ln P)/\ln 2 < 4 \ln P$, as $\ln P > 1$ and $\ln 2 > 1/2$.

Linear vs. quadratic bottleneck. On the Walsh route, the binding gap is the butterfly-layer gap $g_{\text{lay}} = \varepsilon(1 - \lambda')$. Since the layer hypotheses require $\lambda' > \sigma$ and $\varepsilon < 1/C_1$, this gap is at most $(1 - \sigma)/C_1$, which is *linear* in the complex network's exponent deficit $1 - \sigma$. By contrast, if the layout L is executed by width-K chunk exchanges as on a Bluestein middle transform in (59) of Appendix C, the normalized cost term $PK^{\tau-1}$ introduces the additional gap $g_{\text{chunk}} = \varepsilon c(1 - \tau)$. Because Proposition A.11 requires $c < \beta(1 - \tau)/\tau$ and $\varepsilon < 1/C_1$, this forces

$$g_{\text{chunk}} = \varepsilon c(1 - \tau) < \frac{\beta(1 - \tau)^2}{C_1 \tau} < \frac{(1 - \tau)^2}{C_1 \tau},$$

capping κ quadratically in $1 - \tau$.

7.5 Parameter choice

This subsection fixes the explicit rational parameter tuple for Theorem 1 and verifies all admissibility inequalities and positive cost gaps in exact rational arithmetic.

Proposition A.2 in Appendix A fixes the network exponents at $h = 100$ and $m = 10^6$. With $\xi = 2^{-42}$, the exact network counts of Fact 3.4 together with the rational bound $\ln 10^6 < 14$ certify that $\tau = 1 - 4\xi = 1 - 2^{-40}$ and $\sigma = 1 - \xi = 1 - 2^{-42}$ satisfy $s_b/W_b < m^\tau$ and $s_c/W_c < m^\sigma$. Likewise, the bound $B_{\text{net}} < m^{11}$, which gives $\nu_{\text{net}} = 11$ in (47), certifies that the guard exponent $C_1 = \nu_{\text{net}} + 3 = 14$ is admissible in Proposition A.11. Table 6 records the parameter tuple used to prove Theorem 1.

Table 6: Explicit rational parameters for the primary Walsh multiplier. With $\xi = 2^{-42}$, this tuple satisfies all butterfly-layer and root-family constraints and yields the saving exponent $\kappa = 2^{-46}$ in Theorem 1.

κ	C_1	ε	β	c	τ	σ	λ	λ'
2^{-46}	14	1/15	1/64	$\xi/256$	$1 - 4\xi$	$1 - \xi$	$1 - 63\xi/64$	$1 - 31\xi/32$

We verify the admissibility inequalities of Proposition A.11 and the gaps of (45) for Table 6 in exact rational arithmetic:

- *Layer and guard hypotheses.* We have $\max\{\tau, \sigma\} = 1 - \xi < \lambda = 1 - \frac{63}{64}\xi < 1$ with margin $\lambda - \sigma = \frac{\xi}{64} > 0$. Since $c/\beta = \xi/4$, we have $\tau(1 + c/\beta) = (1 - 4\xi)(1 + \xi/4) = 1 - \frac{15}{4}\xi - \xi^2$, giving the strict margin $\lambda - \tau(1 + c/\beta) = \frac{177}{64}\xi + \xi^2 > 0$. Moreover $\sigma + \beta(1 - \sigma) = 1 - \frac{63}{64}\xi = \lambda$, so $\max\{\lambda, \sigma + \beta(1 - \sigma)\} = \lambda < \lambda' = 1 - \frac{31}{32}\xi < 1$ with margin $\lambda' - \lambda = \frac{\xi}{64} > 0$. Finally, $C_1 = 14$ is admissible and $1 - \varepsilon C_1 = \frac{1}{15} > 0$.
- *Gaps and slack.* Evaluating (45) gives

$$g_{\text{lay}} = \frac{31}{480}\xi, \qquad g_{\text{axis}} = \frac{56}{15}\xi, \qquad g_{\text{last}} = \frac{1}{15}, \qquad g_{\text{short}} = \frac{14}{15}, \qquad g_{\text{pre}} = \frac{14}{15} - \frac{\xi}{3840} > \frac{13}{15}.$$

  The unique minimum is $g_{\min} = g_{\text{lay}} = \frac{31}{480}\xi$. Comparing against $\kappa = 2^{-46} = \frac{\xi}{16} = \frac{30}{480}\xi$ gives strict positive slack $s = g_{\min} - \kappa = \frac{\xi}{480} > 0$, with ratio $g_{\min}/\kappa = 31/30$.

The next remark isolates the architectural gain of the Walsh middle transform by evaluating the saving exponent under the untuned network constants printed in [OAI26].

Remark 7.3 (Exponent with the printed constants of [OAI26], and comparison rows). To isolate the gain of the Walsh middle transform from parameter retuning, fix the structural constants $\beta = \frac{1}{2}$, $\tau = \sigma = 1 - 2^{-50} = 1 - 2u$ ($u = 2^{-51}$) and $C_1 = 20$ printed in [OAI26, Prop. 14]. With these constants, the printed statements [OAI26, Lemma 12, Prop. 14] apply directly without Appendix A. Taking $\varepsilon = 1/21$, $c = u/4$, $\lambda = 1 - 129u/128$ and $\lambda' = 1 - 63u/64$ satisfies all hypotheses of [OAI26, Prop. 14] (with $\tau(1 + c/\beta) = 1 - \frac{3}{2}u - u^2 < \lambda$, $\sigma + \beta(1 - \sigma) = 1 - u < \lambda'$, and $1 - \varepsilon C_1 = 1/21$) and gives $g_{\min} = g_{\mathrm{lay}} = 3u/64 = 3 \cdot 2^{-57}$, yielding $\kappa = u/32 = 2^{-56}$ with slack 2^{-57}. Because any choice of the free parameters with these fixed structural constants must satisfy $\lambda' > \sigma + \beta(1 - \sigma) = 1 - u$ and $\varepsilon < 1/C_1 = 1/20$, one has $\kappa < g_{\mathrm{lay}} = \varepsilon(1 - \lambda') < u/20 = \frac{8}{5}\, 2^{-56} < 2^{-55}$; thus 2^{-56} is the largest exponent of the form 2^{-j} ($j \in \mathbb{N}$) that our analysis admits with those constants. Appendix C gives the full comparison with a Bluestein middle transform (reproducing 2^{-182} at the parameters of [OAI26, (38)] and reaching 2^{-84} with retuned parameters) together with the $C_1 = 20$ Walsh row ($\kappa = 2^{-47}$).

7.6 The cutoff and proof of Theorem 1

This subsection defines the symbolic input-length threshold n_* above which all asymptotic bounds activate simultaneously, and completes the proof of Theorem 1.

Because the tape procedures derived from Facts 3.3–3.4 have unspecified implied constants and eventual thresholds, we define the fallback threshold symbolically; Section 9 discusses the resulting astronomical activation sizes.

Cutoff parameters. For the tuple of Table 6, put $s = g_{\min} - \kappa = \xi/480 > 0$. Fix integers $C \geq 2$, $j \geq 1$, and $D_{\mathrm{set}} \geq 1$ such that every normalized cost term in (44) and (42) is at most $CP^{1-g_{\min}}(\lg(2P))^j$ and (43) holds with coefficient at most C and degree at most D_{set}. Let P_{tape} be a common precision threshold under the bands (41) for both the butterfly layer of Proposition A.11 and the arbitrary-width interchange of Lemma A.8. Let C_0 be the guard constant (47) of Proposition A.11 from Appendix A. For $\alpha_0 > 0$ and $b_* \geq 1$ put $\Phi(\alpha_0, b_*) = 2^{\lceil (64 b_*/\alpha_0)^2 \rceil}$, and put $K_{\log} = 2^{\lceil \max\{2, (16j/s)^2, (8/s)\lg(2C)\} \rceil}$. Define the constant

$$K_* = \Big\lceil \max\Big\{ 4096,\ P_{\mathrm{tape}},\ (4C_0)^{1/(1-\varepsilon C_1)},\ \Phi(\varepsilon c, 1),\ 2^{2/\varepsilon},\ 2^{2/(1-\varepsilon(1+c))}, \\ \Phi\big(1-\varepsilon, \lceil D_{\mathrm{set}} + 2 + \lg(8C) \rceil\big),\ K_{\log} \Big\} \Big\rceil \tag{46}$$

and set $n_* = 2^{4K_*}$; here K_* is a fixed threshold rather than the chunk width K.

For $n \geq n_*$ we have $b = \lfloor \log_2 n \rfloor \geq 4K_*$, so $2N_{\mathrm{dig}} \geq 2n/b > 2^{2K_*}$ and hence $k \geq K_*$. We now verify that $k \geq K_*$ activates all asymptotic bounds simultaneously. If $k \geq \Phi(\alpha_0, b_*)$ and $z = \log_2 k$, then $z \geq (64 b_*/\alpha_0)^2$ implies $(\ln 2)\alpha_0 z \geq 32 b_* \sqrt{z} \geq \ln(b_*(z+5)^2)$, so $k^{\alpha_0} \geq b_*(z+5)^2$. Since $q \leq 32k$ gives $\log_2 q \leq z + 5$, taking $\alpha_0 = \varepsilon c$ and $b_* = 1$ makes $K \geq 6$ and $K/\log P$ large. Taking $\alpha_0 = 1 - \varepsilon$ and $b_* = \lceil D_{\mathrm{set}} + 2 + \lg(8C) \rceil$ gives $r \geq 8CP^{D_{\mathrm{set}}+1}$ and $R/r \geq r/2 \geq 4CP^{D_{\mathrm{set}}+1}$, so the setup (43) is at most RP. The term $(4C_0)^{1/(1-\varepsilon C_1)}$ ensures $\Delta \leq C_0 k^{\varepsilon C_1} + 1 \leq k/2$, and the terms $2^{2/\varepsilon}$ and $2^{2/(1-\varepsilon(1+c))}$ ensure $d \geq 2$ and $dK \leq k/4$. Finally, $K_{\log}$ ensures $\ln C + j \ln \lg(2P) \leq \frac{s}{2} \ln P$, so half of the slack s survives all implied constants and polylogarithmic factors.

Proof idea. Below the fixed threshold n_*, the machine delegates multiplication directly to a standard $N(\log N)^{O(1)}$ inner multiplier in bounded time. For $n \geq n_*$, the cutoff (46) makes every size band, guard-bit margin, and setup bound derived in Sections 7.1–7.5 hold simultaneously. Two

calls to the fractional-shift bridge, instantiated with the direct complex tensor FFT and exact Walsh-layer address permutations, recover the exact $2n$-bit product by Proposition 7.2. The strict slack $s = g_{\min} - \kappa = \xi/480 > 0$ absorbs all polylogarithmic factors in $P = \Theta(\log n)$, yielding total step count $O(n(\log n)^{1-2^{-46}})$ on a single uniform multitape machine.

Proof of Theorem 1. Fix the parameter tuple of Table 6 with $\kappa = 2^{-46}$ and the cutoff $n_* = 2^{4K_*}$ from (46). Let the independent inner multiplier have cost $\mathsf{M}(N) = N(\log N)^{O(1)}$, such as Schönhage–Strassen [SS71], and for $n < n_*$ invoke this inner multiplier directly on the two n-bit inputs. We verify parameter admissibility, correctness, outer CRT-freeness, running time, and machine uniformity for $n \geq n_*$.

Admissibility. Proposition A.2 certifies the network exponents $\tau = 1 - 2^{-40}$ and $\sigma = 1 - 2^{-42}$ and the admissibility of $C_1 = 14$ at $h = 100$, and the strict margins of Proposition A.11 hold as checked after Table 6. Since $\varepsilon = 1/15 < 1/14$ and $\varepsilon(1 + c) < 1$, Lemma 7.1 guarantees the comparison bands (41), component widths, complete rectangular shapes, and long-record conditions at both precisions $q = P + 2$ and $q_* = q + 8d$.

Correctness and outer structure. All transform lengths $R = 2^k$ and $t_1, \ldots, t_d \in \{r/2, r\}$ are powers of two; the outer reduction uses no Chinese remainder map, no prime transform length, no coprime-length resampling, and no Fourier-grid expansion. Inside each bridge call, Theorem 5.8 executes the layout map L and the restoring map $U = (LB)^{-1}$ without error on $\widetilde{\mathbb{C}}_q$, returning from the lifted precision $q + 8d$ to $\widetilde{\mathbb{C}}_q$ at proved exact permutation boundaries. Theorem 6.5 and Corollary 6.6 therefore supply a middle transform at grid precision q with $E_{\mathrm{mid}} = 12\ell$ and $\delta_{\mathrm{mid}} = 0$. By Theorem 4.7 and Lemma 4.8, the two bridge transforms satisfy the precision bound (25), and Proposition 7.2 recovers the exact $2n$-bit product.

Inner products and running time. As verified in Section 7.3, every ordinary integer product in the algorithm has at most $O(P^2) = O((\log n)^2)$ bits and is executed by the independent multiplier M, with no recursive call to the algorithm of Theorem 1. By (42), (44), the logarithmic absorption of Section 7.4, and the positive slack $s = g_{\min} - \kappa = \xi/480 > 0$, the running time for $n \geq n_*$ satisfies $\mathsf{M}_{\mathrm{W}}(n) = O(RP \cdot P^{1-\kappa})$. Since $RP = \Theta(n)$ and $P = \Theta(\log n)$ by Lemma 7.1, this is $O(n(\log n)^{1-2^{-46}})$ uniformly for $n \geq n_*$. On the finitely many input lengths $n < n_*$, the direct call to the inner multiplier takes bounded time, absorbed into the implied constant.

Uniformity. All lists of axis lengths, masks, phase and filter tables, and positional-map decompositions are generated deterministically by the machine as tape data. Moreover, every subroutine is a sequential composition of fixed-tape procedures over a fixed finite alphabet and tape count independent of n. □

Finally, the next remark explains how the black-box inner multiplier M is instantiated when one also wishes to avoid prime transform lengths inside the $O((\log n)^2)$-bit scalar products, as in our Lean 4 formalization.

Remark 7.4 (CRT-free inner multiplication). Theorem 1 treats the inner multiplier M as a black box on $O((\log n)^2)$-bit integers and does not depend on [MS]. If one instantiates M instead with the two-dimensional fractional-shift $O(N \log N)$ multiplier of [MS] in place of [SS71], then the inner integer products likewise use only power-of-two Fourier transforms without a Chinese remainder map or prime transform lengths. Its written analysis uses faithful phase and filter tables as in Section 4. In the formal proof of Theorem 2, the inner multiplier is supplied by an independent Lean 4 formalization of the two-dimensional multiplier of [MS] (headline `IntMulTM.intMul_TM2_nlogn`), developed for this paper and kernel-checked as part of [Lean26] (Section 8). The companion manuscript [MS] distributes its own, separate Lean development (headline `IntMul.tmIntMul_evalsToInTime_n_log_n`).

8 Formal verification in Lean 4

Lean 4 [dMU21] is an interactive theorem prover in which mathematical definitions, algorithms, and proofs are written in a typed formal language. Every proof step is checked mechanically by a small software kernel against the foundational rules of the logic. Consequently, a reader who trusts that kernel and reads the short formal theorem statement need not inspect the proof text to be certain that every case distinction, rounding bound, and tape invariant holds. This section explains what the formal proof establishes, what must be trusted, how the proof was checked, and how the code maps to the paper. A reader interested only in the mathematical argument may skip this section on a first reading.

What is proved and the formal statement. Theorem 2 establishes the $O(n(\log n)^{1-2^{-46}})$ bit-complexity bound of Theorem 1 in Lean 4 against its mathematical library Mathlib [mC20]. The formal development is [Lean26]. Our written proof in Sections 4–7 imports two ingredients from [OAI26] as black boxes: the elementary tape-stream primitives of Fact 3.3 and the $h = 100$ finite-network certificates of Fact 3.4. By contrast, the Lean 4 proof assumes no result of [OAI26], of [MS], or of any other paper as an axiom or hypothesis.

Instead, every component is constructed and proved from scratch inside Lean 4. This includes both triple networks of [OAI26, §3] at $h = 100$ together with their rank and endpoint certificates, all multi-stack Turing machine procedures, the parameter-exposed reductions of Appendix A, and the $O(N \log N)$ inner integer multiplier. The entire proof depends only on the three standard logical axioms of Lean 4: `propext`, `Classical.choice`, and `Quot.sound`.

The top-level theorem of the development is

```
theorem SubNLogN.Assembly.intMul_challenge : SubNLogN.Challenge.IntMulSubNLogN
```

The target proposition `IntMulSubNLogN` and its input encoding live in the self-contained 80-line statement file `SubNLogN/Challenge.lean`, inside `namespace SubNLogN.Challenge` with `Computability` open. Below we quote those core definitions verbatim, omitting only docstrings, the matching decoder with its helper lemmas, and two short sanity checks:

```
inductive PairSym
  | bit (b : Bool)
  | sep
  deriving DecidableEq, Fintype

def encodeNatPair (p : ℕ × ℕ) : List PairSym :=
  (encodeNat p.1).map PairSym.bit ++ PairSym.sep :: (encodeNat p.2).map PairSym.bit

def encNatPair : Encoding (ℕ × ℕ) PairSym where
  encode := encodeNatPair
  decode := decodeNatPair
  decode_encode := decodeNatPair_encodeNatPair

def IntMulSubNLogN : Prop :=
  ∃ M : Turing.TM2ComputableInTime encNatPair.encode encodingNatBool.encode
      (fun p : ℕ × ℕ => p.1 * p.2),
    (∀ k, Finite (M.tm.Γ k)) ∧
    ∃ C N₀ : ℕ, 0 < C ∧ ∀ L ≥ N₀,
      (M.time L) ^ (2 ^ 46) ≤ (C * L) ^ (2 ^ 46) * (Nat.log2 L) ^ (2 ^ 46 - 1)
```

In plain language, `IntMulSubNLogN` states that there exists a single multi-stack Turing machine M in Mathlib's model with finite symbol alphabets on all stacks. Given bits(a) `sep` bits(b) for any natural numbers $a, b \in \mathbb{N}$, the machine outputs bits(ab). Moreover, there exist constants $C > 0$ and N_0 such that M halts within $C\,L\,(\log_2 L)^{1-2^{-46}}$ steps on every input of symbol length $L \geq N_0$. Here bits(x) denotes the least-significant-bit-first binary representation of x without trailing high-order zeros, which is empty when $x = 0$.

What must be trusted, and how the proof was checked. Accepting the formal theorem requires trusting only four small components:

- *The proof kernel:* the Lean 4 type-checking kernel at version 4.33.1.
- *Referenced library definitions:* the seven Mathlib definitions named in the statement at Mathlib commit `0df444a3`, together with Lean core's binary logarithm `Nat.log2`. Those Mathlib definitions are `Turing.FinTM2`, `Turing.TM2ComputableInTime`, `Computability.encodeNat`, `Computability.decodeNat`, `encodingNatBool`, `Encoding`, and `Finite`.
- *The statement file:* the 80-line specification `SubNLogN/Challenge.lean` quoted above, which imports only two Mathlib modules. SHA-256: `7aa50930921c494e50803b037c4cc6ed9be880c3f795e4b171e16ab59cf61fb5`.
- *Standard logical axioms:* Lean's three foundational axioms: propositional extensionality `propext`, classical choice `Classical.choice`, and quotient soundness `Quot.sound`.

Everything outside those four items is verified mechanically by the kernel and need not be read line by line. That verified body comprises roughly 299,000 lines across 1,062 files for the sub-$n \log n$ reduction, about 185,000 lines across 949 files for our independent formalization of the two-dimensional multiplier of [MS], and the underlying Mathlib proofs.

On the distributed snapshot [Lean26], whose comment-stripped sources are byte-identical to commit `3480d37`, three independent verification checks passed. All commands and logs appear in the `replay/` directory of the archive:

(1) *Compilation and fresh-kernel replay.* Lean 4.33.1 built every module without error via `lake build` with exit code 0. We then ran `leanchecker`, the proof-replay tool shipped with Lean that reloads compiled proof terms into a fresh kernel environment from scratch. It replayed each of the 2,011 modules against its imports, and also replayed the full import closures of the two headline modules into fresh kernels, covering 874,850 and 555,268 constants including all used Mathlib declarations.

(2) *Axiom and escape-hatch audit.* Lean's `#print axioms` command confirmed that `intMul_challenge` and its corollaries depend only on `propext`, `Classical.choice`, and `Quot.sound`. Automated source scans also confirmed zero occurrences of the proof-bypass or trust-extending keywords `sorry`, `axiom`, `native_decide`, `implemented_by`, `extern`, and `unsafe`.

(3) *Independent external checkers and negative controls.* We ran `comparator`[2], an adversarial verification tool that checks exported proof artifacts against a separate target specification. It confirmed that the exported theorem has the exact type in `SubNLogN/Challenge.lean` and uses only the three standard axioms. It also replayed the entire proof both in Lean's kernel and in `nanoda`[3], an independent type-checking kernel for Lean 4 written separately from Lean itself.

[2] https://github.com/leanprover/comparator
[3] https://github.com/ammkrn/nanoda_lib

To test the harness, we ran the same pipeline on seven negative controls. Three controls altered the statement by writing exponent 2^{45} instead of 2^{46}, dropping the finite-alphabet conjunct, or replacing $\leq$ with $<$. Four controls altered the proof by inserting `sorry`, adding an axiom, routing an extra axiom through a helper lemma, or using `native_decide`. The pipeline rejected all seven.

Correspondence to Theorem 1 and structure of the development. Four points connect the formal proposition `IntMulSubNLogN` to Theorem 1:

(a) *Machine model.* Mathlib's `Turing.FinTM2` model represents a finite-state controller acting on finitely many stacks, where each stack behaves like a tape whose head stays at its top cell. The wrapper `Turing.TM2ComputableInTime` pairs a `FinTM2` machine with input and output encodings and a step-count function `M.time`. Mathlib itself requires finiteness only for the input alphabet, so the explicit conjunct `∀ k, Finite (M.tm.Γ k)` forces every stack alphabet to be finite as well. One `FinTM2` step performs a bounded number of push, pop, peek, and state-branch actions and therefore costs $O(1)$ standard multitape Turing machine steps. Converting between fixed-width n-bit integers and `encNatPair` or `encodingNatBool` takes $O(n)$ steps; these routine model translations are not formalized.

(b) *Input, output, and worst-case time.* Execution begins with `encodeNatPair (a, b)` on the input stack and all other stacks empty. Within $T(L) = $ `M.time L` steps, the machine must reach Mathlib's halting state with bits(ab) on the output stack and all other stacks empty, for every pair $(a, b) \in \mathbb{N}^2$. When $L = |\mathrm{bits}(a)| + |\mathrm{bits}(b)| + 1 \geq N_0$, taking 2^{46}-th roots of $T(L)^{2^{46}} \leq (CL)^{2^{46}} \lfloor \log_2 L \rfloor^{2^{46}-1}$ yields $T(L) \leq C\, L\, (\log_2 L)^{1-2^{-46}}$.

Now consider n-bit operands $0 \leq a, b < 2^n$ with $n \geq 2$. Stripping high-order zeros leaves encoded length $1 \leq L \leq 2n+1$. Short inputs with $L < \max(N_0, 2)$ take at most $\max_{1 \leq L' < \max(N_0,2)} T(L') = O(1)$ steps. Longer inputs with $\max(N_0, 2) \leq L \leq 2n+1$ satisfy $T(L) \leq C(2n+1)(\log_2(2n+1))^{1-2^{-46}}$ by monotonicity. Combining both cases with the $O(n)$ fixed-width wrapper gives $O(n(\log n)^{1-2^{-46}})$ worst-case steps.

(c) *Exponents.* Alongside `intMul_challenge` and `intMul_TM2_sub_nlogn_46`, the development derives weaker bounds for the same machine by exponent monotonicity. Specifically, it exports theorems for exponents 2^{-47} in `intMul_TM2_sub_nlogn_47`, 2^{-56} in `intMul_TM2_sub_nlogn_56`, and qualitative $o(n \log n)$ in `intMul_TM2_littleO_nlogn`. Only the $\kappa = 2^{-46}$ Walsh parameterization is constructed as a formal machine; the printed-constants schedule and the Bluestein comparison route of Appendix C are not formalized separately.

(d) *Natural numbers.* The formal theorem is stated over $\mathbb{N}$. Signed n-bit integers reduce to natural numbers through a linear-time sign-and-magnitude wrapper, which is not formalized.

Although `IntMulSubNLogN` quantifies existentially over the machine M, inspecting the definitions shows that the witness implements our Walsh route at the $\kappa = 2^{-46}$ tuple of Table 6. In `SubNLogN/Math/Cost/Row46.lean`, with $\xi = 2^{-42}$, the parameters are $C_1 = 14$, $\varepsilon = 1/15$, $\beta = 1/64$, $c = \xi/256$, $\tau = 1 - 4\xi$, $\sigma = 1 - \xi$, $\lambda = 1 - 63\xi/64$, and $\lambda' = 1 - 31\xi/32$. The machine combines the $h = 100$ bit and complex networks, the fractional-shift bridge of Section 4, the direct tensor FFT of Theorem 6.5 and Corollary 6.6, and the two-call root packing of Proposition 7.2.

For inner $O(N \log N)$ integer products and inputs below the log-length cutoff K_* of (46), the witness calls an independent Lean 4 formalization of the two-dimensional multiplier of [MS]. That sub-development spans 949 files and about 185,000 lines, with headline theorem `IntMulTM.intMul_TM2_nlogn`, and was built and kernel-checked specifically for this paper. The companion

paper [MS] distributes a separate Lean 4 development with headline theorem `IntMul.tmIntMul_evalsToInTime_n_log_n`. Syntactic absence of Chinese remainder maps is a property of the construction rather than part of the formal proposition. Table 7 lists the Lean 4 theorems corresponding to each section of the paper.

Table 7: Map from each part of the written $\kappa = 2^{-46}$ proof to the Lean 4 theorems that formalize it. Theorem names omit their `SubNLogN` namespace prefixes under `Math`, `Machine`, `Assembly`, and `S5`, except for `IntMulTM.intMul_TM2_nlogn`. Full qualified names appear in `SubNLogN/GateCheck.lean` and the archive `README.md`.

Paper component	Lean 4 theorems
Fact 3.3; Lemmas A.4 and A.5	`FinTM2` stream kernels with proved step bounds (`k1PairPerm_holds`, `k2Distribute_holds`, `k3Add_holds`, `k4RotateCtl_holds`, `c3LongRecordScan_holds`) on a framed, re-entrant procedure calculus.
Fact 3.4; Definition A.1, Proposition A.2	`bitNetCert_holds`, `complexNetCert_holds`, `deficitsH100_holds`.
Lemma A.6, Proposition A.7, Lemmas A.8–A.10	`shearFactorization_holds`, `existsValidFac_holds`, `packedAdd_correct`, `cCoreProc_correct`; machine level: `c6_holds`, `c7Extract_holds`.
Proposition A.11 ($C_1 = 14$, $\beta = 1/64$)	`prop71_final`, `prop71Guard_holds` (layer machine inside `walshMiddle_holds`).
Section 5 (Lemmas 5.1, 5.4; Theorem 5.8)	`walsh_exact`, `cnot_batch`, `positional_perm`; machine level: `cnot1Machine_holds`, `positionalPermMachine_of_leaves` (parametric in the layer contracts; hypotheses discharged by `cnotScan_holds` and `posPermSched_holds`).
Lemmas 4.2–4.6; Theorem 4.7	`lemmaA`; `lemmaB_i`, …, `lemmaB_vi`; `lemmaC_ii` and companions; `bridge_precision` (for arbitrary nearest rounding).
Theorem 6.5, Corollary 6.6	`direct_fft`, `walsh_middle`; machine level: `walshMiddle_holds`.
Proposition 7.2	`root_packing`, `bridgeFixed_contract`, `rootW8_Fnew`.
Cost table, gaps, and $\kappa = 2^{-46}$ row (Sections 7.3–7.5)	`headlineCost_walshNat46_W8`, `kappa_lt_gaps`, `wrec_le_rpow`; step budget: `walshCostClause_walshMidData`.
Inner multiplier, $\mathsf{M}(N) = O(N \log N)$	`mulBig_holds` (assembled from the `WP` stage programs that also yield `IntMulTM.intMul_TM2_nlogn`); below-cutoff pipeline `w9_pipeline_headline_runs`; standalone $O(n \log n)$ machine `IntMulTM.intMul_TM2_nlogn` (transported to `encNatPair` by `S5.intMul_TM2_nlogn_subNLogNSpec`).
Theorems 1 and 2 ($\kappa = 2^{-46}$)	`intMul_TM2_sub_nlogn_46`, `intMul_challenge`.

Departures from the written text. The Lean 4 construction differs from the written presentation in four minor technical choices:

(1) *Root parameters.* The formal proof adopts the root schedule $R = 2^{K_0}$ with $p_0 = 2b + 2K_0 + 6$ and $K_0 \le k + 2$ from [MS], matching our formalization of the two-dimensional multiplier. Here R is at most four times the length in Lemma 7.1, and the cost chain is proved directly for this schedule.

(2) *Rounding.* The formal machine rounds half-way ties upward rather than to nearest even. To match this choice, Theorem 4.7, Proposition 7.2, and all table-error bounds are proved in Lean for every nearest-integer rounding rule.

(3) *Cutoff.* The activation threshold K_* is defined as the smallest integer satisfying all parameter inequalities rather than by the explicit closed form (46).

(4) *Costs.* Machine steps are bounded directly without asymptotic master lemmas, unrolling the recurrences of Appendix A in the exact integer form $WF_k \le sF_{k-1} + Wc$. Here F_k bounds the step count per unit volume at recursion depth k and c is the per-level additive overhead.

In the opposite direction, the formal proof supplies three components that are left as black boxes or stated more briefly in the written text. Specifically, it gives a formally verified proof of the $h = 100$ network certificates of Fact 3.4 and a fully typed statement of the complex-network premises in part (b) of Definition A.1. It also establishes a re-entrant calling contract for the recursive exact complex core of Lemma A.10.

Availability. The two software artifacts of this paper, the Lean 4 formalization [Lean26] and the exact-arithmetic Python reference implementation [Code26], are available in our github repository. The Lean 4 archive includes `SubNLogN/Challenge.lean`, the axiom-check gate modules, and audit scripts that verify digest integrity and scan for `sorry`, `axiom`, `native_decide`, `implemented_by`, `extern`, and `unsafe`.

In [Lean26], the distributed source snapshot `b706618497aa` denotes the first 12 hexadecimal digits of the SHA-256 digest of `MANIFEST.sha256`. Its comment-stripped `.lean` sources are byte-identical to commit `3480d37`, and the snapshot builds cleanly under Lean 4.33.1 and Mathlib `0df444a3` via `lake build` with exit code 0. The authors verified this byte-identity using the comparison mode of `tools/check_semantic_freeze.py`, while the standalone mode of that script checks the archive against the recorded digests `MANIFEST_stripped.sha256` and `SPEC_LOCK.sha256`.

9 Discussion and open problems

This concluding section places our asymptotic bound, our test-scale reference implementation, and our architectural choices in perspective. We first explain why the saving exponent κ is governed by a linear rather than quadratic network bound. Next, we examine the astronomical activation threshold of the sub-$n \log n$ regime and the scope of our positional-permutation theorem, and weigh what our simplified pipeline trades away. We close with four open problems. Every part of this section is self-contained and may be read independently of the technical proofs.

9.1 Where the exponent comes from

The size of the saving exponent κ depends directly on how record-address bits are grouped into chunks for the middle transform; see Section 7. In [OAI26] and on the Bluestein comparison route of Appendix C, the layout map L is executed by width-K chunk interchanges. Under that strategy, the internal layer constraint $c < \beta(1-\tau)/\tau$ and the guard constraint $\varepsilon < 1/C_1$ force the chunk-exchange gap $\varepsilon c(1-\tau)$ below $\beta(1-\tau)^2/(C_1\tau)$. That bound caps κ quadratically in the bit network's deficit $1-\tau$.

Executing L and $(LB)^{-1}$ through exact Walsh-layer positional permutations via Theorem 5.8 removes this external chunk-exchange cost altogether. The binding constraint becomes the simultaneous butterfly-layer gap $\varepsilon(1-\lambda')$. Because Proposition A.11 requires $\lambda' > \lambda > \sigma$ and $\varepsilon < 1/C_1$, every admissible parameter tuple satisfies

$$\kappa < \varepsilon(1-\lambda') < \frac{1-\sigma}{C_1},$$

which is *linear* in the complex network's deficit $1-\sigma$.

This inequality bounds κ in terms of the *selected* rational exponent σ rather than the exact ceiling of the $h=100$ complex network itself. For our certified dyadic choice $1-\sigma=2^{-42}$ and guard exponent $C_1=14$, the ratio $(1-\sigma)/C_1$ is approximately $2^{-45.8}$, so $\kappa=2^{-46}$ is within a factor of two of that limit. At $C_1=20$, the ratio $(1-\sigma)/C_1\approx 2^{-46.3}$ is similarly approached by $\kappa=2^{-47}$.

The underlying $h=100$ complex network admits any σ with $s_c/W_c<m^\sigma$, meaning $1-\sigma<\gamma_c=-\ln(1-\eta_c)/\ln m\approx 2.654\cdot 10^{-13}$, compared with $2^{-42}\approx 2.274\cdot 10^{-13}$. Already the rational test of Proposition A.2 at $L=14$ certifies $\sigma=1-\frac{9}{8}\cdot 2^{-42}$, giving non-dyadic exponents slightly above $2^{-42}/C_1$ up to the fixed-network ceiling γ_c/C_1.

When the structural constants are instead fixed to $\beta=\frac{1}{2}$, $1-\tau=1-\sigma=2^{-50}$, and $C_1=20$ as printed in [OAI26, Prop. 14], the leaf condition $\lambda'>\sigma+\beta(1-\sigma)=1-2^{-51}$ imposes a tighter bound. Specifically, it forces $\kappa<\varepsilon(1-\beta)(1-\sigma)<2^{-51}/20<2^{-55}$, making $\kappa=2^{-56}$ the largest dyadic exponent 2^{-j} admitted by our analysis under those constants. Finally, our guard exponent $C_1=14$ matches the threshold $11+3=14$ noted in [OAI26, §5.4] and proved in Proposition A.11; we make no claim regarding smaller guard exponents.

9.2 Activation sizes, scope, and practical limitations

Although every step of our framework is implemented in exact arithmetic, the $h=100$ networks are certified symbolically rather than executed, and the sub-$n\log n$ asymptotic regime activates only at astronomical input sizes. Theorem 1 is purely an asymptotic bit-complexity result and yields no practical speedup at any feasible input size. Reaching even dimension $d=\lfloor k^\varepsilon\rfloor\geq 2$ at $\varepsilon=1/15$ requires transform length $R=2^k$ with $k\geq 2^{15}$. Satisfying the chunk-width threshold $K=\lfloor d^c\rfloor\geq 6$ at $\kappa=2^{-46}$ requires $\log_2 k\geq 3840\cdot 2^{42}\log_2 6$.

Those thresholds arise even before accounting for the guard constant $C_0\approx 10^{139}$ or the unspecified tape-stream thresholds in Facts 3.3–3.4. Moreover, even if all leading constants were 1, the saving factor $(\log_2 n)^\kappa$ at $\kappa=2^{-46}$ would remain below $1+10^{-12}$ for all $n\leq 10^{12}$. Accordingly, our reference package executes the multi-axis pipeline only under forced test schedules with small substitute networks, while the unforced schedule selects the two-dimensional multiplier at every feasible size; see Appendix E.

We also emphasize the exact scope of Theorem 5.8. The Walsh gadget permutes the m_0 bit *positions* of a complete binary record address that carries a long contiguous payload. It is not a sub-$n\log n$ routine for arbitrary permutations of 2^{m_0} records, and it does not replace the general field interchange of Lemma A.8 used for the bridge's $O(d)$ axis exposures, where lines carry no long retained payload. Because the gadget forms complex Walsh–Hadamard linear combinations between exact permutation boundaries, it operates outside pure data-movement models [HvdH25, §1.4].

9.3 What became simpler, and what it cost

Our outer reduction removes the number-theoretic and polynomial-ring machinery that previously surrounded the two $h=100$ finite networks; see Table 1. Working on one power-of-two grid $R=t_1\cdots t_d=2^k$ eliminates the Chinese remainder map onto distinct odd primes, trial-division prime search, and coprime-length Gaussian resampling with its Fourier-grid expansion. Modulating the sinc-Gaussian kernel by a negative half-bin phase removes the Nyquist special case of [MS] and gives one two-term cyclic solver with $\|B_a^{-1}\|_\infty\leq\frac{4}{7}t_a$ uniformly across all cycle lengths, while direct axis exposures replace $O(d^2)$ adjacent swaps by $O(d)$ interchanges.

In the middle transform, exact Walsh-layer positional permutations replace $O(k/K)$ chunk interchanges, Bluestein's chirp, last-axis twists, and $\mathbb{C}[y]/(y^r+1)$ ring convolutions by a direct

complex tensor FFT. This also eliminates M-fold error amplification, $\lg M$-bit output widening, and $\Theta(rP)$-bit packed products. The direct FFT delegates only $O(P^2)$-bit integer products under a single precision contract $P = O(p + \lg R)$. Consequently, the written proof uses zero results of [HvdH21] and imports from [OAI26] only the stream primitives and $h = 100$ network certificates recorded in Section 3.4, while the Lean 4 proof imports no external results at all.

These simplifications carry concrete costs of their own. The sub-$n \log n$ saving still originates entirely in the two $h = 100$ finite networks of [OAI26]. Turning the simultaneous butterfly layer into an exact positional-permutation engine requires six completed layer calls per slice-batched CNOT stage, totaling at most $36N_{\text{slice}}$ layer calls per positional map. It also requires a temporary precision lift of $8d$ fractional bits so intermediate dyadic denominators never truncate; see Section 5.

Furthermore, reaching $\kappa = 2^{-46}$ beyond the $\kappa = 2^{-56}$ printed-constants bound still requires re-deriving the interchange, packed-addition, and butterfly-layer engines with exposed parameters in Appendix A. Once those network-backed tape primitives are in place, however, the entire outer multiplier reduces to one data-independent convolution kernel per spread axis, a power-of-two FFT, and one two-term cyclic recurrence per spread axis.

9.4 Open problems

Four concrete questions remain open for future work:

(1) *Larger deficits and smaller guards.* Construct finite linear networks with relative rank deficit substantially larger than $\eta_c \approx 3.67 \cdot 10^{-12}$, or establish butterfly-layer guard bounds with exponent $C_1 < 14$. By Section 9.1, either improvement translates linearly into a larger saving exponent κ.

(2) *Explicit numerical cutoff.* Extract explicit constants for all multi-stack or multitape stream primitives to obtain a concrete numerical upper bound on the threshold n_* in Theorem 1.

(3) *Other applications of Walsh-layer permutations.* Determine whether exact positional permutations built from selective butterfly layers improve the exponents of other problems accelerated by the networks of [OAI26]. Two natural candidates are matrix transposition [OAI26, §11] and the exact discrete Fourier transform [OAI26b].

(4) *Downstream arithmetic primitives.* Decide whether the CRT-free fractional-shift bridge also simplifies downstream operations such as integer division and square roots [OAI26, §10]. Because both operations reduce to multiplication via Newton iteration and $\mathsf{M}(n)/n$ is nondecreasing for our bound, they inherit the $O(n(\log n)^{1-2^{-46}})$ complexity in the standard way, though we have not checked those details or base conversion.

References

[Blu70] L. I. Bluestein. A linear filtering approach to the computation of discrete Fourier transform. *IEEE Transactions on Audio and Electroacoustics*, 18(4):451–455, 1970.

[Code26] Piaoyang Cui, Yiming Liu, and David P. Woodruff. Exact-arithmetic Python reference implementation accompanying "A simpler algorithm for integer multiplication below $n \log n$, with a formally verified proof". Exact-arithmetic Python 3.13 reference implementation of Algorithm 1 (Appendix E); directory `python/` of the repository, content snapshot `a4d39ed31d0e` (first 12 hex digits of the SHA-256 of `MANIFEST.sha256`), 2026. Available at https://github.com/lockqueue/intmul.

[DKSS13] Anindya De, Piyush P. Kurur, Chandan Saha, and Ramprasad Saptharishi. Fast integer multiplication using modular arithmetic. *SIAM Journal on Computing*, 42(2):685–699, 2013.

[dMU21] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[Für07] Martin Fürer. Faster integer multiplication. In *Proceedings of the 39th Annual ACM Symposium on Theory of Computing*, pages 57–66. ACM, 2007.

[Für09] Martin Fürer. Faster integer multiplication. *SIAM Journal on Computing*, 39(3):979–1005, 2009.

[HvdH21] David Harvey and Joris van der Hoeven. Integer multiplication in time $O(n \log n)$. *Annals of Mathematics*, 193(2):563–617, 2021.

[HvdH25] David Harvey and Joris van der Hoeven. Integer multiplication is at least as hard as matrix transposition. In *Proceedings of the 66th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1366–1381. IEEE Computer Society, 2025.

[HvdHL16] David Harvey, Joris van der Hoeven, and Grégoire Lecerf. Even faster integer multiplication. *Journal of Complexity*, 36:1–30, 2016.

[Lean26] Piaoyang Cui, Yiming Liu, and David P. Woodruff. Integer multiplication below $n \log n$ on multitape Turing machines: a Lean 4 formalization. Lean 4 formalization accompanying this paper (comment-stripped `.lean` sources byte-identical to commit `3480d37`; source snapshot `b706618497aa` (first 12 hex digits of the SHA-256 of `MANIFEST.sha256`); Lean 4.33.1, Mathlib `0df444a3`), statement file `SubNLogN/Challenge.lean`, 2026. Available at https://github.com/lockqueue/intmul.

[mC20] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020.

[MS] Piaoyang Cui, Yiming Liu, and David P. Woodruff. Integer multiplication in $O(n \log n)$ time on a single two-dimensional grid, with a formally verified proof. Companion manuscript. Available at https://github.com/lockqueue/intmul, 2026.

[NQ78] H. J. Nussbaumer and P. Quandalle. Computation of convolutions and discrete Fourier transforms by polynomial transforms. *IBM Journal of Research and Development*, 22(2):134–144, 1978.

[Nus80] H. J. Nussbaumer. Fast polynomial transform algorithms for digital convolution. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 28(2):205–215, 1980.

[OAI26] OpenAI. Integer multiplication below $n \log n$. OpenAI Math Release preprint, https://github.com/openai/math, September 2026. Preprint dated September 23, 2026 (`preprints/Integer-multiplication-below-n-log-n-September-23-2026/paper.pdf`).

[OAI26b] OpenAI. An explicit power saving for the exact discrete Fourier transform. OpenAI Math Release preprint, 2026. Cited as [22] in [OAI26].

[SS71] Arnold Schönhage and Volker Strassen. Schnelle Multiplikation großer Zahlen. *Computing*, 7:281–292, 1971.

A Butterfly layer and interchanges with general parameters

This appendix builds the two tape primitives with sub-linear overhead that our algorithm uses, the arbitrary-width field interchange (Lemma A.8) and the simultaneous butterfly layer (Proposition A.11), from the two black-box inputs of Section 3.4. Those inputs are the linear-time elementary tape-stream operations of Fact 3.3 [OAI26, Lemmas 3–4] and the $h = 100$ finite-network certificates of Fact 3.4 [OAI26, Props. 7–8]. In [OAI26, Prop. 14], the simultaneous butterfly layer is stated only at fixed conservative constants $\beta = \frac{1}{2}$, $\tau = \sigma = 1 - 2^{-50}$, and $C_1 = 20$, which would limit our main route to $\kappa = 2^{-56}$. Rather than invoking [OAI26, Prop. 14] outside its printed tuple, we follow the arguments of [OAI26, §§4–5] step by step with all structural and guard parameters exposed, certifying $\tau = 1 - 2^{-40}$, $\sigma = 1 - 2^{-42}$, and $C_1 = 14$ to reach $\kappa = 2^{-46}$. Throughout, the finite networks of [OAI26, §§3.1–3.3, Lemmas 5–6] enter solely through the finite algebraic data of Definition A.1 [OAI26, Props. 7–8, §§3.4–3.5]. A reader who accepts the general field interchange of Lemma A.8, the butterfly layer of Proposition A.11, and the $h = 100$ exponents of Proposition A.2 may skip this appendix on a first reading.

Conventions. We use the machine model, array layout, logical bit volume V, and header code Γ (60) of Section 3.2 (restated in Appendix D). Four conventions govern the procedures below. A trailing payload of B bits may be treated as a final field of range $[B]$. Fields that an operation leaves untouched are called *spectators*, and adjacent spectator lengths are multiplied together before a primitive is called. Each primitive receives its array shape as a fixed number of positive integer field lengths written in the header code Γ; when a procedure needs a longer list of lengths, the cost of that list is charged to the caller. Implied constants may depend on the fixed networks, fixed rational exponents, and fixed comparison constants, never on field lengths or data.

As in Sections 5 and 6, q denotes the dyadic grid precision of the layer, and Q_q truncates real and imaginary parts toward zero onto $2^{-q}\mathbb{Z}$, satisfying $\|Q_q z\|_\infty \le \|z\|_\infty$ and $\|Q_q z - z\|_\infty < \sqrt{2} \cdot 2^{-q}$. Finally, q_{rad} denotes a fixed prime radix.

A.1 Fixed network data, certified exponents, and elementary procedures

This subsection packages the finite algebraic data of the two $h = 100$ circuits, certifies their sharpened exponents and guard degree, and establishes two linear-time tape-stream helper lemmas.

Definition A.1 isolates the exact finite data imported from Fact 3.4: a bit network over $\mathbb{F}_2$ and a complex network over $\mathbb{Z}[i, 1/2]$ whose total edge-rank sums s_b and s_c lie strictly below $W_b m$ and $W_c m$. No internal wiring detail beyond these finite counts and endpoint identities is used anywhere in the proofs that follow.

Definition A.1 (Fixed network data)**.** The following finite data are fixed by Fact 3.4:

(a) *Bit network.* A finite pointwise XOR circuit on W_b roles, auxiliary roles included, whose completed scalar action is a permutation ρ_b of all inputs, for arbitrary initial auxiliary values. Each terminal and gate carries a rational $m \times m$ matrix, common to all incidences of a gate. With $A_e = M_{\mathrm{head}(e)} - M_{\mathrm{tail}(e)}$ and $s_b = \sum_e \operatorname{rank}_{\mathbb{Q}} A_e$,

$$M_{\mathrm{out}(\rho_b(w))} - M_{\mathrm{in}(w)} = I_m \quad \text{for every input role } w, \qquad s_b < W_b m.$$

(b) *Complex network.* A finite scalar linear circuit on W_c roles with coefficients in $\{0, \pm 1, \pm\frac{1}{2}\}$ and binary subspace labels as in [OAI26, Prop. 8, §3.5]. Adjacent labels are nested, the nonzero edge residuals have fixed orthonormal bases, and their total dimension is $s_c < W_c m$. The scalar circuit exchanges its two data banks with the prescribed signs and restores its auxiliary values for arbitrary initial values; its source, sink and bank corrections are those of [OAI26, proof of Prop. 8, §3.5, pp. 19–20]. We use only that this scalar circuit is fixed and finite; no bound on its number of gates is assumed (Remark A.13).

For the triple-network constructions at $h = 100$ ([OAI26, §3.1, (3)–(4); Props. 7–8, §§3.4–3.5; §5.4, p. 35]),

$$m = 10^6, \quad W_b = 177\,176\,569\,091\,445\,000\,000, \quad s_b = 177\,176\,569\,088\,785\,861\,287\,000\,000,$$

$$W_c = 1\,873\,807\,244\,643\,542\,670\,000, \quad s_c = 1\,873\,807\,244\,636\,671\,267\,308\,000\,000,$$

with exact relative deficits ([OAI26, §3.6])

$$\eta_b = \frac{W_b m - s_b}{W_b m} = \frac{339}{22\,587\,335\,000\,000}, \qquad \eta_c = \frac{W_c m - s_c}{W_c m} = \frac{73}{19\,906\,842\,167\,500}.$$

Following [OAI26, §5.4], we define from the complex-network counts (m, W_c, s_c) the guard constants

$$E_{\mathrm{net}} = 64(W_c + m + 1)^3, \quad B_{\mathrm{net}} = s_c + E_{\mathrm{net}}, \quad \nu_{\mathrm{net}} = \min\{j \geq 1 : m^j \geq B_{\mathrm{net}}\}, \quad C_0 = 128\, m\, B_{\mathrm{net}}^2 \tag{47}$$

Here we write ν_{net} rather than ν as in [OAI26, §5.4] to avoid collision with the bridge alias index $\nu \in \mathbb{Z}$ of Section 4. At $h = 100$ the exact integer values are

$$\begin{aligned}
W_b m - s_b &= 2\,659\,138\,713\,000\,000, \qquad W_c m - s_c = 6\,871\,402\,692\,000\,000,\\
E_{\mathrm{net}} &= 421\,070\,402\,185\,313\,436\,016\,911\,541\,181\,936\,724\,563\,007\,566\,900\,236\,040\,789\,184\,640\,064,\\
B_{\mathrm{net}} &= 421\,070\,402\,185\,313\,436\,016\,911\,541\,181\,936\,724\,564\,881\,374\,144\,872\,712\,056\,492\,640\,064,\\
C_0 &= 22\,694\,436\,300\,352\,206\,084\,304\,065\,594\,397\,346\,702\,264\,594\,168\\
&\quad\ 261\,607\,007\,508\,662\,043\,469\,612\,184\,775\,064\,463\,128\,060\,707\,693\\
&\quad\ 386\,195\,338\,619\,369\,397\,054\,146\,227\,284\,214\,284\,288\,000\,000,
\end{aligned}$$

where the three lines of C_0 concatenate to a 140-digit integer satisfying $C_0 \approx 2.2694 \times 10^{139}$ and $\log_2 C_0 = 462.93\ldots$. Table 8 records the exact closed-form counts at representative ground-set sizes h, including $h = 22$ and $h = 39$, the smallest sizes at which the complex and the bit family, respectively, have a positive rank saving $Wm - s$.

Proposition A.2 translates the finite integer counts of Definition A.1 into the rational exponents $\tau = 1 - 2^{-40}$ and $\sigma = 1 - 2^{-42}$ and the guard degree $\nu_{\mathrm{net}} = 11$. Replacing the coarse bound $\ln 10^6 < 20$ of [OAI26, §3.6] by the elementary rational check $\ln 10^6 < 14$ sharpens the deficits from 2^{-50} to 2^{-40} and 2^{-42}, contributing directly to the improvement from $\kappa = 2^{-56}$ to $\kappa = 2^{-46}$.

Proposition A.2 (Network exponents at $h = 100$)**.** *For the integers W_b, s_b, W_c, s_c of the triple networks of [OAI26, §3] at $h = 100$ ($m = 10^6$; Definition A.1),*

$$\frac{s_b}{W_b} < m^{1-2^{-40}}, \qquad \frac{s_c}{W_c} < m^{1-2^{-42}},$$

so $\tau = 1 - 2^{-40}$ and $\sigma = 1 - 2^{-42}$ are admissible in Proposition A.11. Moreover $\nu_{\mathrm{net}} = 11$, so $C_1 = 20$ and $C_1 = 14$ are both admissible. These are exact arithmetic properties of the displayed integers; that the integers are the counts of working networks is Fact 3.4.

Table 8: Role counts, edge-rank sums, and rank savings of the triple networks of [OAI26, §3] at representative sizes h. With $m = h^3$, columns list the role count W, edge-rank sum s, signed saving $Wm - s$, and certified rational upper bound on $\ln m$; only $h = 100$ is used in our theorems.

h	network	W	s	$Wm - s$	upper bound $\ln m$
21	bit	3 244 397 863 700	30 046 370 943 598 100	−2 327 872 400	9.133567313171
21	complex	6 145 204 591 400	56 910 739 919 072 200	−198 116 800	9.133567313171
22	bit	5 628 295 349 600	59 930 092 117 403 200	−3 234 862 400	9.273127360075
22	complex	11 249 136 760 400	119 780 808 120 388 800	104 350 400	9.273127360075
39	bit	4 329 455 044 049 525	256 818 943 756 887 996 302	1 085 777 173	10.990684938389
39	complex	16 598 761 236 611 294	984 621 917 049 702 208 108	744 843 140 678	10.990684938389
100	bit	177 176 569 091 445 000 000	177 176 569 088 785 861 287 000 000	2 659 138 713 000 000	13.815510557965
100	complex	1 873 807 244 643 542 670 000	1 873 807 244 636 671 267 308 000 000	6 871 402 692 000 000	13.815510557965

Proof. By Definition A.1 ([OAI26, §3.6, (10)]) the relative deficits are $\eta_b = 339/22\,587\,335\,000\,000$ and $\eta_c = 73/19\,906\,842\,167\,500$, with $s_b/W_b = m(1-\eta_b)$ and $s_c/W_c = m(1-\eta_c)$. For $u > 0$ and any $L \geq \ln m$,

$$m^{1-u} = m\,e^{-u\ln m} > m(1 - u\ln m) \geq m(1 - Lu). \tag{48}$$

Hence $s/W < m^{1-u}$ whenever $\eta \geq Lu$; with $u = 2^{-e}$, it suffices to check $\eta \cdot 2^e \geq L$. Take $L = 14$: since $\sum_{j=0}^{11} 7^j/j! = 41\,438\,755\,442/39\,916\,800 > 1000$, we have $e^7 > 10^3$ and hence $\ln 10^6 < 14$. The exact integer comparisons

$$339 \cdot 2^{40} = 372\,734\,441\,816\,064 > 316\,222\,690\,000\,000 = 14 \cdot 22\,587\,335\,000\,000,$$

$$73 \cdot 2^{42} = 321\,057\,395\,310\,592 > 278\,695\,790\,345\,000 = 14 \cdot 19\,906\,842\,167\,500$$

give $\eta_b > 14 \cdot 2^{-40}$ and $\eta_c > 14 \cdot 2^{-42}$. For the guard degree, $10^{21} < W_c + m + 1 < 2 \cdot 10^{21}$ and $s_c < 2 \cdot 10^{27}$, so $10^{60} < B_{\text{net}} < 512 \cdot 10^{63} + 2 \cdot 10^{27} < 10^{66}$; since $m^{10} = 10^{60}$ and $m^{11} = 10^{66}$, we obtain $\nu_{\text{net}} = 11$. □

The next remark records a tighter rational enclosure for $\ln 10^6$ and verifies that the dyadic deficit exponent $1 - 2^{-40}$ cannot be reached by the $h = 100$ complex network.

Remark A.3 (Sharper logarithm enclosure and sharpness). At $h = 100$ the scaled deficits are $\eta_b 2^{40} = 16.50\ldots$ and $\eta_c 2^{42} = 16.12\ldots$, so the bound $\ln m < 20$ used in [OAI26, §3.6] to certify $1 - 2^{-50}$ does not certify $1 - 2^{-40}$ or $1 - 2^{-42}$. Any certified upper bound $L \leq 16.125$ suffices, and $L = 14$ in Proposition A.2 is already elementary. A twelve-decimal rational enclosure follows from $\ln 10^6 = 36\,\operatorname{atanh}\frac{1}{3} + 12\,\operatorname{atanh}\frac{1}{9}$: with $S_N(z) = \sum_{j<N} z^{2j+1}/(2j+1)$ and $0 < \operatorname{atanh} z - S_N(z) < z^{2N+1}/((2N+1)(1-z^2))$, taking $N = 20$ yields

$$13.815510557964 < \ln 10^6 < 13.815510557965$$

by finite rational comparisons (giving margins $\eta_b 2^{40} - L = 2.6864\ldots$ and $\eta_c 2^{42} - L = 2.3124\ldots$). Moreover, the exponent $1 - 2^{-40}$ is false for the complex network at $h = 100$: with $u = 2^{-40}$ we have $\eta_c 2^{40} < 5$ and $10 < \ln m < 20$, so $1 - e^{-u\ln m} \geq u\ln m - \frac{1}{2}u^2(\ln m)^2 > u(10 - 200u) > 5u > \eta_c$, whence $s_c/W_c = m(1-\eta_c) > m^{1-2^{-40}}$ (and similarly for the bit network at $h = 39$).

Lemma A.4 packages the linear-time stream, arithmetic, and controlled-shift operations on a fixed number of tapes that follow from Fact 3.3. Every recursive interchange and butterfly routine in this appendix uses these $O(V)$ passes for its base cases, digit moves, role-stream splits, and guarded shifts.

Lemma A.4 (Elementary streams)**.** *Fix a radix $q_{\mathrm{rad}} \geq 2$. The following operations have linear bit cost on a fixed number of tapes:*

(1) any fixed permutation of the pair of values at two specified radix-q_{rad} digit positions;

(2) moving one specified radix-q_{rad} digit position to another position, preserving the order of the intervening positions;

(3) splitting at a specified radix-q_{rad} digit, applying a fixed pointwise map to the aligned streams, and merging;

(4) distributing complete rows cyclically among a fixed number W of streams and, when the number of rows in each prefix is a multiple of W (so that all streams have the same shape), merging them in a fixed order of the row classes;

(5) exact fixed-width addition, subtraction, sign changes, and binary shifts by a supplied integer exponent t, when the necessary guard bits are present; a shift of N words costs $O(N(w_{\mathrm{in}}+w_{\mathrm{out}})+\lg(2+|t|))$, linear in the input and output widths plus the length of the exponent's encoding, which is read once.

Moreover, let a target field of range $[Q]$, $Q = q_{\mathrm{rad}}^{b}$, follow its control fields, each relevant control having $O(b)$ digits. A cyclic shift of the target by an offset computable from the controls in $b^{O(1)}$ time costs $O(V)$, and so does multiplication of the target coordinate by a fixed rational unit modulo Q whose numerator and denominator are prime to q_{rad}.

Proof idea. We realize digit permutations, digit moves, and cyclic row distributions by splitting the input across a fixed number of auxiliary tapes and merging in the target order. Counter updates, arbitrary-exponent guarded shifts, and controlled modular shifts or unit multiplications are then amortized into single linear scans over the active words or fibers.

Proof. Items (1)–(4). For (1), split the array across q_{rad}^2 tapes according to the two digit values; within every stream the remaining address fields retain their order, so merging with permuted labels gives the desired swap. For (2) and (3), use q_{rad} streams and change the place in the nested merge where the selected digit is inserted. The number of streams is fixed, and complete spectator fields and trailing payloads are copied without decoding. For (4), send row r to stream $r \bmod W$; each stream preserves row order, and merging reads the streams in the prescribed order of row classes, copying every row a constant number of times.

Overheads and item (5). Binary counters are stored least significant bit first. An increment, including the head return over the changed prefix, has constant amortized cost, while wraparound and reset costs are charged to the preceding traversal. Initial shape preparation takes at most $C(\log_2(2V))^J$ steps for fixed C, J, which is at most $C'V$ for all $V \geq 1$; this covers counters, head returns, and clearing visited regions.

Fixed-width coefficient arithmetic uses standard carry scans, and bit reversal inside a coefficient, when needed, is a charged linear pass. For a shift by a supplied exponent t, we partition the source and destination words into the retained interval and the intervals of known zeros or redundant sign bits. We then copy the retained interval and fill the remaining bits in a single traversal. Exact right shifts discard only known zero low bits, and local counter preparation costs $O(\log(2w))$ per w-bit word, absorbed by its scan. Thus a shift by t runs in one traversal rather than t single-bit shifts, as [OAI26, §5.4, p. 36] also notes, proving (5).

Controlled shifts and units. Fix the preceding coordinates. The target fiber consists of Q consecutive suffix blocks, and a cyclic rotation splits that fiber at the computed cut and concatenates

the two intervals in reverse order. Computing the cut takes time polynomial in b, whereas the fiber holds at least q_{rad}^b bits, so the fiber scan absorbs the offset work uniformly in b.

Multiplication by a fixed positive integer a prime to Q splits the target coordinates into the at most a intervals where $x \mapsto ax - kQ$ lies in $[Q]$. On each interval the outputs increase and fall in one residue class modulo a, and those classes are distinct because $\gcd(a, Q) = 1$. Merging the fixed number of streams in output order produces the product, while the inverse uses the reversed schedule.

A negative multiplier composes with negation $y \mapsto -y \bmod Q$, which fixes the block at $y = 0$ and reverses the order of the remaining $Q - 1$ blocks while preserving each block's internal order. To do so, push the blocks onto a temporary tape and reverse each popped block once more, as in [OAI26, proof of Lemma 4, §2.1]. A rational unit composes an integer multiplication with an inverse one, using the fixed-tape details of [OAI26, Lemmas 3–4, §2.1] summarized in Fact 3.3. Those same primitives are applied in [OAI26, proof of Lemma 10, proof of Prop. 11, §5.2, and §5.3 (25)]. □

Lemma A.5 shows that whenever each record carries a sufficiently long payload of at least q^J bits, any polynomial-in-q address calculation per record amortizes into linear total bit time $O(V)$. We need this amortization to execute the address-dependent Walsh diagonals S_U and CZ of Section 5 and the twiddle-table lookups of Section 6 with $O(V)$ overhead.

Lemma A.5 (Long-record address-dependent scans)**.** *Fix a recordwise address routine and payload routine and constants C_0, C_1, C_2, J. Suppose that, on an array of $M \geq 1$ records, the address preparation of every record takes at most $C_1 q^J$ steps, the payload processing of a record of ρ bits takes at most $C_2\rho$ steps, and a one-time setup takes at most $C_0 q^J$ steps. Then the total time is at most $C_0 q^J + M(C_1 q^J + C_2\rho_{\max}) + O(V)$; in particular, if every record has at least q^J bits (which holds for records of $\Theta(rq)$ bits once $r \geq q^J$, as in every family with $r/q^j \to \infty$ for each fixed j), the total time is $O(V)$ with a constant depending only on C_0, C_1, C_2. Consequently, an address-dependent phase in $\{1, -1, i, -i\}$ may be computed once per record and applied to all its coefficients in $O(V)$ time, and selecting a twiddle from a table of $O(rq)$ bits by one scan per record has $O(V)$ overhead (with general coefficient multiplications charged separately).*

Proof. Let the M records have bit widths $\rho_1, \ldots, \rho_M$, so $V = \rho_1 + \cdots + \rho_M \leq M\rho_{\max}$. Summing over all records, setup, address preparation, and payload work take at most $C_0 q^J + MC_1 q^J + C_2 V$ steps, and traversing the array on a fixed number of tapes adds $O(V)$. When every record has at least q^J bits, both $q^J \leq V$ and $Mq^J \leq V$ hold, yielding $O(V)$ total steps.

For records of $\Theta(rq)$ bits, maintaining and decoding the record address, scanning a polynomial-size mask or position list, and running a fixed polynomial-time address routine cost at most Cq^J steps per record. Summing over M records gives $O(M(q^J + rq)) = O(Mrq) = O(V)$, while scanning and rewinding an $O(rq)$-bit twiddle table also costs $O(rq)$ per record. The same long-record amortization appears in [OAI26, §5, p. 26 and §5.3, pp. 33–34]. □

A.2 Rational shears and power-width interchanges

This subsection uses the $h = 100$ bit network to swap two disjoint chunks of $e = m^k$ digits in sub-linear overhead $O(Ve^\tau)$.

Lemma A.6 shows how any rational $m \times m$ matrix A of rank a acts as an additive address shear $(H, D) \mapsto (H + AD, D)$ modulo q_{rad}^b using only a smaller field interchanges plus $O(V)$ linear-stream work. We need this factorization so that an edge of rank a in the bit network spawns a recursive interchanges rather than m.

Lemma A.6 (Lower triangular factorization and shear)**.** *Every rational $m \times m$ matrix A of rank a has a factorization $A = E_1 \Pi E_2$, where E_1, E_2 are invertible lower triangular rational matrices and Π is a partial permutation matrix with exactly a ones. For a fixed finite collection of such matrices there is a fixed odd prime q_{rad} such that, on two ordered groups $H = (H_1, \dots, H_m)$ and $D = (D_1, \dots, D_m)$ of fields of range $[q_{\mathrm{rad}}^b]$ with every H_i before every D_j and arbitrary spectator fields, the shear $(H, D) \mapsto (H + AD, D)$ uses exactly a interchanges of pairs of b-digit fields, each between an H field and a D field, and $O(V)$ further work.*

Proof. Factorization. While the active submatrix is nonzero, pick its topmost nonzero row and the rightmost nonzero entry in that row as pivot. Right multiplication by lower triangular elementary matrices clears entries to the left of the pivot, and left multiplication by lower triangular elementary matrices clears entries below it. Scale the pivot to 1 and remove its row and column from the active index sets while keeping the original coordinate order. Later steps remain lower triangular and preserve earlier pivots, leaving a partial permutation matrix Π with a ones after a steps; inverting the accumulated triangular operations gives $A = E_1 \Pi E_2$ [OAI26, Lemma 9, §4.1].

The shear. Fix the factorizations for the finite matrix collection once and for all. By finite rational arithmetic and trial division, pick an odd prime q_{rad} that avoids all denominators of the triangular factors and their inverses, as well as all numerators of their diagonal entries [OAI26, proof of Lemma 10, §4.1]. Every identity then reduces modulo q_{rad}^b with invertible lower triangular factors.

Because m is fixed, any lower triangular map runs in $O(V)$ time by updating coordinates in descending physical order from earlier, still-unmodified coordinates via Lemma A.4. We transform D by E_2 and H by E_1^{-1}, add ΠD to H, and undo both triangular maps. Each entry 1 of Π at (i, j) is realized by

$$D_j \leftarrow D_j + H_i, \qquad (H_i, D_j) \leftarrow (D_j, H_i), \qquad D_j \leftarrow H_i - D_j,$$

which maps $(h, d) \mapsto (h + d, d)$ using one interchange of b-digit fields and two linear updates to the later field [OAI26, proof of Lemma 10, §4.1]. □

Proposition A.7 combines the bit network of Definition A.1 with the shears of Lemma A.6 to swap two disjoint m^k-digit chunks in $O(Ve^\tau)$ time whenever $s_b/W_b < m^\tau$. Each recursion level distributes rows across W_b role streams of volume V/W_b and makes only $s_b < W_b m$ child calls on chunks m times shorter, turning the finite rank deficit into the exponent $\tau < 1$.

Proposition A.7 (Parameterized power-width interchange)**.** *Take the bit network of Definition A.1, and choose the prime q_{rad} of Lemma A.6 for the edge matrices A_e, also avoiding the denominators of the assigned terminal and gate matrices. For every fixed rational $\tau \in (0, 1)$ with $s_b/W_b < m^\tau$, two disjoint contiguous chunks of $e = m^k$ radix-q_{rad} digits can be interchanged in $O(Ve^\tau)$ time if a row field preceding both chunks has length divisible by W_b^k. All other fields are preserved, and the machine, tape count and time constant are uniform in all field lengths and data.*

Proof idea. We split the array cyclically into W_b role streams of volume V/W_b and interpret each vertex matrix M as a shear frame $\Phi_M(H, D) = (H + MD, D)$. Executing edge differences Φ_{A_e} via Lemma A.6 makes s_b recursive calls of width e/m per level, unrolling to $O(Ve^\tau)$ on a fixed number of tapes via depth-first stream parking.

Proof. Construction and frame invariant. Write $W = W_b$ and $s = s_b$. The base case $k = 0$ is Lemma A.4(1). For $k > 0$, subdivide each chunk into m fields of $b = e/m$ digits and split whole rows cyclically across W role streams. Writing the parent row index as $Wg + (w - 1)$, role w receives row index g, volume V/W, and row count divisible by W^{k-1}.

For any rational matrix M, let Φ_M map (H, D) to $(H + MD, D)$, so $\Phi_{M'}\Phi_M = \Phi_{M'+M}$ and $\Phi_M^{-1} = \Phi_{-M}$. On each circuit edge we apply Φ_{A_e} via Lemma A.6 using recursive pivot interchanges, and at each gate we apply the circuit operation pointwise to aligned role streams. Defining the logical array at a vertex with frame Φ_M as Φ_M^{-1} of the physical array, edges preserve the logical array and common address permutations commute with pointwise gates. Input role w therefore arrives at output role $\rho_b(w)$ carrying $\Phi_{M_{\mathrm{out}(\rho_b(w))}}\Phi_{-M_{\mathrm{in}(w)}} = \Phi_{I_m}$.

Merging output role $\rho_b(w)$ back into row class w applies $H \leftarrow H + D$ on every original row, including auxiliary rows with arbitrary initial contents [OAI26, proof of Prop. 11, §4.2]. Three updates $D \leftarrow D - H$, $H \leftarrow H + D$, and $D \leftarrow H - D$ then realize $(H, D) \mapsto (D, H)$ from one circuit call and two $O(V)$ passes [OAI26, proof of Prop. 11, pp. 22–23].

Recurrence, fixed tapes, and descriptors. One level makes s recursive calls on volume V/W plus $O(V)$ work, giving $F_0 = O(1)$ and $F_k \le (s/W)F_{k-1} + O(1)$. Because $a = s/W < m^\tau$, we have $\sum_{j=0}^{k} a^j = O(m^{k\tau})$ and $F_k = O(e^\tau)$ [OAI26, proof of Prop. 11, (12)]. Depth-first scheduling uses W role tapes, fixed I/O and work tapes, one parking-stack tape, and one descriptor-stack tape.

Before each child call, inactive streams are pushed onto the parking stack while parent role areas are erased during traversal. Popped streams are written back from the right end of their known destination intervals to restore direction without scanning ancestor data, costing $O(V)$ at the parent [OAI26, §4.2, p. 23]. At depth j from root width $e_0 = m^{k_0}$, inactive digits act as spectators, so $V_j \ge q_{\mathrm{rad}}^{2e_0}$ and $\log V_0 = O(\log(2V_j))$, absorbing $\mathrm{poly}(\log(2V_j))$ child-descriptor work into $O(V_j)$ [OAI26, §4.2, pp. 23–24]. □

A.3 Arbitrary-width interchanges

This subsection removes both the power-of-m width restriction and the row-divisibility precondition of Proposition A.7.

Lemma A.8 swaps any two disjoint u-bit binary address fields in $O(Vu^\tau)$ time, uniformly across all surrounding field sizes. Section 4 invokes this general interchange directly for the bridge's $O(d)$ axis exposures, and Lemma A.9 uses it to align chunks for packed selected-bit addition.

Lemma A.8 (Parameterized arbitrary-width interchange)**.** *Under the hypotheses of Proposition A.7, for the same fixed τ, a bit array on $[A] \times [2^u] \times [G] \times [2^u] \times [B]$, $u \ge 1$, can be transformed by $(x, h, g, d, z) \mapsto (x, d, g, h, z)$ in $O(Vu^\tau)$ time, uniformly in all positive A, G, B and all data (including shape processing, padding, unpadding and cleanup, with $B = 1$ permitted). Two fields whose widths differ by one bit are interchanged across the intervening field by one such interchange and one single-bit move.*

Proof. Radix-q_{rad} chunks. First consider chunks of range $[q_{\mathrm{rad}}^e]$. Set $k = \lceil \log_m e \rceil$ and $k_{\mathrm{hi}} = \min\{j \ge 0 : q_{\mathrm{rad}}^{2j} \ge W_b^k\}$. Bounded widths use individual digit swaps; for large e, we have $1 \le k_{\mathrm{hi}} < e$ with $k_{\mathrm{hi}} = O(\log(2e))$. Split each chunk into its top k_{hi} digits and the rest, writing $h = (h_+, h_-)$ and $d = (d_+, d_-)$.

We swap h_+ and d_+ digit by digit and move h_+ right after d_+, mapping $(h_+, h_-, g, d_+, d_-) \mapsto (d_+, h_+, h_-, g, d_-)$ in $O(Vk_{\mathrm{hi}})$ steps. Now (d_+, h_+) forms a preceding row field of range $R_{\mathrm{row}} = q_{\mathrm{rad}}^{2k_{\mathrm{hi}}} \ge W_b^k$. We pad that row range in each prefix to $R'_{\mathrm{row}} = W_b^k \lceil R_{\mathrm{row}}/W_b^k \rceil < 2R_{\mathrm{row}}$.

Next, expand $e - k_{\mathrm{hi}} = \sum_{j=0}^{k} a_j m^j$ with $0 \le a_j < m$, and swap matching power-of-m pieces via Proposition A.7 in cost $O(V)\sum_j a_j m^{j\tau} = O(Ve^\tau)$. Deleting the padded rows and reversing only the joining moves produces (d_+, d_-, g, h_+, h_-), with descriptor and movement overhead $O(V \log(2e)) = O(Ve^\tau)$. These steps follow [OAI26, proof of Lemma 12, §4.3], and none uses the specific exponent $1 - 2^{-50}$ fixed in [OAI26, §4.2, p. 24].

Binary chunks. Pick the smallest e with $q_{\mathrm{rad}}^e \geq 2^u$ and pad both ranges numerically from $[2^u]$ to $[q_{\mathrm{rad}}^e]$. Because numerical and lexicographic orders coincide, this padding increases volume by less than q_{rad}^2 without per-address radix conversion. After running the radix-q_{rad} interchange, scan away padded entries where either coordinate is at least 2^u; since $e = O(u)$, the total cost is $O(Vu^\tau)$ [OAI26, proof of Lemma 12, p. 25]. Widths differing by one bit use one equal-width interchange plus one single-bit move [OAI26, §4.3, p. 25]. □

A.4 Packed selected-bit addition

This subsection shows how to XOR one selected bit offset across f consecutive K-bit address chunks simultaneously in sub-linear overhead.

Lemma A.9 executes $y_{\rho+iK} \leftarrow y_{\rho+iK} \oplus x_{\rho+iK}$ for $0 \leq i < f$, while restoring the scratch chunk z, with explicit step bound (49). For large q, when $\log_2 V = O(q)$, $f \leq q$, records are at least q^4 bits wide, and the chunk width K exceeds $3\log_2 q$ by a suitable constant, this bound simplifies to $O(V(L^\tau + 1))$ steps. We need this packed binary row addition in Lemma A.10 to perform $\mathbb{F}_2$-linear basis changes on circuit edges of the complex network.

Lemma A.9 (Parameterized packed selected-bit addition)**.** *Fix τ as in Lemma A.8. Let $R_{\mathrm{rec}} \geq 1$, $K \geq 6$, $f \geq 1$, $L = fK$ and $0 \leq \rho < K$, let $N_0, \ldots, N_3$ be positive integers, and let (χ_1, χ_2, χ_3) be a permutation of the chunk names (x, y, z). Consider R_{rec}-bit records on the complete rectangle $[N_0] \times [2^L]_{\chi_1} \times [N_1] \times [2^L]_{\chi_2} \times [N_2] \times [2^L]_{\chi_3} \times [N_3]$, and put $M = 2^{3L} \prod_{j=0}^{3} N_j$, $V = MR_{\mathrm{rec}}$ and $A_{\mathrm{rect}} = \lceil \log_2(2V) \rceil$. One fixed-tape procedure performs $y_{\rho+iK} \leftarrow y_{\rho+iK} \oplus x_{\rho+iK}$ for $0 \leq i < f$, preserves every other address bit and every payload, and restores the entire auxiliary chunk z for every initial address value. Its time is*

$$O(V(L^\tau + 1) + MA_{\mathrm{rect}}^3 + \delta_{\mathrm{bad}} MA_{\mathrm{rect}}(R_{\mathrm{rec}} + A_{\mathrm{rect}})), \qquad \delta_{\mathrm{bad}} = \min\{1, 80(f-1)2^{-K}\}. \tag{49}$$

In particular, if $A_{\mathrm{rect}}^3 \leq R_{\mathrm{rec}}$ and $80(f-1)2^{-K} A_{\mathrm{rect}}(1 + A_{\mathrm{rect}}/R_{\mathrm{rec}}) \leq 1$, the bound is $O(V(L^\tau + 1))$ with an absolute implied constant; both hold for all sufficiently large q, uniformly in shapes and payloads, whenever $A_{\mathrm{rect}} \leq C_A q$, $f \leq d \leq q$, $R_{\mathrm{rec}} \geq q^4$ and $K \geq 3\log_2 q + \log_2(160\, C_A)$.

Proof idea. An eight-step modular-shift program toggles the low bit of a segment without disturbing its partner whenever the upper $K-1$ bits stay away from the segment boundary, using only $O(1)$ whole-field shifts and interchanges of width L. The tiny $O(f2^{-K})$ fraction of addresses near a segment boundary is then extracted, radix-sorted to its exact destination ranks, and reinserted deterministically.

Proof. Scalar program and packing. For $c \in \{0, 1\}$ and integers u, w, run eight updates in chronological order, evaluating congruences modulo 2 on current values:

$$\begin{array}{llll} u \leftarrow u - c[w \equiv 0], & w \leftarrow w - c[u \equiv 0], & u \leftarrow u + c[w \equiv 0], & w \leftarrow w + c[u \equiv 0], \\ w \leftarrow w + c[u \equiv 1], & u \leftarrow u + c[w \equiv 1], & w \leftarrow w - c[u \equiv 1], & u \leftarrow u - c[w \equiv 1]. \end{array}$$

When $c = 0$ this program is the identity. When $c = 1$ its two halves induce the 3-cycles (00 01 10) and (01 10 11) on each 2×2 square $\{2u_0, 2u_0 + 1\} \times \{2w_0, 2w_0 + 1\}$, composing to $(u', w') = (u + c(1 - 2(u \bmod 2)), w)$ with total displacement at most 4 [OAI26, proof of Lemma 13, (20)–(21)].

Omitting the top selected bit j_{f-1}, write the K-bit segments at $j_i = \rho + iK$ for $i < f - 1$ inside y and z as $u_i = 2g_i + a_i$ and $w_i = 2h_i + b_i$. Across all $i < f - 1$ simultaneously, each program line is a single cyclic shift modulo 2^L, such as $y \leftarrow y + \epsilon \sum_{i=0}^{f-2} 2^{j_i}[x_{j_i} = 1][z_{j_i} = b] \bmod 2^L$, and is therefore a bijection of the rectangle. Let B be the exceptional set of addresses where some guard g_i or h_i fails $10 \leq g \leq 2^{K-1} - 11$.

Outside B, every segment starts in $[20, 2^K - 21]$ and remains inside $[0, 2^K - 1]$ with zero carries between adjacent segments, so the eight-shift composite S_0 matches the ideal XOR map T_0 across $i < f - 1$. Taking a union bound over $2(f-1)$ guards, each excluding 20 values out of 2^{K-1}, gives $|B| \le \delta_{\mathrm{bad}} M$; because T_0 preserves every guard, $T_0 S_0^{-1}$ permutes B and fixes B^c [OAI26, proof of Lemma 13, (22)–(24)].

Movement and deterministic repair. Moving each shift's target chunk to the rightmost position via Lemma A.8, running the controlled rotation of Lemma A.4, and restoring the chunk order uses at most 16 interchanges and 8 rotations in time $O(V(L^\tau + 1))$ [OAI26, proof of Lemma 13, pp. 30–31]. Next we scan the array with a record-rank counter, test membership in B in $O(A_{\mathrm{rect}}^3)$ steps per record, extract records $q_{\mathrm{addr}} \in B$ tagged with the destination rank of $T_0(S_0^{-1}(q_{\mathrm{addr}}))$, and mark their holes.

We then radix-sort the extracted bank in $O(|B| A_{\mathrm{rect}}(R_{\mathrm{rec}} + A_{\mathrm{rect}}))$ steps and reinsert those records in one forward pass. Finally, we apply the omitted two-bit XOR at j_{f-1} directly via Lemma A.4(1) in $O(V)$ steps [OAI26, proof of Lemma 13, (19), p. 31]. Neither B nor the repair depends on τ. □

A.5 Complex frames and the simultaneous butterfly layer

This subsection connects the $h = 100$ complex network to the simultaneous normalized butterfly layer $H_0^{\otimes D}$ with exposed parameters. Put $H_{\mathrm{u}} = \sqrt{2} H_0$, $S = \mathrm{diag}(1, i)$, $Z = \mathrm{diag}(1, -1)$, and $C = H_{\mathrm{u}} S H_{\mathrm{u}} = aI + bX$ with $a = \frac{1+i}{2}$, $b = \frac{1-i}{2}$, and $X = \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$. Direct 2×2 multiplication gives $H_0 = bSCS$, $C^{-1} = -iZCZ$, and $b^2 = -i/2$ [OAI26, §5.1, (13)–(14) and §5.4, p. 36].

Lemma A.10 unrolls one level of the complex network over the exact dyadic Gaussian ring $\mathbb{Z}[i, 1/2]$ on mf consecutive K-bit chunks with a common selected offset. It reduces $C^{\otimes mf}$ on all W_c role streams at once to a total of $s_c < W_c m$ child calls $C^{\otimes f}$, plus packed binary row additions, scalar gates, and linear-time diagonal phases. Crucially, inverse edge factors reduce to forward child calls via $C^{-1} = -iZCZ$, so the child count stays exactly s_c.

Lemma A.10 (Exact recursive complex contract)**.** *For the complex network of Definition A.1, on a group of $e = mf$ consecutive K-bit chunks with one common selected offset, $C^{\otimes e}$ is obtained on every role, exactly over $\mathbb{Z}[i, 1/2]$, by*

(1) exactly s_c forward child calls $C^{\otimes f}$, each on one slot of f consecutive chunks;

(2) a fixed number of packed binary row additions and their inverses;

(3) a fixed number of coefficientwise Gaussian dyadic scalar gates, address signs and fourth-root phases.

No extra child call is needed for an inverse directional factor or for an endpoint correction.

Proof. For any binary subspace label U, define $q_U(x) = \mathrm{wt}(P_U x) \bmod 4$ together with the frame operator $C_U = H_{\mathrm{u}}^{\otimes m} \mathrm{diag}_x(i^{q_U(x)}) H_{\mathrm{u}}^{\otimes m}$. Orthogonal binary vectors share an even number of ones, so Hamming weight is additive modulo 4 on orthogonal sums. On a nested edge with orthonormal residual basis $\mathcal{B}$, we have $q_V(x) - q_U(x) = \sum_{v \in \mathcal{B}} \mathrm{wt}(v)[v \cdot x] \bmod 4$, negated on a decreasing edge, where each $\mathrm{wt}(v)$ is odd. Because $i^{[v \cdot x]} = a + b(-1)^{v \cdot x}$ and conjugating $\mathrm{diag}_x((-1)^{v \cdot x})$ by $H_{\mathrm{u}}^{\otimes m}$ yields $(X_v f)(x) = f(x + v)$, each vector $v \in \mathcal{B}$ contributes $aI + bX_v$ or its inverse [OAI26, proof of Prop. 8, §3.5].

Extend $\mathcal{B}$ to an invertible matrix $M \in \mathrm{GL}(m, \mathbb{F}_2)$ whose first columns are $\mathcal{B}$. The induced address move P_M satisfies $P_M X_v P_M^{-1} = X_{Mv}$, realizing the edge across f columns via P_M^{-1}, one

coordinate kernel per $v \in \mathcal{B}$, and P_M. Here inverse factors use $(C^{-1})^{\otimes f} = (-i)^f Z^{\otimes f} C^{\otimes f} Z^{\otimes f}$ with zero extra child calls [OAI26, §5.1, (15)–(17)].

Each fixed matrix $M^{\pm 1}$ factors into at most $m(m-1)+3m$ elementary row additions, and Lemma A.9 executes each row addition across two f-chunk slots using a third slot as restored scratch since $m \geq 3$ [OAI26, §5.1, p. 27 and §5.2, pp. 31–32]. Because common address-linear frames commute with pointwise role gates, applying the source and sink signs $Z_{a,f}$, the scalar i^{27f}, and bank restoration produces $C^{\otimes e}$ on every role from s_c child calls [OAI26, proof of Prop. 8, §3.5 and §5.1, pp. 27–28]. Finally,

$$H_0^{\otimes D} = b^D \mathcal{S}_D C^{\otimes D} \mathcal{S}_D, \qquad (\mathcal{S}_D z)(x) = i^{\sum_{j=1}^{D} x_j} z(x), \qquad b^D = (-i/2)^{\lfloor D/2 \rfloor} b^{D \bmod 2}, \tag{50}$$

as in [OAI26, §5.1, (18)], where $\mathcal{S}_D$ runs in $O(V)$ time by Lemma A.5. □

Proposition A.11 is the central sub-linear butterfly engine of the paper. Given an array of M disk-valued polynomial records indexed by $D \leq d$ consecutive K-bit chunks, it applies H_0 simultaneously to one selected bit offset in every chunk in $O(Vd^{\lambda'})$ steps. Internally, it evaluates the entire layer without intermediate rounding inside $\Delta = \lceil C_0 d^{C_1} \rceil = o(q)$ guard bits before one final truncation Q_q. Exposing β, τ, σ, and C_1 here allows Section 7 to pair Proposition A.2 with a small leaf exponent $\beta = 1/64$ and guard exponent $C_1 = 14$.

Proposition A.11 (Simultaneous butterfly layer with general parameters)**.** *Let ν_{net} and C_0 be as in (47). Fix rationals $0 < \tau < 1$ and $0 \leq \sigma < 1$ with*

$$s_b/W_b < m^{\tau}, \qquad s_c/W_c < m^{\sigma},$$

and rational parameters $0 < \beta < 1$, $c > 0$, $0 < \varepsilon < 1$, λ, λ' with

$$\max\{\tau, \sigma\} < \lambda < 1, \qquad \tau(1 + c/\beta) < \lambda, \tag{51}$$

$$\max\{\lambda,\ \sigma + \beta(1-\sigma)\} < \lambda' < 1. \tag{52}$$

Fix an integer $C_1 \geq \nu_{\mathrm{net}} + 3$ with $\varepsilon C_1 < 1$, and positive constants $a_d \leq b_d$, $a_r \leq b_r$, C_M, C_{desc} and $C_w > 1$. Then there are a cutoff $q_{\min}$, one fixed finite-alphabet multitape procedure and a uniform time constant with the following property. Suppose that

$$q \geq q_{\min}, \qquad a_d q^{\varepsilon} \leq d \leq b_d q^{\varepsilon}, \qquad 2^{a_r q/d} \leq r \leq 2^{b_r q/d}, \qquad K = \lfloor d^c \rfloor, \tag{53}$$

and let $1 \leq D \leq d$, $A, S \geq 1$ and $0 \leq \rho < K$ be integers with

$$M = AS\, 2^{KD}, \qquad \lg(2M) \leq C_M q, \qquad q + 2 \leq w \leq C_w q.$$

The input is a disk-grid array in lexicographic order on $[A] \times [2^K]^D \times [S]$. Each address carries a record in $\mathcal{R}_r$, stored in coefficient order, and each component is 2^{-q} times a signed w-bit two's-complement integer, most significant bit first. The supplied header is $\Gamma(q)\Gamma(d)\Gamma(r)\Gamma(D)\Gamma(A)\Gamma(S)\Gamma(K)\Gamma(\rho+1)\Gamma(w)$, of length at most $C_{\mathrm{desc}} q$, with Γ as in (60). Let H apply H_0 to bit ρ, counted from the least significant end, of each of the D chunks. The procedure returns $Q_q(Hz)$, in the original record and component format, in $O(Vd^{\lambda'})$ steps, where $V = 2Mrw$. It evaluates the entire layer exactly before this final truncation; the output is a disk array and $\|Q_q(Hz) - Hz\|_{\infty} < \sqrt{2} \cdot 2^{-q}$. The recursion stops when $e < d^{\beta}$ (with $e = 1$ always a leaf). With $\Delta = \lceil C_0 d^{C_1} \rceil$, the procedure uses at most $q + \Delta$ fractional and $\Delta + 3$ integer and sign bits per real component internally; these widths are $O(q)$. The time includes padding, deterministic repair, role-stream scheduling, unpadding and workspace cleanup.

Before proving Proposition A.11, the next remark summarizes where each exposed structural parameter β, τ, σ, and C_1 enters the recursion.

Remark A.12 (Where the parameters enter). As printed, [OAI26, Prop. 14] fixes $\beta = \frac{1}{2}$, $\tau = \sigma = 1 - 2^{-50}$ and $C_1 = 20$. In the proof below, following [OAI26, §§5.1–5.4], the parameters enter only through the hypotheses of Proposition A.11. The parameter β enters the leaf test $e < d^\beta$ (evaluated for $\beta = \beta_1/\beta_2$ as the integer comparison $e^{\beta_2} < d^{\beta_1}$) and the bound $K \le e^{c/\beta}$. The exponents τ and σ enter the interchange costs of Lemmas A.8–A.9 and the time unrolling via $s_c/W_c < m^\sigma$, respectively. Finally, $C_1 \ge \nu_{\mathrm{net}} + 3$ enters the actual guard allocation $\Delta = \lceil C_0 d^{C_1} \rceil$ together with $\varepsilon C_1 < 1$, which ensures $\Delta = o(q)$ (already noted in [OAI26, §5.4, p. 35] as requiring only exponent $14 = \nu_{\mathrm{net}} + 3$).

A.6 Proof of Proposition A.11

This final subsection proves Proposition A.11 by assembling Lemma A.10 into a complete guarded tape procedure.

Proof idea. We peel off $d_0 = O(\log d)$ chunks to form a row field divisible by $W_c^{k_0}$, decompose the remaining chunks into power-of-m pieces, and recurse via Lemma A.10 down to leaves of size $e < d^\beta$. Because every operation is dyadic Gaussian and total arithmetic depth grows at most polynomially in d, $\Delta = \lceil C_0 d^{C_1} \rceil = o(q)$ guard bits hold every intermediate value without rounding until a single final truncation Q_q.

Individual kernels. Applying $C(u, v) = (\frac{u+v+i(u-v)}{2}, \frac{u+v-i(u-v)}{2})$ at one selected bit splits on that bit, scans aligned coefficient pairs, and merges in $O(V)$ time at any $O(q)$ guarded width. Hence e individual kernels cost $O(Ve)$ [OAI26, §5.3, (25)].

Rows, pieces, and scheduling. Set $d_0 = \lceil \log_2 W_c \rceil \lceil \log_m(2d) \rceil$ and $k_0 = \lceil \log_m d \rceil$. When $D \le d_0$, we run individual kernels directly. When $D > d_0$, we apply individual kernels to the first d_0 selected bits. Their complete consecutive chunks form a preceding row field of range $R_{\mathrm{row}} = 2^{d_0 K} \ge W_c^{k_0}$, which we pad in each prefix to the next multiple of $W_c^{k_0}$, at most doubling the volume. The remaining $D - d_0$ chunks split into at most $(m-1)(1 + \lfloor \log_m d \rfloor)$ consecutive power-of-m pieces.

At each internal node, cyclic row splitting produces W_c role streams of volume V/W_c whose row count is divisible by $W_c^{k_0 - j}$ at depth $j \le k_0$. That divisibility supports recursion down to one chunk regardless of β and preserves the three-slot rectangles required by Lemma A.9 [OAI26, §5.3, pp. 33–34]. Depth-first stream parking as in Proposition A.7 adds $O(V)$ overhead per node across the s_c children, while completed calls commute with the row-bit kernels and keep padded zero rows equal to zero for clean removal [OAI26, §5.3, pp. 33–34].

Arithmetic depth and guard. Let E_* denote the constant number of elementary coefficient operations, namely additions, subtractions, halvings, sign flips, and multiplications by i, executed outside child calls at one internal node by the fixed scalar circuit and its phase corrections. Dependency depth then obeys $A(e) \le s_c A(e/m) + E_*$ at internal nodes and $A(e) \le 8e$ at leaves [OAI26, §5.4, p. 35]. Across $j_{\max} \le \lceil \log_m d \rceil$ levels with $1 < s_c < m^5$, we have $s_c^{j_{\max}} \le s_c d^5$ and $A(e) \le C_{\mathrm{dep}} d^6$ for $C_{\mathrm{dep}} = s_c(8 + E_*/(s_c - 1))$.

We sum over at most md pieces for large d, at most d preprocessing kernels, the two outer diagonals $\mathcal{S}_D$, and the scalar b^D, whose shift by $t = \lfloor D/2 \rfloor$ counts as t halvings. The total denominator and magnitude growth then satisfies [OAI26, §5.4, p. 36]

$$G(d) \le mC_{\mathrm{dep}} d^7 + 8d + t + 9 \le (mC_{\mathrm{dep}} + 18)\, d^7 \le C_0 d^{C_1} \qquad (d \ge d_*), \tag{54}$$

where $d_* = \lceil ((mC_{\mathrm{dep}} + 18)/C_0)^{1/(C_1 - 7)} \rceil$ is finite because $C_1 \ge \nu_{\mathrm{net}} + 3 = 14 > 7$, and $d \ge d_*$ holds via $d \ge a_d q^\varepsilon$ once $q \ge q_{\min}$.

Starting from disk inputs on $2^{-q}\mathbb{Z}[i]$, values at arithmetic depth j lie in $2^{-(q+j)}\mathbb{Z}[i]$ with modulus at most 2^j. Allocating $\Delta = \lceil C_0 d^{C_1} \rceil$ guard bits, namely $q+\Delta$ fractional bits and $\Delta+3$ integer and sign bits, therefore represents every intermediate value exactly, and temporary bit-level interchanges cause zero coefficient rounding [OAI26, §5.4, p. 36]. Because $d \le b_d q^{\varepsilon}$ and $\varepsilon C_1 < 1$, we have $\Delta = o(q)$, so $\Delta \le q$ once $q \ge q_{\min}$ and guarded records have width $R_g = 2r(q+2\Delta+3) \in [2rq, 12rq]$.

Uniform packed-call cost and time recurrence. Every packed-addition child satisfies $A_* = \lceil \log_2(2M_{\text{child}} R_g) \rceil \le C_A q$ with $C_A = C_M + b_r + 7$, while $K \ge \frac{1}{2} a_d^c q^{\varepsilon c}$ and $r \ge 2^{(a_r/b_d)q^{1-\varepsilon}}$. Both extra normalized terms in (49) therefore vanish uniformly across all children, and Lemma A.5 absorbs all descriptor and mask scans [OAI26, §5.3, pp. 33–34 and proof of Prop. 14, §5.4, pp. 37–38]. Writing $a_c = s_c/W_c$ and $F(e)$ for step count divided by current logical volume gives

$$F(e) \le a_c F(e/m) + O((eK)^{\tau} + 1), \qquad F(e) = O(e) \quad (e < d^{\beta}). \tag{55}$$

For $e \ge d^{\beta}$ we have $K \le d^c \le e^{c/\beta}$, so (51) bounds node overhead by $O(e^{\lambda})$, and internal nodes sum to $O(e^{\lambda})$ since $a_c < m^{\sigma} < m^{\lambda}$. Leaves of size $u = e/m^j < d^{\beta}$ contribute $O(a_c^j u) \le O(e^{\sigma} u^{1-\sigma}) \le O(e^{\sigma} d^{\beta(1-\sigma)})$. Summing over all power-of-m pieces and preprocessing kernels yields

$$O\big(\log(2d)\,[d^{\lambda} + d^{\sigma+\beta(1-\sigma)} + 1]\big) = O(d^{\lambda'}) \tag{56}$$

by (52) [OAI26, §5.3, p. 34].

Order of operations and output. First re-encode the input into guarded format by sign-extending or trimming redundant sign bits to $\Delta+3$ integer and sign bits and appending Δ low fractional zero bits, reaching exact width $q+2\Delta+3$. In that format, apply $\mathcal{S}_D$ in the original layout, run the row-bit kernels, row padding, and power-of-m pieces for $C^{\otimes D}$, unpad, apply $\mathcal{S}_D$ again together with b^D as in (50), and finally apply Q_q once.

Chronological order matters: at $D = 1$, placing both $\mathcal{S}_1 = S$ factors after C would yield $bS^2C(1,0)^{\mathsf{T}} = (\frac{1}{2}, \frac{i}{2})^{\mathsf{T}} \ne H_0(1,0)^{\mathsf{T}}$. The final truncation drops low bits on the absolute numerator and restores the sign, realizing componentwise truncation toward zero Q_q in two's complement. Because every row of $H = H_0^{\otimes D}$ has absolute ℓ_1 sum 1, H preserves the unit disk, and Q_q preserves the disk with error below $\sqrt{2} \cdot 2^{-q}$ in width $w \ge q+2$ [OAI26, proof of Prop. 14, p. 38]. Choosing $q_{\min}$ above these finitely many thresholds completes the proof of Proposition A.11. □

The next remark notes that counting scalar circuit gates explicitly removes the threshold d_* from (54), although no theorem here needs that bound.

Remark A.13 (Explicit gate-count threshold). The proof above uses only finiteness of the scalar circuit. Counting at most $12W_c$ scalar gates of at most $2W_c^2$ operations each, $4s_c$ inverse-child operations and $4W_c + 4$ endpoint operations as in the proof text of [OAI26, §5.4, pp. 35–36] gives $E_* \le 24W_c^3 + 4s_c + 4W_c + 4 < E_{\text{net}}$, whence $A(e) \le 8B_{\text{net}}^2 d^{\nu_{\text{net}}+1}$ and $G(d) < 128 m B_{\text{net}}^2 d^{\nu_{\text{net}}+3} \le C_0 d^{C_1}$ as soon as the piece count is at most md, without any unspecified d_*. No theorem of this paper relies on that gate count.

The last remark covers the lifted precision used by the Walsh gadgets of Section 5.

Remark A.14 (Lifted precision). Proposition A.11 applies directly at the lifted precision $q_* = q + 8d$ used by the Walsh gadgets of Section 5 (with width $w_* = w + 8d$ and halved lower band constants $(a_d/2, b_d, a_r/2, b_r)$ once $q_* \le 2q$; Lemma 5.3).

B Deferred proofs for the fractional-shift bridge

This appendix supplies two deferred pieces from the fractional-shift bridge of Section 4. First, Section B.1 explains why our smoothing kernel uses a negative half-bin complex phase rather than

a positive phase or a purely real window. Second, Section B.2 proves Lemma 4.6(iii) by showing how a multitape Turing machine computes every spreading tap, solver ratio, and unit root of unity from truncated rational power series with certified rounding bounds. Section B.1 provides design motivation and is not needed for Theorem 1, whereas Section B.2 closes the table-generation step of the main proof. Throughout we follow the notation of Sections 3 and 4: $a \in \{2, \dots, d\}$ indexes the spread axes, $\nu \in \mathbb{Z}$ indexes the frequency aliases, and $\alpha_\pi = \sqrt{\pi}\,\alpha$.

B.1 Limits of positive-carrier and real-kernel designs

This subsection explains two design choices of our carrier. With the default Gaussian width $\alpha = 2\sqrt{\omega \ln 2/\pi}$ and retained alias pair $\{0,1\}$ of Lemma 4.4, a positive half-bin phase loses the uniform alias bound (Proposition B.1). Likewise, at the Nyquist sample $\xi = \frac{1}{2}$, no real-valued Schwartz kernel gives a uniformly conditioned two-alias cyclic solver (Proposition B.2). On any spread axis a of power-of-two length $t = t_a \geq 2$ (so t is even), Lemma 4.4 keeps the two aliases $\nu \in \{0,1\}$ in (18). That choice achieves an omitted-alias bound below $0.120 \cdot 2^{-\omega}$ uniformly over all $t \geq 2$, together with an inverse norm $\|B_a^{-1}\|_\infty \leq 4t_a/7$ independent of the cycle length $L = A_a$.

To see why the carrier sign matters, consider instead the *positive* half-bin carrier $g_a^+(s) = e^{+\pi i s/t_a}\,\mathrm{sinc}(s)\,e^{-\pi s^2/\alpha^2}$. Its Fourier transform is $\hat{g}_a^+(\xi) = G(\xi - \frac{1}{2t_a})$. For $u \in [t]$, the frequency samples sit at $y = (u - \frac{1}{2})/t$. In particular, the $u = 0$ sample lands at $y = -\frac{1}{2t} \notin (0,1)$, within distance $\frac{1}{2} - \frac{1}{2t}$ of the passband of the omitted alias $\nu = -1$.

Proposition B.1 shows that when $\nu \in \{0,1\}$ is retained, this positive-carrier shift loses three quarters of the target decay exponent at $t = 2$, leaving error $\Theta(\omega^{-1/2}2^{-\omega/4})$ instead of $O(2^{-\omega})$. That failure explains why Definition 4.1 modulates by $e^{-\pi i s/t_a}$, which shifts all samples into $(0,1)$ away from every omitted alias.

Proposition B.1 (No uniform alias bound for the positive carrier with retained pair $\{0,1\}$)**.** *With the positive carrier g_a^+, Gaussian width $\alpha = 2\sqrt{\omega \ln 2/\pi}$ ($\omega \geq 8$), and retained alias pair $\nu \in \{0,1\}$, the omitted alias weight on an axis of length $t = t_a \geq 2$ at frequency digit $u = 0$ is at least $\frac{1}{2}\,\mathrm{erfc}(\alpha_\pi(\frac{1}{2} - \frac{1}{2t}))$. For $t = 2$ this equals $\frac{1}{2}\,\mathrm{erfc}(\frac{1}{2}\sqrt{\omega \ln 2}) = \Theta(\omega^{-1/2}2^{-\omega/4})$. Consequently there is no constant C such that the omitted alias weight is at most $C \cdot 2^{-\omega}$ for all $\omega \geq 8$ and all powers of two $t \geq 2$. This already occurs in dimension $d = 2$.*

Proof. At $u = 0$ we have $y = -\frac{1}{2t}$. As in the proof of Lemma 4.4(ii), removing the intervals $[y - \nu - \frac{1}{2}, y - \nu + \frac{1}{2}]$ of the retained aliases $\nu \in \{0,1\}$ leaves $(-\infty, y - \frac{3}{2}] \cup [y + \frac{1}{2}, \infty)$, so the omitted alias weight is

$$\Psi^+(y) = \tfrac{1}{2}\,\mathrm{erfc}(\alpha_\pi(y + \tfrac{1}{2})) + \tfrac{1}{2}\,\mathrm{erfc}(\alpha_\pi(\tfrac{3}{2} - y)) \geq \tfrac{1}{2}\,\mathrm{erfc}\Big(\alpha_\pi\Big(\frac{1}{2} - \frac{1}{2t}\Big)\Big).$$

With $\alpha_\pi = 2\sqrt{\omega \ln 2}$ and $t = 2$, the right-hand side is $\frac{1}{2}\,\mathrm{erfc}(z)$ where $z = \frac{1}{2}\sqrt{\omega \ln 2} \geq \sqrt{2 \ln 2} > 1$. Integrating $\frac{2}{\sqrt{\pi}}e^{-s^2}$ over $[z, z + 1/z]$ and using $(z + 1/z)^2 = z^2 + 2 + z^{-2} \leq z^2 + 3$ yields

$$\mathrm{erfc}(z) \geq \frac{2}{\sqrt{\pi}\,z}\,e^{-(z+1/z)^2} \geq \frac{2e^{-3}}{\sqrt{\pi}\,z}\,e^{-z^2},$$

so substituting $z = \frac{1}{2}\sqrt{\omega \ln 2}$ and $e^{-z^2} = 2^{-\omega/4}$ gives

$$\Psi^+\Big(-\frac{1}{4}\Big) \geq \frac{1}{2}\,\mathrm{erfc}(z) \geq \frac{2e^{-3}}{\sqrt{\pi\omega \ln 2}}\,2^{-\omega/4},$$

whose ratio to $2^{-\omega}$ grows without bound as $\omega \to \infty$. More generally, for any fixed $t \geq 2$ the omitted weight at $u = 0$ is $\Theta(\omega^{-1/2}2^{-\omega(1-1/t)^2})$, which exceeds $C \cdot 2^{-\omega}$ once ω is large relative to t. □

Three modifications restore uniform $O(2^{-\omega})$ decay. One can reverse the sign of the carrier as in Definition 4.1, switch the retained pair at $u = 0$ to $\nu \in \{-1, 0\}$, or widen α by a factor $t/(t-1)$. Numerically, at $\omega = 64$ the largest omitted weight over all frequency digits u drops from $1.5 \times 10^7 \cdot 2^{-64}$ under the positive carrier to $5 \times 10^{-13} \cdot 2^{-64}$ under the negative carrier at $t = 4$. At $t = 8$ it drops from $1564 \cdot 2^{-64}$ to $3 \times 10^{-7} \cdot 2^{-64}$, and at $t = 64$ from $0.17 \cdot 2^{-64}$ to $0.01 \cdot 2^{-64}$.

Notice that Proposition B.1 concerns uniformity over all pairs (ω, t). When $t \geq \omega$, as along the root family of Lemma 7.1 where $t_a \geq r/2 \gg \omega$, the bound $(1 - 1/t)^2 \geq 1 - 2/t$ still keeps the positive carrier's omitted weight within $O(2^{-\omega})$. By contrast, the negative carrier holds uniformly for every $t \geq 2$ without coupling t to ω.

Proposition B.2 explains why a purely real smoothing kernel cannot replace complex modulation in dimension three and higher. Because the Fourier transform of any real kernel is conjugate-symmetric, its two retained aliases at the Nyquist frequency $\xi = \frac{1}{2}$ always have equal magnitude $|c_0| = |c_1|$. Consequently, the two-term cyclic operator is either singular or amplifies rounding errors by at least $\Omega(L)$, which grows as $A_a = \prod_{i<a} t_i$ across spread axes instead of staying bounded by $O(t_a)$.

Proposition B.2 (No real kernel gives a uniformly conditioned two-alias inverse)**.** *Let g be any real-valued Schwartz kernel and retain the two aliases $\nu \in \{0, 1\}$ at the Nyquist sample $\xi = \frac{1}{2}$, with coefficients $c_0 = \hat{g}(\frac{1}{2})$ and $c_1 = \hat{g}(-\frac{1}{2})$. Then $|c_0| = |c_1| =: c$, and whenever the cyclic bidiagonal operator $B\colon Z_j \mapsto c_0 Z_j + c_1 Z_{(j-1) \bmod L}$ on a cycle of length $L \geq 1$ is invertible, every row of B^{-1} has absolute sum*

$$\|B^{-1}\|_\infty = \frac{L}{c\,|1 - (-c_1/c_0)^L|} \geq \frac{L}{2c}.$$

In particular, if g is even, then $c_0 = c_1$ and B is singular for every even cycle length L.

Proof. Because g is real-valued, $\hat{g}(-\xi) = \overline{\hat{g}(\xi)}$, so $|c_0| = |c_1| = c$. If $c = 0$, then $B = 0$ is never invertible. Assume $c > 0$ and write $B = c_0(I + \zeta T)$ where $\zeta = c_1/c_0$ satisfies $|\zeta| = 1$ and $(TZ)_j = Z_{(j-1) \bmod L}$. If $(-\zeta)^L = 1$, the nonzero vector $v_j = (-\zeta)^j$ $(0 \leq j < L)$ is well-defined on $\mathbb{Z}_L$ and satisfies $v_j + \zeta v_{(j-1) \bmod L} = 0$ for all $j \in [L]$, so B is singular. If $(-\zeta)^L \neq 1$, then

$$B^{-1} = c_0^{-1}(1 - (-\zeta)^L)^{-1} \sum_{i=0}^{L-1} (-\zeta T)^i,$$

each row of which has L nonzero entries in distinct columns, every one of modulus $\big(c\,|1-(-\zeta)^L|\big)^{-1} \geq (2c)^{-1}$. When g is even, $\hat{g}$ is even, so $\zeta = 1$ and $(-\zeta)^L = 1$ for every even L. □

On the bridge's deconvolution cycles of even length $L = A_a = \prod_{i<a} t_i$, any real kernel therefore either fails to be invertible or amplifies errors by at least $A_a/(2c)$ rather than $O(t_a)$.

B.2 Certified evaluation of filter, solver, and phase tables

This subsection proves Lemma 4.6(iii) by constructing every transcendental table entry of the bridge on a deterministic multitape Turing machine. No exact-rounding or real-arithmetic oracle is assumed anywhere.

Lemma B.3 states that all unfolded kernel samples, folded spreading taps, cyclic deconvolution ratios, and unit roots of unity can be evaluated to certified precision 2^{-b} or 2^{-q} in polynomial bit time. We need this result in Section 7.3 so that precomputing all bridge and FFT lookup tables takes negligible time compared with the main transform. In particular, folded taps land in $\widetilde{\mathbb{C}}_q$ within $0.95 \cdot 2^{-q}$ of the infinite folded sum, while unit phases land inside the closed discrete unit disk $\mathbb{D}_q$ within $2 \cdot 2^{-q}$.

Lemma B.3 (Certified evaluation of bridge and phase tables)**.** *Let $P \geq 20$, $q = P + 2$, $v = 2^{-q}$, and $8 \leq \omega \leq P$. Let $\alpha^2 = 4\lambda_G \omega \ln 2/\pi$ with a fixed rational $\lambda_G \in [1, 4]$, and let $R = t_1 \cdots t_d = 2^k$ with each $t_i \geq 2$ a power of two and $A_a = \prod_{i<a} t_i$.*

(a) For every integer precision $b \geq \max(8, P)$, each unfolded kernel sample $g_a(e + \delta)$ ($2 \leq a \leq d$, $|e| \leq P$, $\delta \in A_a^{-1}\mathbb{Z} \cap [0, 1)$), each alias weight $c_0 = G(y)$ and $c_1 = G(y - 1)$ at $y = (u + \frac{1}{2})/t_a$ ($u \in [t_a]$), the solver parameters $c_{\max}^{-1}$ and $\rho = \min(c_0, c_1)/c_{\max}$, and every unit phase $e^{\pi i \vartheta}$ ($\vartheta \in \mathbb{Q}$ with $O(k + \lg P)$-bit numerator and denominator) can be computed deterministically with certified error at most 2^{-b} in $O((b+k)^3)$ bit operations on a fixed number of tapes, without any transcendental oracle.

(b) Consequently, at grid precision $q = P + 2$:

- *every truncated folded tap $\sum_{s \in e + t_a\mathbb{Z}, |s| \leq P} g_a(s + \delta)$ ($e \in [t_a]$) is computed as an entry $\hat{H}_\delta^{(a)}(e) \in \widetilde{\mathbb{C}}_q$ with certified error below $0.71\, v < v$ relative to the truncated sum and below $0.95\, v < v$ relative to the full folded tap $H_\delta^{(a)}(e)$, in $O(P(P + k)^3)$ bit operations per tap;*
- *the solver parameters $\hat{\mu}, \hat{\rho} \in 2^{-q}\mathbb{Z}$ satisfy $\hat{\rho} \geq 0$, $|\hat{\mu} - c_{\max}^{-1}| < v$, and $|\hat{\rho} - \rho| < v$;*
- *every unit phase $e = e^{\pi i \vartheta}$ has a disk-grid approximation $\hat{e} \in \mathbb{D}_q$ with $|\hat{e}| \leq 1$ and $|\hat{e} - e| < 2v$ (with fourth roots of unity exact), and a d-factor tensor chirp formed by sequentially multiplying d one-axis disk phases with Q_q truncation after each product lies in $\mathbb{D}_q$ with error at most $4dv$.*

Proof idea. We assemble every filter tap, alias weight, and phase factor from the constants π and $\ln 2$ together with four elementary functions at exact rational arguments: trigonometric functions, sinc, e^{-x}, and erf. We work on a fine dyadic grid $2^{-S}\mathbb{Z}$ with $S \approx 2b + 64$ fractional bits, giving roughly $b + 64$ guard bits beyond the target precision b. Range reduction places every power-series argument inside a small interval where term-recurrence rounding errors either contract geometrically or stay bounded by $e^{w_z} < 2^{b+7}$. One final guarded rounding from $2^{-S}\mathbb{Z}$ onto the q-grid then certifies every entry within its stated error budget in $O((b + k)^3)$ bit operations per sample.

Proof. 1. Primitives and exact rational reductions. Recall that $\alpha^2 = 4\lambda_G \omega \ln 2/\pi$ with $\lambda_G \in [1, 4]$, where Theorem 4.7 takes $\lambda_G = 1$. Then $\alpha_\pi^2 = \pi\alpha^2 = 4\lambda_G \omega \ln 2$, and the Gaussian spatial exponent becomes $x = \pi s^2/\alpha^2 = \pi^2 s^2/(4\lambda_G \omega \ln 2)$, avoiding any square root of π. Every table entry is assembled from five scalar primitives at exact rational arguments:

(i) the constants π, $\ln 2$, $\sqrt{\pi}$, and $\alpha_\pi = 2\sqrt{\lambda_G \omega \ln 2}$;
(ii) $\cos(\pi\vartheta)$ and $\sin(\pi\vartheta)$ for $\vartheta \in \mathbb{Q}$, used for the carrier $e^{-\pi i(e+\delta)/t_a}$, for $\sin(\pi s)$, and for unit twiddle, twist, and chirp phases;
(iii) $\operatorname{sinc}(s) = \sum_{n \geq 0} (-1)^n (\pi s)^{2n}/(2n + 1)!$ for $|s| \leq \frac{1}{2}$;
(iv) e^{-x} for $x \geq 0$;
(v) $\operatorname{erf}(z)$ at $z \in \{\alpha_\pi(y + \frac{1}{2}), \alpha_\pi(y - \frac{1}{2}), \alpha_\pi(y - \frac{3}{2})\}$ for $y = (u + \frac{1}{2})/t_a$, giving $c_0 = \frac{1}{2}[\operatorname{erf}(\alpha_\pi(y + \frac{1}{2})) - \operatorname{erf}(\alpha_\pi(y - \frac{1}{2}))]$ and $c_1 = \frac{1}{2}[\operatorname{erf}(\alpha_\pi(y - \frac{1}{2})) - \operatorname{erf}(\alpha_\pi(y - \frac{3}{2}))]$.

The unfolded offsets $s = e + \delta$ with $|e| \leq P$ and $\delta \in A_a^{-1}\mathbb{Z} \cap [0, 1)$ for $A_a \leq R = 2^k$, the angles ϑ, and the half-bin coordinates y are all rationals with $O(k + \lg P)$-bit numerators and denominators. We manipulate these arguments in exact rational arithmetic. Each angle ϑ is reduced modulo 2 and folded by exact octant symmetries of sin and cos into $[-\frac{1}{4}, \frac{1}{4}]$, using $\sin(\pi s) = (-1)^e \sin(\pi\delta)$ for $s = e + \delta$. Thus only reduced angles $|\vartheta| \leq \frac{1}{4}$ multiply our approximation to π in (ii). Because $y \in (0, 1)$, each shifted offset satisfies $|y + \frac{1}{2}|, |y - \frac{1}{2}|, |y - \frac{3}{2}| \leq \frac{3}{2}$, so every erf argument obeys $z^2 \leq \frac{9}{4}\alpha_\pi^2 = 9\lambda_G \omega \ln 2 \leq 36\omega \ln 2 < 25P$. Similarly, $|s| < P + 1$ and $\omega \geq 8$ give $x \leq \pi^2(P + 1)^2/(32 \ln 2) < 0.45(P + 1)^2$.

2. Working scale and fundamental constants. Given $b \geq \max(8, P) \geq 20$, set $S = 2b+8\lceil \lg b \rceil + 64$, $N = 4S$, and $u_S = 2^{-S}$. All intermediate values live on the dyadic grid $2^{-S}\mathbb{Z}$. Additions and subtractions on $2^{-S}\mathbb{Z}$ are exact, while each multiplication or division rounds once to $2^{-S}\mathbb{Z}$ with error at most u_S.

- We sum Machin's identity $\pi = 16\arctan\frac{1}{5} - 4\arctan\frac{1}{239}$ and the series $\ln 2 = 2\sum_{j\geq 0}(2j+1)^{-1}3^{-2j-1}$ to N terms for $0 \leq j < N$, computing each signed term at scale 2^S by one rounded integer division. Their truncation tails lie below $20 \cdot 5^{-2N-1} < u_S$ and $3^{-2N} < u_S$, giving $|\tilde{\pi} - \pi| \leq 21Nu_S$ and $|\widetilde{\ln 2} - \ln 2| \leq 3Nu_S$.
- Taking integer square roots of the 2^{2S}-scaled approximations to π and $4\lambda_G\omega\ln 2$ yields $|\widetilde{\sqrt{\pi}} - \sqrt{\pi}| \leq \frac{21Nu_S}{2\sqrt{3}} + u_S \leq 7Nu_S$. Since $\sqrt{4/\ln 2}\cdot 3 < 7.21$, it also yields $|\tilde{\alpha}_\pi - \alpha_\pi| \leq \sqrt{\frac{\lambda_G\omega}{\ln 2}}\,3Nu_S + u_S \leq 8N\sqrt{P}\,u_S$. Consequently, $|\tilde{z} - z| \leq \frac{3}{2}|\tilde{\alpha}_\pi - \alpha_\pi| + u_S \leq 13N\sqrt{P}\,u_S$ holds for every erf argument z in (v).

3. Trigonometric, sinc, and Gaussian series. Primitives (ii)–(iv) reduce to alternating power series in a nonnegative argument of size at most $\pi/4$ for cos and sin, at most $\pi/2$ for $\mathrm{sinc}(s)$ on $|s| \leq \frac{1}{2}$, or at most $\ln 2$ for e^{-r} after range reduction. Up to sign, consecutive terms obey $T_j = T_{j-1}w_0/((2j)(2j \pm 1))$ with $w_0 \leq (\pi/2)^2$ and step multiplier at most $\pi^2/24 < 0.42$, or $T_n = T_{n-1}r/n$ with $r \leq \widetilde{\ln 2} < 0.70$. Every step multiplier therefore lies strictly below 0.71.

Because each step contracts by at least 0.71, rounding errors in the term recurrence damp out geometrically. Every computed term stays within $4u_S$ of its exact value at the approximate argument, and the N-term sum accumulates rounding error at most $4Nu_S$. The series tail beyond $N = 4S$ is at most $2(1.6\,\mathrm{e}/N)^N \ll u_S$, and all four functions are 1.01-Lipschitz.

For e^{-x} in (iv), if $\tilde{x} \geq (b+5)\widetilde{\ln 2}$, then $x \geq (b+4)\ln 2$ and $e^{-x} \leq 2^{-b-4}$, so returning 0 incurs error at most 2^{-b-4}. Otherwise $x < (b+6)\ln 2$. We set $q_0 = \lfloor \tilde{x}/\widetilde{\ln 2} \rfloor \leq b+5$ and $\tilde{r} = \tilde{x} - q_0\widetilde{\ln 2} \in [0, \widetilde{\ln 2})$, evaluate the N-term Taylor series of $e^{-\tilde{r}}$, and shift right by q_0 bits, incurring range-reduction error at most $23N(b+6)u_S$. For $\mathrm{sinc}(s)$ when $|s| > \frac{1}{2}$, we divide $(-1)^e\sin(\pi\delta)$ from (ii) by $\tilde{\pi}s$, where $|\tilde{\pi}s| > 1.5$. Summing argument, recurrence, and tail bounds certifies every primitive in (ii)–(iv) and every unfolded sample $g_a(s)$ within $50N(b+P+8)u_S \leq 2^{-b-4}$, including when the e^{-x} cutoff triggers.

4. Error-function series. Because erf is odd, take $z \geq 0$. If $\tilde{z}^2 \geq (b+5)\widetilde{\ln 2}$, then $z^2 \geq (b+4)\ln 2$ and $z \geq 1/\sqrt{\pi}$, so $1 - \mathrm{erf}(z) = \mathrm{erfc}(z) \leq e^{-z^2}/(\sqrt{\pi}z) \leq 2^{-b-4}$ and returning 1 has error at most 2^{-b-4}. Otherwise $w_z = \tilde{z}^2$, formed exactly on $2S$ fractional bits, satisfies $w_z < (b+6)\ln 2 + u_S$. Then $e^{w_z} < 2^{b+7}$, and $N = 4S > 8(b+6)\ln 2 + 8$ gives $\mathrm{e}\,w_z/N \leq \mathrm{e}/8$.

We evaluate $\frac{2}{\widetilde{\sqrt{\pi}}}\sum_{n=0}^{N-1}(-1)^n T_n/(2n+1)$ with $T_0 = \tilde{z}$ and $T_n = T_{n-1}w_z/n$, forming $T_{n-1}w_z$ on $3S$ fractional bits before rounding to u_S. Any rounding error introduced at step n_0 is scaled at step $n > n_0$ by $\prod_{n_0<j\leq n}(w_z/j) \leq w_z^{n-n_0}/(n-n_0)! \leq e^{w_z}$, keeping the total recurrence and division error below $2N^2e^{w_z}u_S$. The series tail beyond N is below $2\tilde{z}(\mathrm{e}/8)^N$, while $|\,\mathrm{erf}(\tilde{z}) - \mathrm{erf}(z)| \leq \frac{2}{\sqrt{\pi}}|\tilde{z} - z|$. Since $S = 2b + 8\lceil \lg b \rceil + 64$ and $b \geq 20$, summing these terms bounds the error in $\mathrm{erf}(z)$, and hence in c_0 and c_1, by $2N^2 2^{b+8}u_S \leq 2^{-b-4}$.

5. Assembly of q-grid entries. By Lemma 4.4(iv), $c_0 > c_1$ holds if and only if $u < t_a/2$, so $c_{\max}$ is selected without numerical error. Lemma 4.4(iii) also gives $c_{\max} > 0.49999999$. One guarded division each therefore yields $c_{\max}^{-1}$ and $\rho = \min(c_0, c_1)/c_{\max}$ within $4.01 \cdot 2^{-b-4} + u_S < 2^{-b-1} < 2^{-b}$, proving part (a). Taking $b = q+1$ and applying $\mathcal{Q}_q$, followed by $\max(0, \cdot)$ on $\hat{\rho}$ which cannot increase error since $\rho \geq 0$, gives $|\hat{\mu} - c_{\max}^{-1}| < 2^{-q-2} + 2^{-q-1} < v$ and $|\hat{\rho} - \rho| < v$.

Each truncated folded tap $\sum_{s\in e+t_a\mathbb{Z},\,|s|\leq P} g_a(s+\delta)$ sums at most $2P+1$ terms. We evaluate each unfolded sample at precision $b = q+g$ with $g = \lceil \lg(2P+2) \rceil + 12$, sum the residue class exactly at scale 2^{-S}, and apply one complex nearest rounding $\mathcal{Q}_q$. That bounds the error relative to the

truncated sum by $(2P+1)2^{-q-g-4} + \frac{\sqrt{2}}{2}v < 0.71\,v < v$. Adding the spatial tail $0.06 \cdot 2^{-P} = 0.24\,v$ from Lemma 4.6(i) certifies the stored tap within $0.95\,v < v$ of the full folded tap $H_\delta^{(a)}(e)$.

For a unit phase $e = e^{\pi i\vartheta}$, fourth roots of unity with $2\vartheta \in \mathbb{Z}$ are exact. Otherwise evaluating (ii) at $b = q+5$ produces e_0 with $|e_0 - e| < 2^{-q-9} < v/16$. Setting $\hat{e} = Q_q((1-v/8)e_0)$ satisfies $|(1-v/8)e_0| \le (1-v/8)(1+v/16) < 1$, so $\hat{e} \in \mathbb{D}_q$ and $|\hat{e} - e| < (\sqrt{2} + \frac{3}{16} + v)v < 2v$ by (7). Multiplying any disk value $|z| \le 1$ by $\hat{e} \in \mathbb{D}_q$ and applying Q_q stays in the closed unit disk and adds error below $(2+\sqrt{2})v < 4v$. Iterating across d single-axis factors yields a tensor chirp in $\mathbb{D}_q$ within $2v + 4(d-1)v \le 4dv$ of the exact chirp, proving part (b).

6. Bit complexity. Each series uses $N = O(b)$ operations on $O(b)$-bit integers, since erf numerators remain below $2^{b+8}\sqrt{b} \cdot 2^S$. Even under schoolbook arithmetic this takes $O(b^3)$ bit operations, while exact rational reductions on $O(k + \lg P)$-bit fractions take $O((k+\lg P)^2)$ bit operations. Each primitive or sample in (a) therefore costs $O((b+k)^3)$ bit operations on a fixed number of tapes. In part (b), each folded tap uses $b = q + O(\lg P) = P + O(\lg P)$ and therefore costs $O(P(P+k)^3)$ bit operations, using at most $3S + O(k)$ fractional bits for exact products before rounding. □

C Comparison with a Bluestein middle transform and additional parameter rows

This appendix analyzes an alternative middle transform for our fractional-shift bridge and explains why our main text does not use it. In Sections 5–6, we evaluate the middle power-of-two tensor DFT directly over $\mathbb{C}$ using Walsh-layer positional permutations, reaching $\kappa = 2^{-46}$. Instead, one can keep the synthetic-ring architecture of [OAI26, §§6–7], reducing the tensor DFT to a normalized cyclic convolution over $\mathcal{R}_r = \mathbb{C}[y]/(y^r+1)$ via Bluestein's chirp identity [Blu70] and a last-axis twist. Recording that Bluestein route on the same bridge isolates why chunk-exchange layouts cap κ quadratically at $(1-\tau)^2/(C_1\tau)$, and establishes all secondary comparison rows checked by `tools/h5_rows_check.py`. No result of this appendix is used in the proof of Theorem 1, so a first-time reader may skip it entirely.

C.1 Synthetic transforms and normalized ring convolution

This subsection sets up polynomial-ring transforms over $\mathcal{R}_r = \mathbb{C}[y]/(y^r+1)$ when record-address bits are grouped by chunk interchanges rather than by Walsh permutations. For a power of two $r \ge 2$, let $\mathcal{R}_r = \mathbb{C}[y]/(y^r+1)$ carry the coefficient norm $\|\sum_{j<r} z_j y^j\|_\infty = \max_j |z_j|$. For any divisor $t \mid 2r$, set $\omega_t = y^{2r/t}$, define $(F_t^{\mp} z)_j = t^{-1}\sum_{u<t} \omega_t^{\mp ju} z_u$ on $\mathcal{R}_r^t$, and let B_t map each index j to its $(\lg t)$-bit reversal. For $\mathbf{t} = (t_1, \dots, t_{d-1})$ with $t_i \in \{r/2, r\}$ and $M = \prod_{i<d} t_i$, put $F_{\mathbf{t}}^{\pm} = \bigotimes_{i<d} F_{t_i}^{\pm}$ and $B = \bigotimes_{i<d} B_{t_i}$.

Both the direct Walsh route and the Bluestein route use the named-slot layout map L of Lemma 6.3, matching the layout in [OAI26, proof of Lemma 16, pp. 41–42]. The crucial difference lies in how L is executed on tape. Here Lemma C.1 executes L by width-K chunk exchanges from Lemma A.8 rather than by exact Walsh-layer positional permutations.

Lemma C.1 states the forward and inverse synthetic polynomial transforms across the $d-1$ record axes and bounds their tape cost. We need it to isolate the $O(VqK^{\tau-1})$ chunk-exchange layout term that becomes the binding bottleneck of the Bluestein route. Notice that both directions keep all intermediate records inside the unit disk with total rounding error below $\sqrt{2}\,\ell\,2^{-q}$.

Lemma C.1 (Synthetic transform interface)**.** *Fix the rational parameters of Proposition A.11 and its band constants $a_d \le b_d$, $a_r \le b_r$. Fix constants $C_\ell > 0$ and $C_w > 1$, and instantiate Proposition A.11*

with $C_M = C_\ell + 1$ and $C_{\mathrm{desc}} = 32(C_\ell + C_w + 1)$, as in [OAI26, Lemma 16, §6.2]. There are a cutoff q_1, fixed-tape procedures and a time constant with the following property. Let $q \geq q_1$ and d, r satisfy the bands (53), with $d \geq 2$, $r = 2^\ell$, $2 \leq \ell < q$, $d\ell \leq C_\ell q$ and $1 \leq K = \lfloor d^c \rfloor \leq \ell - 1$; let $t_i \in \{2^{\ell-1}, 2^\ell\}$ for $1 \leq i < d$, $M = \prod_{i<d} t_i$, $R = rM$ and $q + 2 \leq w \leq C_w q$. Given the header $\Gamma(q)\Gamma(d)\Gamma(r)\Gamma(K)\Gamma(w)\Gamma(t_1)\cdots\Gamma(t_{d-1})$ and M contiguous disk-grid records in $\mathcal{R}_r$ at grid precision q and width w, the forward procedure has exact counterpart $LBF_{\mathbf{t}}^-$ and the opposite procedure, which consumes that order, has exact counterpart $F_{\mathbf{t}}^+ B^{-1} L^{-1}$. Both return disk arrays on the q-grid in the same format, have error less than $\sqrt{2}\,\ell\,2^{-q}$, and cost

$$O(V[dK + qK^{\tau-1} + \ell d^{\lambda'}]), \qquad V = 2Rw.$$

Proof. Step 1: Layout rearrangement. Set $b_0 = (\ell - 1) \bmod K$ and $J_{\mathrm{blk}} = (\ell - 1 - b_0)/K$ as in Lemma 6.3. Moving the at most $(d-1)K$ prefix slots $\mathcal{P}$ to the front via single-bit moves from Lemma A.4(2) costs $O(VdK)$. Rearranging the remaining $N = (d-1)J_{\mathrm{blk}} \leq C_\ell q/K$ consecutive K-bit chunks into chunk-level-major order $\mathcal{C}$ factors into at most $N - 1$ transpositions. Each transposition is an equal-width chunk exchange costing $O(VK^\tau)$ by Lemma A.8. Hence L, and in reverse L^{-1}, runs in time $O(V[dK + qK^{\tau-1}])$.

Step 2: Parallel butterfly rounds. Once in layout L, we run the normalized decimation-in-frequency recursion of [OAI26, §6.1, (27)] across all $d-1$ record axes in parallel at named slots $v = \ell - 1, \ldots, 0$. At slot v, every axis with $\lg t_i > v$ applies H_0 on slot v and then multiplies branch $b_i = 1$ by $\omega_{2^{v+1}}^{-k_i} = y^{-(2r/2^{v+1})k_i}$. Multiplying those factors across axes produces a single recordwise monomial y^E. By Lemma A.5, that monomial is a signed cyclic shift of the r coefficients costing $O(rq)$ per record.

The at most $1 + b_0 \leq K$ prefix rounds evaluate their at most $d-1$ factors of H_0 exactly using $q + d$ fractional bits before a single truncation Q_q, in total cost $O(VdK)$. Each of the at most ℓ chunk rounds for $b_0 \leq v \leq \ell - 2$ invokes Proposition A.11 on $d-1$ consecutive K-bit chunks, costing $O(V\ell d^{\lambda'})$ in total.

Step 3: Error bound and inverse transform. Monomials are isometries that commute with Q_q, and normalized butterfly groups are contractions, so the per-round truncation errors below $\sqrt{2} \cdot 2^{-q}$ sum to less than $\sqrt{2}\,\ell\,2^{-q}$. Reversing the schedule with H_0 in place of $2H_0$ and inverting each monomial before its butterfly realizes $M^{-1}(LBF_{\mathbf{t}}^-)^{-1} = F_{\mathbf{t}}^+ B^{-1} L^{-1}$, since $F_{\mathbf{t}}^+ F_{\mathbf{t}}^- = M^{-1} I$ [OAI26, Lemma 15, §6.1]. □

After transforming along the first $d-1$ axes, corresponding polynomial records in $\mathcal{R}_r$ must be multiplied pointwise. Lemma C.2 packs each polynomial product into four $\Theta(rq)$-bit integer multiplications via Kronecker substitution. Unlike our direct complex FFT, which multiplies only $O(q)$-bit scalars, this step requires an inner integer multiplier on $\Theta(rq)$-bit operands.

Lemma C.2 (Packed polynomial products)**.** *Let $r = 2^\ell < 2^q$ with $q \geq 2$, and let $f, g \in \mathcal{R}_r$ be disk-grid polynomials at precision q. For any independent integer multiplier of cost $\mathsf{M}(N)$ on operands of at most N bits, one can return $Q_q(fg/r)$ in time $O(\mathsf{M}(4rq + 2) + rq)$, using four signed integer products of at most $4rq + 2$ bits. The output is a disk-grid polynomial with error less than $\sqrt{2} \cdot 2^{-q}$ from fg/r.*

Proof. Pack each of the four signed real and imaginary numerator lists, whose entries lie in $[-2^q, 2^q]$, into base $B_0 = 2^{4q}$. Every ordinary convolution coefficient has modulus at most $r2^{2q} < 2^{3q} < B_0/2$, so carries between blocks cannot overlap. Blockwise balanced-base decoding in $O(rq)$ bit operations therefore recovers the exact ordinary polynomial product. Subtracting the upper half at lag r reduces modulo $y^r + 1$, and dividing by r followed by Q_q returns a disk-grid output within $\sqrt{2} \cdot 2^{-q}$ of fg/r, since $\|fg/r\|_\infty \leq 1$. □

Proposition C.3 combines the forward and inverse synthetic transforms with pointwise record products to compute a normalized cyclic convolution over $\mathcal{R}_r$. Notice two features that distinguish this ring convolution from our direct tensor FFT: its runtime carries the chunk-exchange term $O(VqK^{\tau-1})$, and its final multiplication by M widens components by $\lg M$ bits while amplifying rounding error by M.

Proposition C.3 (Normalized ring convolution)**.** *Under the hypotheses of Lemma C.1, the cyclic convolution in the first $d-1$ axes, normalized as $\mathcal{M}_r(f,g) = (f * g)/(rM)$, of two disk-grid arrays has an approximation with error less than $\sqrt{2}\,M(3\ell+1)2^{-q}$, computed in time*

$$O\big(V[dK + qK^{\tau-1} + \ell d^{\lambda'}] + M\,\mathsf{M}(O(rq))\big),$$

returned in the original array and coefficient order at component width $w + \lg M$. If one exact input has norm at most $\varrho < 1$ and $\sqrt{2}\,M(3\ell+1)2^{-q} \le 1-\varrho$, the output is a disk array.

Proof. Recall that $F_{\mathbf{t}}^{+}F_{\mathbf{t}}^{-} = M^{-1}I$ and $F_{\mathbf{t}}^{-}(f*g) = M(F_{\mathbf{t}}^{-}f)(F_{\mathbf{t}}^{-}g)$ by [OAI26, Lemma 15, §6.1]. We apply the forward procedure of Lemma C.1 to f and g, multiply aligned records in their common order LB using Lemma C.2, and apply the opposite transform. Appending $\lg M$ low zero bits then multiplies the result exactly by M, canceling the permutation LB to match $(f*g)/(rM)$. Because $(u,v) \mapsto uv/r$ is a bilinear contraction on disk arrays, the two forward transforms, the record product, and the opposite transform accumulate error below $(3\sqrt{2}\,\ell + \sqrt{2})2^{-q}$ prior to scaling by M. □

C.2 The power-of-two Bluestein adapter

This subsection connects the ring convolution of Proposition C.3 to the complex tensor DFT required by our bridge. Lemma C.4 uses Bluestein's quadratic-phase chirp identity together with a negacyclic twist along the last axis to turn $F_{\mathbf{t}}^{+}$ into a single normalized convolution over $\mathcal{R}_r$. Doing so satisfies the middle-transform contract of Section 4, at the price of error factor $\mathrm{E}_{\mathrm{mid}} = \Theta(M\ell)$ and bit widening $\delta_{\mathrm{mid}} = \lg M$.

Lemma C.4 (Power-of-two Bluestein adapter)**.** *Let every t_i $(1 \le i \le d)$ be even and put $a_{\mathbf{j}} = \exp(\pi i \sum_{i=1}^{d} j_i^2/t_i)$. Then a is periodic in every axis and*

$$F_{\mathbf{t}}^{+} z = a \odot \big(\bar{a} * (a \odot z)\big)/R, \tag{57}$$

where $*$ is cyclic tensor convolution over $\prod_{i=1}^{d} \mathbb{Z}/t_i\mathbb{Z}$. The twist $x_d \mapsto \zeta y$, $\zeta = e^{\pi i/r}$, of the last axis maps cyclic convolution along that axis to multiplication in $\mathcal{R}_r = \mathbb{C}[y]/(y^r+1)$, and untwisting recovers the cyclic coefficients. Assume that (q,d,r,K), $t_1,\dots,t_{d-1}$ and $w = q+2$ satisfy the hypotheses of Lemma C.1 and the disk margin $(\sqrt{2}\,M(3\ell+1) + 12d + 16)\,v \le \frac{7}{8}$, where $v = 2^{-q}$ and $M = R/r$. Evaluating (57) via Proposition C.3 satisfies the bridge middle-transform contract (Section 4) with

$$\mathrm{E}_{\mathrm{mid}} = \sqrt{2}\,M(3\ell+1) + 12d + 16, \qquad \delta_{\mathrm{mid}} = \lg M.$$

Proof. Because every t_i is even, $a_{\mathbf{j}+t_i\mathbf{e}_i}/a_{\mathbf{j}} = e^{2\pi i j_i + \pi i t_i} = 1$, and $R^{-1}a_{\mathbf{j}}\overline{a_{\mathbf{j}-\mathbf{k}}}a_{\mathbf{k}} = R^{-1}e^{2\pi i \sum_i j_i k_i/t_i}$ establishes (57). For $k'+l' = j + hr$, we have $\zeta^{k'+l'}(-1)^h = \zeta^j$, which verifies the negacyclic twist along the last axis.

Forming the chirp array a from its d one-dimensional phase tables, each certified to $2v$ inside the unit disk, with componentwise truncations yields $\|\hat{a} - a\|_\infty \le 4dv$ and $\|\hat{a}\|_\infty \le 1$. On bridge-normalized input satisfying $\|z\|_\infty \le 1/8$, the twisted data operand incurs error at most $(4d+6)v$, while the twisted kernel $\bar{a}$ incurs error at most $(4d+4)v$.

By bilinear contraction of $\mathcal{M}_r$ on operands of norms at most $1/8$ and 1, Proposition C.3 shows that the convolution output at width $w + \lg M$ lies within $(\sqrt{2}M(3\ell+1)+8d+10)v \le \frac{7}{8}$ of a vector of norm at most $1/8$. Hence that output remains inside the unit disk, and its redundant sign bits can be stripped cleanly. Untwisting and multiplying by the output chirp add less than $(4d+6)v$, bringing the total error below $\mathrm{E}_{\mathrm{mid}}\, v$. Finally, $7\sqrt{2}Mv \le \frac{7}{8}$ implies $\lg M \le q - 4 \le P$. □

The next remark verifies that the unit-disk margin hypothesis of Lemma C.4 holds automatically under the bridge precision schedule of Section 4.

Remark C.5 (Automatic disk margin under the bridge schedule). Since $M \le R$, $\ell \le k$, $d \le k$ and $R \ge 4$, we have $\sqrt{2}\,M(3\ell+1) + 12d + 16 \le 16R(k+1)$. Under the bridge precision schedule of Theorem 4.7, $16R(k+1)v < \frac{1}{100} \le \frac{7}{8}$, so the disk margin of Lemma C.4 holds automatically along the root family. (In [OAI26, Lemma 21], the chirp is written with the opposite sign $e^{-\pi i \sum_i s_i j_i^2/t_i}$, placing conjugation bars on the outer factors instead of the convolution kernel; for $s_i = 1$ both forms coincide.)

C.3 Bluestein cost table and the quadratic layout bottleneck

This subsection sums the step costs of one Bluestein-backed bridge call and derives the resulting quadratic ceiling on the saving exponent κ. Combining Theorem 4.7, Lemma 4.8, Proposition C.3, and Lemma C.4 gives the term-by-term accounting in Table 9, with total step count

$$\mathsf{T}_{\mathrm{B}}(R,P) = O\big(RP\,[dK + PK^{\tau-1} + \ell d^{\lambda'} + d(1+\ell^{\tau}) + d(\mathsf{m}(P) + \mathsf{m}(P^2)) + \mathsf{m}(rP)]\big). \tag{58}$$

Here $\mathsf{m}(N) = \mathsf{M}(N)/N$ is the per-bit cost of the independent inner multiplier (Section 3.2). Notice that the packed ring products of Lemma C.2 multiply integers of $\Theta(rq) = \Theta(rP)$ bits, where $\lg(rP) = \Theta(P^{1-\varepsilon})$. Thus the $\mathsf{m}(rP)$ term achieves gap ε up to logarithms only when the inner multiplier satisfies $\mathsf{m}(N) = O(\lg N\,(\lg\lg N)^{O(1)})$, such as Schönhage–Strassen multiplication [SS71]. By contrast, our direct Walsh route multiplies only $O(P^2)$-bit integers and needs only $\mathsf{m}(N) = (\lg N)^{O(1)}$.

Table 9: Step-cost breakdown of one bridge call using the Bluestein middle transform. Costs are normalized by the bit volume $V = RP$ under the scaling $d = \Theta(P^{\varepsilon})$, $\ell = \Theta(P^{1-\varepsilon})$, $K = \Theta(P^{\varepsilon c})$, and $q = \Theta(P)$.

operation	normalized cost	gap below P^1
prefix-slot moves and individual rounds	dK	$1 - \varepsilon(1+c)$
synthetic transform layout (chunk exchanges)	$PK^{\tau-1}$	$\varepsilon c(1-\tau)$
simultaneous butterfly rounds	$\ell d^{\lambda'}$	$\varepsilon(1-\lambda')$
axis exposures and restorations	$d(1+\ell^{\tau})$	$(1-\varepsilon)(1-\tau)$
spreading and cyclic solves	$d(\mathsf{m}(P) + \mathsf{m}(P^2))$	$1-\varepsilon$, up to logarithms
packed polynomial products	$\mathsf{m}(rP)$	ε, up to logarithms
chirps, twists, and scalar products	$d\,\mathsf{m}(P)$	$1-\varepsilon$, up to logarithms

Why Bluestein is quadratically capped. On the Bluestein route, executing the layout permutation L via chunk exchanges introduces the exponent gap $g_{\mathrm{chunk}} = \varepsilon c(1-\tau)$. Inside Proposition A.11, the basis-change inequality $\tau(1 + c/\beta) < \lambda < 1$ forces $c < \beta(1-\tau)/\tau$, while the guard inequality

$\varepsilon C_1 < 1$ forces $\varepsilon < 1/C_1$. Multiplying those three factors gives

$$\kappa < g_{\text{chunk}} = \varepsilon c(1-\tau) < \frac{\beta(1-\tau)^2}{C_1\tau} < \frac{(1-\tau)^2}{C_1\tau}, \tag{59}$$

which is *quadratic* in the bit-network deficit $1-\tau$. Even with our certified deficit $1-\tau = 2^{-40}$, this inequality caps κ below $(1-\tau)^2/(20\tau) \approx 2^{-84.3}$ at $C_1 = 20$ and below $(1-\tau)^2/(14\tau) \approx 2^{-83.8}$ at $C_1 = 14$. On the Walsh route, Theorem 5.8 executes both L and $U = (LB)^{-1}$ in $O(V\ell d^{\lambda'})$ steps, eliminating g_{chunk} altogether so the binding gap becomes $g_{\text{lay}} = \varepsilon(1-\lambda') < (1-\sigma)/C_1$, which is *linear* in $1-\sigma$.

C.4 Additional parameter rows and full exponent comparison

This final subsection compares concrete certified parameter tuples across both middle-transform architectures and both sets of network constants. Set $\xi = 2^{-42}$, $\zeta = 2^{-40}$, and $u = 2^{-51}$; in this subsection and in Table 10, ζ and u denote these numbers, not the twist root $\zeta = e^{\pi i/r}$ of Lemma C.4 or the field width u of Lemma A.8. Table 10 collects all seven verified parameter rows across the Walsh and Bluestein routes, starting from the baseline schedule of [OAI26, (38)].

Table 10: Side-by-side comparison of seven certified parameter schedules across the Bluestein and direct Walsh routes. With $\xi = 2^{-42}$, $\zeta = 2^{-40}$, and $u = 2^{-51}$, "printed" rows hold $\beta = \frac{1}{2}$, $\tau = \sigma = 1 - 2^{-50}$, and $C_1 = 20$ fixed as in [OAI26, Prop. 14], whereas "exposed" rows invoke Propositions A.2 and A.11. The bottom row gives Theorem 1 in Section 7, verified in Lean 4.

Route	Layer	C_1	β	ε	c	(τ, σ)	(λ, λ')	Binding gap $g_{\min}$	κ (slack)
Bluestein	printed	20	$\frac{1}{2}$	2^{-75}	2^{-56}	$(1-2^{-50}, 1-2^{-50})$	$(1-2^{-52}, 1-2^{-54})$	$g_{\text{chunk}} = 2^{-181}$	2^{-182} (2^{-182})
Bluestein	printed	20	$\frac{1}{2}$	$\frac{1}{32}$	2^{-52}	$(1-2^{-50}, 1-2^{-50})$	$(1-2^{-51}, 1-2^{-52})$	$g_{\text{chunk}} = 2^{-107}$	2^{-108} (2^{-108})
Bluestein	exposed	20	$\frac{15}{16}$	$\frac{1}{21}$	$\frac{7}{8}\zeta$	$(1-\zeta, 1-\frac{\zeta}{4})$	$(1-\frac{\zeta}{32}, 1-\frac{\zeta}{128})$	$g_{\text{chunk}} = \frac{\zeta^2}{24}$	2^{-85} $(\frac{\zeta^2}{96})$
Bluestein	exposed	14	$\frac{15}{16}$	$\frac{9}{128}$	$\frac{29}{32}\zeta$	$(1-\zeta, 1-\frac{\zeta}{4})$	$(1-\frac{\zeta}{32}, 1-\frac{\zeta}{128})$	$g_{\text{chunk}} = \frac{261}{4096}\zeta^2$	2^{-84} $(5{\cdot}2^{-92})$
Walsh	printed	20	$\frac{1}{2}$	$\frac{1}{21}$	$\frac{u}{4}$	$(1-2u, 1-2u)$	$(1-\frac{129}{128}u, 1-\frac{63}{64}u)$	$g_{\text{lay}} = \frac{3u}{64}$	2^{-56} (2^{-57})
Walsh	exposed	20	$\frac{1}{8}$	$\frac{1}{21}$	$\frac{\xi}{4}$	$(1-4\xi, 1-\xi)$	$(1-\frac{7}{8}\xi, 1-\frac{3}{4}\xi)$	$g_{\text{lay}} = \frac{\xi}{28}$	2^{-47} $(\frac{\xi}{224})$
Walsh	exposed	14	$\frac{1}{64}$	$\frac{1}{15}$	$\frac{\xi}{256}$	$(1-4\xi, 1-\xi)$	$(1-\frac{63}{64}\xi, 1-\frac{31}{32}\xi)$	$g_{\text{lay}} = \frac{31\xi}{480}$	2^{-46} $(\frac{\xi}{480})$

Verification of the secondary rows. The primary $\kappa = 2^{-46}$ row is checked in Section 7.5. Here we record the exact rational margins for the remaining six rows:

- *Walsh with printed structural constants, $\kappa = 2^{-56}$.* Fix $\beta = \frac{1}{2}$, $\tau = \sigma = 1-2u$, and $C_1 = 20$. Then $\sigma + \beta(1-\sigma) = 1-u$ and $\tau(1+c/\beta) = (1-2u)(1+u/2) = 1 - \frac{3}{2}u - u^2$. The resulting layer margins are $\lambda - \tau = \frac{127}{128}u$, $\lambda - \tau(1+c/\beta) = \frac{63}{128}u + u^2$, $\lambda' - (1-u) = \frac{u}{64}$, $\lambda' - \lambda = \frac{3u}{128}$, and $1 - \varepsilon C_1 = \frac{1}{21}$. The binding gap is $g_{\text{lay}} = \frac{3u}{64} = 3 \cdot 2^{-57}$ against $\kappa = 2^{-56}$, while the axis gap of (45) is $g_{\text{axis}} = \frac{40}{21}u$. That 2^{-56} is the largest exponent of the form 2^{-j} admitted under these constants is shown in Remark 7.3.
- *Walsh with $C_1 = 20$, $\kappa = 2^{-47}$.* Using $\tau = 1-4\xi$ and $\sigma = 1-\xi$ from Proposition A.2, we have $\tau(1+c/\beta) = (1-4\xi)(1+2\xi) = 1 - 2\xi - 8\xi^2$. That gives positive margins $\lambda - \sigma = \frac{\xi}{8}$, $\lambda - \tau(1+c/\beta) = \frac{9}{8}\xi + 8\xi^2$, $\lambda' - \lambda = \frac{\xi}{8}$ with $\sigma + \beta(1-\sigma) = \lambda$, and $1 - \varepsilon C_1 = \frac{1}{21}$, yielding binding gap $g_{\text{lay}} = \frac{\xi}{28}$ above $\kappa = \frac{\xi}{32} = 2^{-47}$.

- *Bluestein rows,* $\kappa = 2^{-182}, 2^{-108}, 2^{-85}, 2^{-84}$. At the schedule of [OAI26, (38)], our bridge paired with the Bluestein middle transform reproduces $\kappa = 2^{-182}$ with $g_{\text{chunk}} = 2^{-181}$, confirming that replacing the CRT front end alone leaves κ unchanged. Retuning (ε, c) under the printed structural constants raises the bound to $\kappa = 2^{-108}$ with $g_{\text{chunk}} = 2^{-107}$, using the $O(d)$ axis exposures of Lemma 4.8 in place of the $O(d^2)$ adjacent swaps of [OAI26, §9].

  Exposing (β, τ, σ) via Propositions A.2 and A.11 yields $\kappa = 2^{-85}$ at $C_1 = 20$, since $\tau(1 + c/\beta) = (1 - \zeta)(1 + \frac{14}{15}\zeta) < 1 - \frac{\zeta}{15} < \lambda$ and $g_{\text{chunk}} = \frac{\zeta^2}{24}$. Lowering the guard exponent to $C_1 = 14$ gives $\kappa = 2^{-84}$, with $\lambda - \tau(1 + c/\beta) = \frac{\zeta}{480} + \frac{29}{30}\zeta^2 > 0$, $1 - 14\varepsilon = \frac{1}{64}$, and $g_{\text{chunk}} = \frac{261}{4096}\zeta^2$.

At equal guard exponents, removing the chunk-layout bottleneck through Walsh-layer positional permutations replaces the quadratic ceiling (59) in $1-\tau$ by a binding gap linear in $1-\sigma$. Consequently, κ rises from 2^{-85} to 2^{-47} at $C_1 = 20$, from 2^{-84} to 2^{-46} at $C_1 = 14$, and from 2^{-108} to 2^{-56} under the printed structural constants.

D Statements and notation from [OAI26]

This appendix bridges the notation and black-box statements of [OAI26] to the conventions of the present paper. Throughout, all citations to [OAI26] refer to its version of 23 September 2026. Several single-letter symbols in [OAI26] collide with standard parameters of our fractional-shift bridge. In particular, both papers use R and P, with different meanings. We first give the complete symbol-renaming dictionary so formulas can be compared across the two papers without ambiguity. Next, we restate the four elementary stream and network results imported as black boxes in Section 3.4. Those four statements constitute Facts 3.3 and 3.4. Finally, we quote the printed butterfly-layer bound of [OAI26, Prop. 14] alongside our parameter-exposed Proposition A.11. A reader who accepts Facts 3.3–3.4 from Section 3.4 may use this appendix solely as a lookup reference.

Notation map. Table 11 lists every symbol of [OAI26] that is renamed in this paper. No mathematical definition changes under this renaming. Only letter choices change so bridge parameters and tape-array parameters stay distinct across Sections 4–7 and Appendix A. Tape procedures receive headers that concatenate positive integer parameters. Those headers use the self-delimiting binary code of [OAI26, (1)]:

$$b_v = \lfloor \log_2 v \rfloor + 1, \qquad \Gamma(v) = 0^{\,b_v - 1}\,\mathrm{bin}_{b_v}(v), \qquad |\Gamma(v)| = 2b_v - 1 \qquad (v \geq 1). \tag{60}$$

Here $\mathrm{bin}_{b_v}(v)$ writes the positive integer v in standard binary of exact bit length b_v. Its leading bit is always 1. Thus the prefix of $b_v - 1$ zeros uniquely determines where $\mathrm{bin}_{b_v}(v)$ begins and ends. Any concatenated header $\Gamma(v_1)\Gamma(v_2)\cdots\Gamma(v_m)$ can therefore be parsed left to right in linear time without extra delimiters.

D.1 Machine model and elementary stream lemmas

This subsection restates the tape-array conventions and the two linear-time stream lemmas behind Fact 3.3. Following [OAI26, §2], every algorithm runs on a deterministic multitape Turing machine. Each machine has a fixed finite symbol alphabet and a fixed finite number of one-dimensional tapes. Multi-index arrays on a product box $[a_1] \times \cdots \times [a_t]$ sit sequentially on a tape in lexicographic order. In that order, the first coordinate is most significant. Array addresses are implicit in the physical position of each entry rather than stored in the record. We write V for the logical bit volume of

Table 11: Dictionary translating notation between [OAI26] and the present paper. Each row gives the symbol in [OAI26], our replacement symbol, and the role of both quantities.

[OAI26]	This paper	Meaning
P (§2, Lemmas 3, 12, Prop. 14)	A	collapsed prefix address range (P is our bridge working precision)
q (§2, Lemmas 3–4; §4, Lemmas 10–12)	q_{rad}	fixed integer or odd prime radix (q is our layer grid precision)
p (§§5–9, Prop. 14)	q	dyadic grid precision of layers and middle transforms ($q = P + 2$ in §4)
R (§2, Lemma 3; §5.2, Lemma 13)	R_{row}, R_{rec}	row-field range; record bit width ($R = t_1 \cdots t_d$ is our DFT length)
E, B, ν, k (§5.4)	E_{net}, B_{net}, ν_{net}, $j_{\max}$	complex-network guard constants, guard degree, and recursion depth ($k = \lg R$)
T, $L = \log_2 T$ (§§6–9)	R, $k = \lg R$	total power-of-two transform box size and its binary logarithm
ϵ, log	ε, ln	dimension exponent; natural logarithm ($\lg = \log_2$)

the active array. When an address permutation f acts on an array A, we use the push-forward convention $(fA)(f(x)) = A(x)$. That rule moves the entry originally at address x to address $f(x)$.

The next two propositions reproduce Lemmas 3 and 4 of [OAI26, §2.1]. Together they make up Fact 3.3 in Section 3.4.

Proposition D.1 covers four basic digit-level and row-stream rearrangements. Each runs in linear bit time $O(V)$ on a fixed number of tapes. We use these operations as the base case for every multi-stream split, merge, and digit move in Appendix A. Their $O(V)$ cost already includes counter setup, head resets, and workspace cleanup.

Proposition D.1 ([OAI26, Lemma 3, §2.1])**.** *Fix a radix $q_{\mathrm{rad}} \geq 2$. On an array of volume V, each of the following takes $O(V)$ steps on a fixed number of one-dimensional tapes (including shape preparation, counter initialization/resets, and workspace cleanup): (1) any fixed permutation of the pair of digit values at two specified radix-q_{rad} positions; (2) moving one specified radix-q_{rad} digit to another position, preserving the order of intervening positions; (3) splitting on a specified radix-q_{rad} digit, applying a fixed pointwise map to the aligned streams, and merging; and (4) within each prefix of an array on $[A] \times [R_{\mathrm{row}}] \times [B]$, splitting whole rows cyclically into a fixed number W of streams and, when $W \mid R_{\mathrm{row}}$, merging them in any fixed order of the row classes.*

Proposition D.2 extends those linear-time primitives to controlled modular shifts, rational unit multiplications modulo q_{rad}^b, and fixed-width payload arithmetic. We need it for the triangular matrix shears of Lemma A.6 and the eight-shift packed addition of Lemma A.9. All control fields must precede the target field and occupy $O(b)$ digits. Scanning a fiber of size q_{rad}^b then absorbs the polynomial-in-b offset work.

Proposition D.2 ([OAI26, Lemma 4, §2.1])**.** *Fix a prime q_{rad}, a fixed number of control fields, and finitely many rational coefficients with denominators prime to q_{rad} (and numerators prime to q_{rad} when used as units). Put $Q = q_{\mathrm{rad}}^b$ ($b \geq 1$), and let a target field $y \in [Q]$ follow its control fields, each control used in an offset having $O(b)$ digits, with arbitrary spectator fields and suffix records. An update $y \mapsto y + a(\mathit{controls}) \pmod{Q}$ costs $O(V)$ whenever the offset is computable in $b^{O(1)}$ time from the controls (in particular for fixed rational linear combinations modulo Q), and so does multiplication of y by any of the specified rational units or its inverse. Fixed-width signed*

fixed-point addition, subtraction, sign change, and shifts by fixed powers of two (with sufficient guard bits), as well as intra-record bit reversal, also have $O(V)$ bit cost.

D.2 Finite-network certificates and printed butterfly layer

This subsection summarizes the $h = 100$ finite linear networks behind Fact 3.4. It closes with the printed butterfly-layer statement of [OAI26, Prop. 14]. At ground-set size $h = 100$, one sets $v = \binom{h}{3} = 161\,700$, $N = v^3$, and $m = h^3 = 10^6$. On these parameters, [OAI26, §3.1] builds two fixed three-stage linear circuits. Each circuit acts on two primary data banks X_a, Y_a indexed by triples $a \in \binom{[h]}{3}^3$, plus per-invocation scratch wires.

The first circuit is a bit network over $\mathbb{F}_2$ on W_b wires. Holding two coordinates of a fixed per stage, its stages connect pairs of wires whose active coordinate sets S, T, three-element subsets of $[h]$ as in Fact 3.4, satisfy the intersection rule $|S \cap T| = 1$. The second circuit is a complex network over $\mathbb{Z}[i, 1/2]$ on W_c wires. Its stages connect pairs of wires with distinct active coordinate sets $S \neq T$ satisfying $|S \cap T|$ even. By [OAI26, Lemmas 5–6, §§3.1–3.3], a three-stage forward-inverse-forward schedule maps the data banks via $(X_a, Y_a) \mapsto (-Y_a, X_a)$. That same schedule restores every scratch wire to its initial value regardless of its starting contents.

To bound the edge cost, [OAI26, §§3.1–3.3] labels every wire with a nondegenerate subspace of $\mathcal{F} = F^{\otimes 3}$, where $\dim \mathcal{F} = m$. Adjacent subspace labels along circuit edges are nested. Only decreasing edges on central wires lose dimension, each dropping by h. Their total losses L_b and L_c, which enter the rank sums s_b and s_c in Proposition D.3 below, have the exact ratios $L_b/N = 100/539$ and $L_c/N = 101/539$. For the bit network, the base space is $F = \mathbb{Q}^h$ under the bilinear form $x^{\mathsf{T}}(I_h - J_h/9)y$. Here I_h is the $h \times h$ identity matrix and J_h is the all-ones matrix. For the complex network, the base space is $F = \mathbb{F}_2^h$ under the standard dot product.

Proposition D.3 packages the resulting rank and endpoint certificates of [OAI26, Props. 7–8]. In our main text, this statement appears as Fact 3.4. Part (a) certifies that the bit network carries every input role to its output role with net frame matrix I_m, so it implements the address shear $(H, D) \mapsto (H + D, D)$ of Fact 3.4, while its edge ranks sum to $s_b < W_b m$. Part (b) certifies that the complex network applies $(C^{\otimes m})^{\otimes f}$ across all roles using $s_c < W_c m$ translation kernels in total. Those two strict inequalities $s_b < W_b m$ and $s_c < W_c m$ are the sole source of the sub-$n \log n$ time saving.

Proposition D.3 ([OAI26, Props. 7–8, §§3.4–3.5])**.** *(a)* Bit network ([OAI26, Prop. 7])*. Assigning orthogonal projections P_U at gates/sinks, $-P_{U_a}$ at sources X_a and 0 at other sources, the bit network uses only pointwise XOR gates on W_b roles, induces the role permutation ρ_b exchanging X_a and Y_a while fixing scratch roles, satisfies $M_{\mathrm{sink}(\rho_b(w))} - M_{\mathrm{source}(w)} = I_m$ for every input role w, and has total rational edge-rank sum $s_b = W_b m - N + 2L_b < W_b m$ with the exact integers of Definition A.1.*

(b) Complex network ([OAI26, Prop. 8])*. With $H_{\mathrm{u}} = \frac{1}{\sqrt{2}}\left(\begin{smallmatrix} 1 & 1 \\ 1 & -1 \end{smallmatrix}\right)$, $C = H_{\mathrm{u}} \operatorname{diag}(1, i) H_{\mathrm{u}} = aI + bX$ ($a = \frac{1+i}{2}$, $b = \frac{1-i}{2}$), $q_U(x) = \mathrm{wt}(P_U x) \bmod 4$ over $\mathbb{F}_2$ and $C_U = H_{\mathrm{u}}^{\otimes m} \operatorname{diag}_x(i^{q_U(x)}) H_{\mathrm{u}}^{\otimes m}$, each edge frame difference $C_V C_U^{-1}$ is a product of one translation kernel $aI + bX_v$ or its inverse per vector in an orthonormal basis of the edge residual, with total basis length $s_c = W_c m - 2N + 2L_c < W_c m$ as in Definition A.1. After source/sink diagonal sign corrections ($Z_{a,f}$ before X_a and $i^{27f} Z_{a,f}$ after Y_a on f columns) and undoing the signed bank exchange, telescoping yields $(C^{\otimes m})^{\otimes f}$ on every role, including scratch roles with arbitrary initial values.*

Once $C^{\otimes D}$ is available across D selected bits, a diagonal fourth-root phase conjugation converts it into the normalized butterfly layer $H_0^{\otimes D}$. Specifically, [OAI26, (18), §5.1] uses the identity (61),

already used as (50) in the proof of Lemma A.10:

$$H_0^{\otimes D} = b^D\, \mathcal{S}_D\, C^{\otimes D}\, \mathcal{S}_D, \qquad (\mathcal{S}_D z)(x) = i^{x_1+\cdots+x_D} z(x), \qquad b^D = (-i/2)^{\lfloor D/2 \rfloor} b^{D \bmod 2}. \tag{61}$$

Here $\mathcal{S}_D$ multiplies each record by an address-dependent power of i, and b^D is a global dyadic Gaussian scalar.

Finally, Proposition D.4 records the simultaneous butterfly-layer theorem of [OAI26, Prop. 14] with its printed structural constants. Our main proof does not invoke Proposition D.4; instead, Appendix A proves the parameter-exposed generalization in Proposition A.11. Recording the printed version here lets Section 7 and Appendix C separate the gain of the Walsh route itself, $\kappa = 2^{-56}$ at the printed constants, from the later sharpening of the network constants.

Proposition D.4 ([OAI26, Prop. 14, §5.4], printed constants)**.** *Proposition A.11 holds as stated in [OAI26, Prop. 14] at the fixed structural constants* $\beta = \frac{1}{2}$, $\tau = \sigma = 1 - 2^{-50}$ *and* $C_1 = 20$ *(with* q *written* p *and* A *written* P *there), for any fixed positive rationals* $c, \varepsilon, \lambda, \lambda'$ *satisfying* (51)–(52) *and* $\varepsilon C_1 < 1$.

E Reference implementation

This appendix documents the exact-arithmetic Python 3.13 reference implementation that accompanies the written proofs and the Lean 4 formalization of Section 8. We built this package so that every data layout, rounding rule, error budget, and finite-network certificate in the paper can be inspected and tested on concrete inputs. Its purpose is clarity and bit-exact verification at small sizes; it is a RAM program, not a fast multiplier, does not simulate multitape Turing-machine head movements, and measures or claims no running time. Inside the engines, some address permutations are executed directly as index maps, and built-in integer multiplication handles inner products unless the internal multiplier-hierarchy mode is enabled. Table 12 summarizes each component, whether it is executed or certified, how it is tested, its largest configuration, and its measured versus proved error. A reader interested only in the mathematical proofs may skip this appendix.

Exact-arithmetic design and implemented components. No floating-point arithmetic occurs on any data path of the multiplier or network certifier; floating-point numbers appear only in test-suite reference oracles, an optional table cross-check mode, and report formatting. Every stored and intermediate number is represented in exact integer or rational arithmetic, and every rounding step is an explicit integer operation. On the dyadic grids of the fractional-shift bridge in Section 4, values round to nearest. On the disk-valued records of the butterfly layers and middle transforms in Sections 5–6 and Appendix A, values truncate componentwise toward zero.

All transcendental lookup tables, namely sinc-Gaussian spreading taps, cyclic deconvolution coefficients, and roots of unity, are generated from certified rational interval enclosures. Each enclosure is recomputed at doubling precision until it determines the rounded value or reaches a precision cap (Ziv's precision refinement), guaranteeing faithful rounding onto the target dyadic grid.

The package implements every algorithmic step in four modular parts:

- The d-dimensional fractional-shift bridge with negative half-bin modulation, overlap-add spreading in descending axis order $d, \ldots, 2$, chronological cyclic deconvolution $B_2^{-1}, \ldots, B_d^{-1}$, and two-call root packing and carry recovery.

- The direct complex tensor FFT of Section 6, including the three-layer selective H_0 gadget, slice-batched CNOT flips, two-involution positional permutations from Section 5, and combined per-record twiddles. For comparison, the package also implements the Bluestein middle transform and normalized $\mathcal{R}_r$-convolution of Appendix C.
- The recursive power-width and arbitrary-width chunk interchanges, the packed selected-bit addition with its eight-shift program and deterministic radix-sort repair on the exceptional set, and the recursive $C^{\otimes e}$ phase-frame engine of Appendix A.
- A non-recursive inner-multiplier hierarchy that routes every packed and word-scale integer product above 64 bits through the internal multiplier rather than Python's built-in big-integer multiplication.

Test-scale execution and asymptotic caveat. As discussed in Section 9, the sub-$n \log n$ schedule activates only at astronomically large input sizes. Reaching dimension $d \geq 2$ at $\varepsilon = 1/15$ already requires a transform length $R = 2^k$ with $k \geq 2^{15}$, and the guard allocation $\Delta = \lceil C_0 d^{14} \rceil$ involves $C_0 \approx 2.27 \times 10^{139}$. In addition, the $h = 100$ triple networks have $m = 10^6$ and $W_c \approx 1.87 \times 10^{21}$ roles. Consequently, at every feasible input size, the parameter schedule of Theorem 1, when no test override is in force, falls back to a two-dimensional $O(n \log n)$ fractional-shift multiplier with negative half-bin carrier. That fallback is the $d = 2$ case of Section 4; see [MS] for the real-kernel two-dimensional variant.

To test the multi-axis pipeline end to end, test schedules override the asymptotic thresholds and force small shapes with dimensions $d \in \{2, \ldots, 10\}$, transform lengths up to $R = 4096$, and chunk widths $K \in \{1, 2\}$. Those tests run the recursive interchange and butterfly-layer engines on small substitute network fixtures with $W = 3$ and $m \in \{2, 3\}$ in place of the $h = 100$ networks, using guard widths compiled from the active fixture's depth recurrence. These fixtures satisfy every structural, scratch-restoration, frame, and endpoint identity of Definition A.1, exercising all recursive, basis-change, and exceptional-repair code paths, although they have $s > Wm$ and hence no rank saving. The $h = 100$ networks themselves are certified symbolically and never executed; requesting their execution raises a runtime error.

What the test suites verify. Three test suites exercise the implementation against independent references and against our analytic bounds:

(1) *Bit-exact products and component equivalence.* Across 2,067 tests, 1,832 pass and one is deferred in the fast suite while all 234 pass in the slow suite, checking every returned integer product bit for bit against Python's built-in multiplication. The same bit-exact check passes on full-cascade runs with no stub on 70- and 96-bit inputs under a forced $d = 2$ test schedule with substitute network fixtures for the Walsh and Bluestein transforms. In those runs, every inner product above 64 bits goes through the internal multiplier hierarchy.

Component tests also verify bit-identical agreement between the network-driven interchange and butterfly-layer engines and direct dense reference operators. At runtime, assertions check exact zero-bit halvings, guarded-width bounds, unit-disk containment, scratch restoration, and the hole-key bijection in the sorting repair.

(2) *Measured errors against proved bounds.* Across 71 transform configurations spanning dimensions $d \in \{2, \ldots, 5\}$ and lengths up to $R = 1024$, the measured final rounded error never exceeds $2^{-2.51} \cdot 2^{-p}$. That value is just below the final rounding term $(\sqrt{2}/8) \cdot 2^{-p} = 2^{-2.5} \cdot 2^{-p}$ of Theorem 4.7; the pre-rounding errors reported next are many bits smaller. On the 15 non-leaf

runs among the 17 two-dimensional fallback configurations, with the other 2 being base-FFT leaves, pre-rounding errors lie at least 12.9 bits below the nominal node certificate. That certificate is called nominal because the fallback rounds negacyclic root-of-unity twist factors and leaf twiddles to nearest rather than into the unit disk.

On the d-dimensional bridge with a direct reference middle transform across 24 configurations, pre-rounding errors lie at least 21.5 bits below our proved analytic budget. On the Walsh and Bluestein middle-transform routes across 30 configurations, pre-rounding errors lie at least 23.9 bits below the proved budget. Direct tests of the Walsh middle transform against multiprecision reference DFTs also confirm its $12\ell \cdot 2^{-q}$ error bound from Theorem 6.5.

(3) *Exact symbolic network certificates.* For the $h = 100$ triple networks of [OAI26, §3], a symbolic certifier exploits the $(S_h)^3$ permutation symmetry of the three ground sets to inspect one representative role per orbit. On each representative edge, it decomposes $F^{\otimes 3}$ into mutually orthogonal tensor-product subspaces, called *cells* in [Code26], and computes residuals and ranks in exact arithmetic without materializing $10^6 \times 10^6$ matrices. For each network, it verifies the closed-form role counts W_b, W_c, edge-rank sums s_b, s_c, endpoint identities, relative deficits η_b, η_c, and guard constants $\nu_{\text{net}} = 11$ and the 140-digit integer C_0. An exact rational interval series for $\ln m$ computes a width-2^{-80} enclosure that rounds outward to $13.815510557964 \le \ln 10^6 \le 13.815510557965$, matching Table 8 and Remark A.3 and confirming $\tau = 1 - 2^{-40}$ and $\sigma = 1 - 2^{-42}$ in Proposition A.2.

Separately, we examined the bit network's odd prime radix q_{rad}, whose existence follows by finite avoidance in Lemma A.6. A symbolic check first verifies that every $h^3 \times h^3$ edge matrix rewrites into a finite grammar of single-factor $h \times h$ projection blocks. Direct Lemma A.6 elimination on all 96,172 resulting $h \times h$ and $2h \times 2h$ building-block matrices at $h \in \{7, 8, 10, 11, 12\}$ then finds only the primes $\{2, 3\}$ in denominators and diagonal numerators; see [Code26] for details. This gives computational evidence that $q_{\text{rad}} = 5$ suffices, although no theorem in this paper depends on that specific prime.

Availability. The exact rational parameter-verification script `tools/h5_rows_check.py` ships inside the LaTeX source bundle of this paper. The artifact [Code26] provides the complete Python 3.13 exact-arithmetic reference package described above. Together with the Lean 4 formalization [Lean26] of Section 8, both software artifacts are available in our github repository.

Table 12: Executability at a glance: each component in the Python 3.13 reference package [Code26], whether it is executed on inputs or certified symbolically, how it is tested, its largest configuration, and its measured versus proved error. Module paths are under `python/subnlogn/` unless prefixed by `scripts/`; test files are under `python/tests/`.

Component	Implemented in (module)	Executed / certified	How tested	Largest configuration	Measured vs. proved error
Fractional-shift bridge	`bridge/` (`bridge_dft.py`, `spread.py`, `deconv.py`, `geometry.py`, `movement.py`, `middle.py`), `kronecker.py`, `solver.py`, `tables.py`, `rigorous.py`	Executed (forced test schedules; direct middle or substitute $W = 3$, $m \in \{2, 3\}$ fixtures)	Stagewise and end-to-end exact-oracle tests (`test_bridge_*.py`, `test_stage_errors.py`, `test_m6_slow.py`, `test_m7_end_to_end.py`, `test_walsh_e2e.py`); `scripts/error_report.py` (54 rows)	Shapes up to $d = 10$, $R = 2048$ ($K = 1$) and $d = 4$, $R = 4096$ ($K = 2$); 10^5-bit products ($R_0 = 16{,}384$, direct/Bluestein) and 10^4-bit Walsh ($R_0 = 2048$); report up to $d = 5$, $R = 1024$	Final error $\le 2^{-2.51} \cdot 2^{-p}$ vs. $2^{-2.5} \cdot 2^{-p}$ rounding term ($< 0.177021 \cdot 2^{-p} \max(1, \|x\|_\infty)$ contract); pre-rounding ≥ 21.5 bits below bound (24 direct rows) and ≥ 23.9 bits (30 Walsh/Bluestein rows)
Walsh gadgets	`walsh/gadget.py` (`selective_walsh`, `cnot_batch`), `walsh/positional.py` (`make_slices`, `two_involutions`, `execute_positional_map`, `compile_layout`)	Executed (base layer or substitute `C1` fixture, $W = 3$, $m = 3$)	Exact $\mathbb{Q}(i)$ matrix checks for H_T and $\mathrm{CNOT}^{\otimes s}$, base vs. `C1` engine checks, Hypothesis bijections (`test_walsh_gadget.py`, `test_walsh_positional.py`)	Standalone $D = 9$ chunks ($2^9 = 512$ records on `C1`), `make_slices` up to $m_0 = 11$, Hypothesis up to $m_0 = 10$; in `tensor_fft_walsh` up to $m_0 = 9$ (512 records: $R = 16{,}384$ at $r = 32$, $R = 4096$ at $d = 4$, $R = 2048$ at $d = 10$)	Bit-exact (0 error): records lift to $q_* = q + 8d$; runtime checks assert every Q_{q_*} and lowering to q discards only zero bits
Direct tensor FFT	`walsh/fft.py` (`tensor_fft_walsh`, `last_axis_fft`, `twiddle_index`, `walsh_middle_dft`), `bridge/middle.py`	Executed (base layer or substitute `C1/B1` fixtures, $W = 3$, $m \in \{2, 3\}$)	Bit-identical vs. dense oracle (base and `C1/B1`), exact $\mathbb{Z}[i, 1/2]$ mode (`truncate=False`), three-way `mpmath` check (`test_walsh_fft.py`, `test_walsh_e2e.py`), `scripts/error_report.py` (8 rows)	Standalone $\mathbf{t} = (32, 16, 32)$, $R = 16{,}384$ (base) and $d = 10$, $R = 2048$ (network); bridge up to $R = 4096$ ($d = 4$, $K = 2$) and 10^4-bit product ($R_0 = 2048$); report up to $d = 4$, $R = 256$	Exact mode 0 error; standalone error $< 2.20 \cdot 2^{-q} < 12\ell \cdot 2^{-q}$ ($< 5\%$ of Theorem 6.5 bound at $R = 2048$, $\ell = 4$, and $< 8\%$ across all tested shapes); bridge pre-rounding margin ≥ 23.9 bits
Chunk interchanges and butterfly-layer engine	`stream.py`, `interchange.py`, `m6/` (`lemma13.py`, `cnet.py`, `prop14.py`, `policy.py`, `engine.py`), `net/` (`fixtures.py`, `triple_dense.py`, `primes.py`)	Executed ($W = 3$, $m \in \{2, 3\}$ fixtures; $h = 7$ scalar wires; $h \le 12$ block matrices)	Engine vs. dense operators on $W = 3$, $m \in \{2, 3\}$ fixtures (`test_interchange.py`, `test_lemma13.py`, `test_cnet.py`, `test_m6_*.py`); $h = 7$ scalar gate runs (`test_net_triple.py`); $h \in \{7, 8, 10, 11, 12\}$ prime-support checks (`test_net_primes.py`, `scripts/prime_support_bruteforce.py`)	Engine on `C1/B1`: $d = 20$, $D = 19$ (524,288 records) and 262,144-record Lemma A.8/A.9 runs; triple: $h = 7$ ($m = 343$, 529,200 gates on $W_b = 2{,}426{,}725$, $W_c = 2{,}173{,}150$ wires), $10^3 \times 10^3$ edges ($h = 10$), 96,172 blocks ($h \le 12$)	Interchanges, selected-bit adds, guarded $C^{\otimes e}$, $h = 7$ scalar runs, and prime-support factorizations exact (0 error; primes $\{2, 3\}$); `simultaneous_H0_network` bit-identical to direct H_0 ($< \sqrt{2} \cdot 2^{-q}$ disk truncation)
$h = 100$ network certificates	`net/` (`triple.py`, `symbolic.py`, `certify.py`, `taubound.py`, `primes.py`), `scripts/certify_report.py`	Certified symbolically; not executed	$(S_h)^3$ orbit-cell rank/endpoint certifier, dense checks at $h \in \{7, 8\}$, tampered-network negative tests, `RealNetwork` `NotExecutable` guards (`test_net_certify.py`, `test_theorem_networks.py`, `scripts/certify_report.py`)	At $h = 100$, $m = 10^6$: $W_b \approx 1.77 \times 10^{20}$, $W_c \approx 1.87 \times 10^{21}$, $\nu_{\mathrm{net}} = 11$ ($C_1 = \nu_{\mathrm{net}} + 3 = 14$), 140-digit $C_0 \approx 2.27 \times 10^{139}$	Exact integer/rational (0 error) for $W, s, \eta, \nu_{\mathrm{net}}, C_0$; width-$< 2^{-80}$ interval [13.815510557964, 13.815510557965] for $\ln 10^6$ certifying $\tau = 1 - 2^{-40}$ (margin +2.69) and $\sigma = 1 - 2^{-42}$ (margin +2.31)
Two-dimensional fallback multiplier	`fastdft2d.py`, `synthetic.py`, `routing.py`, `basefft.py`, `multiply.py`	Executed (unforced default at all feasible sizes; no network)	Bit-exact products, tape vs. index routing bit-identity, stage/node certificates (`test_fastdft2d.py`, `test_routing.py`, `test_multiply.py`, `scripts/error_report.py` 17 rows)	$n = 10^6$-bit product under unforced `faithful_schedule()` ($R_0 = 131{,}072$, depth 1); 1.4×10^5-bit tape vs. index; 10^5-bit forced depth ≥ 3; 17 report rows up to $R = 1024$ (depth ≤ 4)	All integer products bit-exact; across 17 report rows, final error $\le 2^{-2.51} \cdot 2^{-p}$, and across all 15 non-leaf rows pre-rounding error ≥ 12.9 bits below nominal node certificate
End-to-end cascade (no stub)	`hierarchy.py`, `context.py`, `multiply.py`, `scripts/full_cascade_demo.py`, `scripts/hierarchy_coverage.py`	Executed (forced $d = 2$ with substitute `C1/B1` fixtures; or 2D, no network)	Unstubbed cascades (`scripts/full_cascade_demo.py`, `test_products.py`); stage-isolated shapes with real `BasicMul` verified per size pair (`scripts/hierarchy_coverage.py`, `test_bigint_sites.py`); AST guards (`test_optrace_guard.py`, `test_arith_lint.py`)	Unstubbed bridge: 70-bit Walsh ($d = 2$, $R_0 = 32$: 1,280 `Multiply1`, 5,277,162 `BasicMul`) and 96-bit Bluestein (1,682 `Multiply1`, 11,399,742 `BasicMul`); unstubbed 2D: 600-bit faithful, 24-bit deep; stage-isolated up to $R_0 = 4096$	Bit-for-bit exact product AB (0 error); counters and AST guards confirm zero unrouted > 64-bit data-path products above `BasicMul` and zero recursive re-entry into the bridge